Vorlesungsnotizen zu Angewandte Mathematik für das Web

Andreas Beham

Wintersemester 2026/27

Deutsche Übersetzung, vollständig KI-generiert.
Grundlage ist die englische Fassung dieser Vorlesungsnotizen.
Die Übersetzung wurde nicht fachlich durch eine Person geprüft.

Inhaltsverzeichnis

1 Grundlagen der Graphentheorie **6**
- 1.1 Ungerichtete Graphen . . . 6
- 1.2 Gerichtete Graphen . . . 9
- 1.3 Isomorphie von Graphen . . . 10
- 1.4 Graphfamilien . . . 11
- 1.5 Teilgraphen . . . 12
- 1.6 Gewichtete Graphen . . . 13

2 Wanderungen und Zusammenhang **15**
- 2.1 Zusammenhangskomponenten, Brücken, Abstand und Durchmesser . . . 16
  - 2.1.1 Tarjans Algorithmus zur Bestimmung starker Zusammenhangskomponenten 19
  - 2.1.2 Topologische Sortierung . . . 20

3 Hamiltonsche und eulersche Graphen **22**
- 3.1 Hamiltonsche Graphen . . . 22
  - 3.1.1 Problem des Handlungsreisenden . . . 23
- 3.2 Eulersche Graphen . . . 25
  - 3.2.1 Fleurys Algorithmus . . . 27
  - 3.2.2 Hierholzers Algorithmus . . . 28
  - 3.2.3 Briefträgerproblem . . . 28

4 Transitive Hülle **30**
- 4.1 Transitive Hülle mittels Matrixmultiplikation . . . 30
- 4.2 Warshall-Algorithmus . . . 33

5 Kürzeste Pfade **34**
- 5.1 Kürzeste Pfade mittels Breitensuche . . . 35
- 5.2 Floyd-Algorithmus . . . 37
- 5.3 Bellman-Moore-Ford-Algorithmus . . . 38
- 5.4 Dijkstras Algorithmus . . . 41
  - 5.4.1 Korrektheitsbeweis . . . 43
- 5.5 A*-Algorithmus . . . 44
- 5.6 Bidirektionale Suche . . . 47
- 5.7 A* mit iterativer Vertiefung . . . 50

6 Minimale Spannbäume **52**
- 6.1 Bäume und Wurzelbäume . . . 52
- 6.2 Algorithmus von Prim . . . 54
- 6.3 Algorithmus von Kruskal . . . 57
- 6.4 Warum die gierigen Entscheidungen funktionieren: ein Austauschargument . . . 57
- 6.5 Chu-Liu/Edmonds-Algorithmus . . . 58

7 Suche mit Gegenspielern und der Minimax-Algorithmus **62**
- 7.1 Problemstellung . . . 62
- 7.2 Ein kleines Gegenbeispiel zum gierigen Spielen . . . 63
- 7.3 Algorithmus . . . 63
- 7.4 Alpha-Beta-Beschneidung . . . 64
- 7.5 Tiefenbegrenzte Suche . . . 66
- 7.6 Expectimax . . . 67

8 PageRank und Zentralität **68**
8.1 Von Webseiten zu einem mathematischen Modell . . . 68
8.2 Empfehlungen und der zufällige Surfer . . . 69
8.3 Existenz, Eindeutigkeit und Konvergenz . . . 70
8.4 Seiten ohne ausgehende Links, Rangfallen und Teleportation . . . 71
8.5 PageRank mit der Potenzmethode berechnen . . . 72
8.5.1 Der Zusammenhang mit der transitiven Hülle . . . 73
8.5.2 Die Berechnung an unserem Webgraphen . . . 74
8.6 Weitere Bedeutungen von Zentralität . . . 75
8.6.1 Gradzentralität . . . 75
8.6.2 Nähezentralität . . . 75
8.6.3 Betweenness-Zentralität . . . 76
8.6.4 Eigenvektorzentralität . . . 77

9 Flussnetzwerke **78**
9.1 Problem des maximalen Flusses . . . 78
9.1.1 Vom Transportnetzwerk zum Fluss . . . 78
9.1.2 Schnitte liefern eine obere Schranke . . . 80
9.1.3 Der Residualgraph: Was können wir noch ändern? . . . 81
9.1.4 Ford-Fulkerson-Algorithmus . . . 82
9.1.5 Warum die Pfadwahl wichtig ist . . . 84
9.1.6 Edmonds-Karp-Algorithmus . . . 84
9.2 Bipartites Matching . . . 86
9.2.1 Vom Matching zum maximalen Fluss . . . 86
9.2.2 Problem der stabilen Heirat . . . 87

10 Kostenminimaler Fluss **91**
10.1 Angebote und Nachfragen . . . 91
10.2 Residualkosten: Eine Entscheidung rückgängig zu machen hat einen Preis . . . 92
10.3 Kleins Kreisaufhebungsalgorithmus . . . 93
10.4 Ein allgemeineres Modell . . . 95
10.5 Ein Einblick in die lineare Programmierung . . . 95
10.5.1 Dasselbe Beispiel im Excel-Solver . . . 96

11 Graphfärbung **98**
11.1 Knotenfärbung und chromatische Zahl . . . 98
11.1.1 Wann zwei Farben ausreichen . . . 99
11.1.2 Zurück zu den Landkarten . . . 99
11.2 Planung eines Sportwettbewerbs . . . 100
11.2.1 Dieselbe Idee in anderen Anwendungen . . . 102
11.3 Gierige Färbung . . . 102
11.3.1 Ein Graph, zwei Reihenfolgen . . . 103
11.3.2 Was garantiert die gierige Färbung? . . . 104
11.4 Eine Färbung finden und das Optimum finden . . . 104

Auswahl eines Algorithmus **106**

Abbildungsverzeichnis

1 Ist das ein Graph? . . . 6
2 Ein 2-regulärer Graph mit 4 Knoten. . . . 8
3 Sind diese beiden Graphen „dieselben“? . . . 10
4 Einige besondere Graphen. . . . 12
5 Zwei Kreise mit gemeinsamen Knoten A und D. Alle vier Knoten gehören zu einer starken Zusammenhangskomponente. . . . 20

6 Das Ersetzen jeder starken Zusammenhangskomponente durch einen Knoten ergibt einen DAG. Sowohl (X, Y, Z) als auch (X, Z, Y) sind topologische Ordnungen. . . . 21
7 Der Dodekaedergraph, der in Hamiltons Icosian Game verwendet wurde. 23
8 Die sieben Brücken von Königsberg (Quelle: Wikimedia Commons, Autor: User:Twotwos, 17. September 2025), verändert durch Hinzufügen von Brückenbezeichnungen. 25
9 Verschiedene Graphen für die sieben Brücken von Königsberg 26
10 Verbindungen außerhalb der Diagonale in den booleschen Potenzen A^1 bis A^6 und B^1 bis B^3. Es handelt sich um Graphzeichnungen; Diagonaleinträge werden nicht dargestellt. 31
11 Warshalls Algorithmus auf einem gerichteten Dreierkreis. Gestrichelte blaue Pfeile und fette blaue Matrixeinträge zeigen die in jeder Stufe hinzugefügten Verbindungen. Zeilen und Spalten sind in der Reihenfolge $1, 2, 3$ angeordnet. 33
12 Graph mit negativem Kreis 35
13 Ein gerichteter Graph zum Vergleich von Kürzeste-Pfade-Algorithmen. Die Zahlen neben den Kanten sind ihre Gewichte. 36
14 Schematische Darstellung der Knotenexpansion bei Dijkstra im Vergleich zu A* . . 44
15 Pathologischer Fall, in dem Algorithmus 10 scheitert, da die Heuristik nicht konsistent ist. 46
16 Schematische Darstellung der Knotenexpansion bei bidirektionaler Suche 47
17 Pathologisches Beispiel, in dem die naive Abbruchbedingung den kürzesten Pfad nicht findet 47
18 Beispiel eines Baums (links) und eines Graphen, der kein Baum ist (rechts) 53
19 Vergleich von Wurzelbaum und gerichtetem azyklischem Graphen 54
20 Dasselbe ungerichtete Dreieck: ein Kürzeste-Pfade-Baum von A aus (links) und ein minimaler Spannbaum (rechts). Durchgezogene farbige Kanten sind ausgewählt; die gestrichelte Kante wird weggelassen. 55
21 Beispiel eines gerichteten Graphen, bei dem die gierige Kantenauswahl scheitert . . 59
22 Beispiel für die Kontraktion eines Kreises 60
23 Wahl der Wurzel mit $M = 55$: Zuerst $\{a, b\}$, dann $\{ab, c\}$ kontrahieren; bei der Expansion zuerst bei c, dann bei a eintreten. Nach Entfernen von R bleibt $c \to a \to b$ mit Kosten 4. 61
24 Minimax-Beispiel 63
25 Ein kleiner Webgraph. Ein Pfeil zeigt von der Seite, die einen Link enthält, auf dessen Ziel. 69
26 Drei Probleme beim ausschließlichen Folgen von Hyperlinks: eine Seite ohne ausgehende Links, eine abgeschlossene Seitengruppe und periodische Bewegung. 71
27 Derselbe Webgraph mit seinen PageRank-Werten für $\alpha = 0.85$. 75
28 Zwei Gruppen, verbunden durch einen Knoten geringen Grades. Die Interpretation von Wichtigkeit hängt vom Zentralitätsmaß ab. 76
29 Das Netzwerk aus dem einführenden Flussbeispiel der Folien. Die Kapazitäten aus s summieren sich zu 7, aber die Senke kann höchstens 5 aufnehmen. Die rechte Zeichnung zeigt einen zulässigen Fluss mit Wert 5; die Beschriftungen bedeuten dort Fluss/Kapazität. 79
30 Eine ursprüngliche Kante ergibt zwei mögliche Anpassungen. Eine Vorwärts-Residualkante erlaubt zusätzlichen Fluss; eine Rückwärts-Residualkante erlaubt, Fluss zurückzunehmen. Residualbeschriftungen geben die maximal erlaubte Anpassung an. 81
31 Alle ursprünglichen Kapazitäten sind eins. Nach der Wahl von s, A, B, t ist keine reine Vorwärtsaugmentation möglich. Der Residualpfad s, B, A, t nimmt $A \to B$ zurück und erzeugt einen Fluss mit Wert zwei. 82
32 Eine zulässige, aber ungünstige Pfadwahl. Die mittlere Kante abwechselnd zu verwenden und zurückzunehmen erhöht den Fluss pro Augmentation nur um eine Einheit. Große äußere Kapazitäten helfen bei dieser Pfadwahl nicht. 84

33 Ein bipartiter Graph und sein Flussnetzwerk. Orange Kanten zeigen ein Matching der Größe zwei und den entsprechenden Fluss. Jede Netzwerkkante hat Kapazität eins; die Kreuzung zweier gezeichneter Kanten ist kein Knoten. 87
34 Verbesserung eines Flusses mit festem Wert durch einen negativen Residualkreis. Die Rückwärts-Residualkanten nehmen die teure Lieferung zurück. 93
35 Zwei Modelle für vier Mannschaften. Modell A stellt Mannschaften als Knoten und Spiele als Kanten dar. Modell B stellt Spiele als Knoten und Konflikte als Kanten dar; seine Knotenfarben geben Tage an. Die Zahlen kennzeichnen die Farben auch ohne Farbausdruck. 101
36 Derselbe Graph, gierig in zwei unterschiedlichen Reihenfolgen gefärbt. Das abwechselnde Besuchen entsprechender Knoten verwendet vier Farben; der Besuch einer ganzen Reihe zuerst verwendet zwei. Die Farbnummern sind ausdrücklich angegeben. Kantenkreuzungen sind keine zusätzlichen Knoten. 103

Algorithmenverzeichnis

1 Tarjans Algorithmus zur Bestimmung starker Zusammenhangskomponenten 19
2 Topologische Sortierung (Toposort) für DAGs 22
3 Fleurys Algorithmus zur Suche nach einem eulerschen Weg oder Kreis 28
4 Hierholzers Algorithmus zur Suche nach einem eulerschen Kreis 29
5 Algorithmus zur Berechnung der transitiven Hülle 33
6 Kürzeste Pfade mittels Breitensuche 36
7 Algorithmus zur Berechnung kürzester Pfade zwischen allen Paaren 37
8 Bellman–Moore–Ford mit einer FIFO-Warteschlange 39
9 Dijkstras Algorithmus 41
10 A*-Algorithmus 45
11 A*-Algorithmus ohne abgeschlossene Menge 46
12 Bidirektionaler Dijkstra 49
13 Tiefensuche mit iterativer Vertiefung (IDDFS) 50
14 A* mit iterativer Vertiefung (IDA*) 51
15 Algorithmus von Prim (minimaler Spannbaum) 55
16 Algorithmus von Kruskal (minimaler Spannbaum) 57
17 Rekursiver Minimax-Algorithmus 64
18 Alpha-Beta-Minimax 65
19 Tiefenbegrenztes Minimax 66
20 PageRank durch Potenziteration 73
21 Ford-Fulkerson-Methode 83
22 Edmonds-Karp-Algorithmus 85
23 Gale–Shapley mit antragstellender Seite A 88
24 Kleins Kreisaufhebungsalgorithmus 93
25 Gierige Knotenfärbung (First Fit) 102

1 Grundlagen der Graphentheorie

Was ist ein Graph? Vielleicht haben Sie schon Graphen gesehen. Typischerweise werden sie als Punkte gezeichnet, die durch Linien miteinander verbunden sind. Jede Linie verbindet genau zwei Punkte. In diesem Kurs benötigen wir jedoch eine formale Definition, um anschließend über Graphen argumentieren zu können. In Abbildung 1 sehen Sie eine Zeichnung, die bereits eine gewisse Komplexität zeigt.

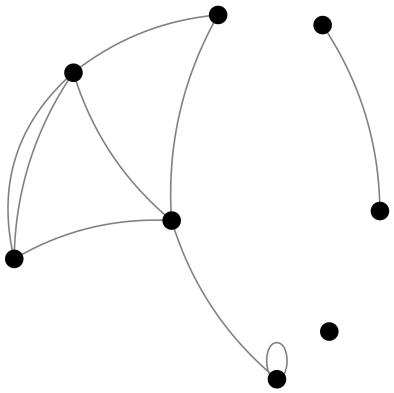


Abbildung 1: Ist das ein Graph?

Neben der einfachen Tatsache „durch Linien verbundene Punkte“ können wir in Abbildung 1 zumindest Folgendes beobachten:

- Es gibt einen Punkt, der durch keine Linie verbunden ist.
- Zwei Punkte sind miteinander verbunden, aber nicht mit dem Rest.
- Zwei Punkte sind durch zwei Linien verbunden.
- Ein Punkt ist mit sich selbst verbunden.

Mathematische Beschreibungen können schon beim Lesen anspruchsvoll sein, beim Schreiben sind sie noch schwieriger. Versuchen Sie, die formalen Definitionen selbst aufzuschreiben. Sie werden die hier gegebenen Definitionen vermutlich viel mehr schätzen, wenn Sie selbst darüber nachgedacht haben, auch wenn Ihre eigenen nicht perfekt waren. Zu erkennen, wo man Fehler gemacht hat, ist eine ausgezeichnete Art zu lernen.

Innehalten und nachdenken – Kurzcheck

Welche Eigenschaften können Sie aus dieser Zeichnung ableiten? Beschreiben Sie selbst in der Sprache der Mathematik, was Sie sehen, bevor Sie weiterlesen!

1.1 Ungerichtete Graphen

Die Punkte erscheinen als atomare, also nicht zerlegbare Einheiten ohne weitere Eigenschaften, abgesehen davon, dass wir wohl jeden von ihnen identifizieren können. Zur Beschreibung der Punkte verwenden wir daher die mathematische Notation einer Menge. Die Menge V nennen wir die Menge der *Knoten*, auch Ecken oder Punkte (engl. vertices). Auch eine Linie erscheint als identifizierbare, ansonsten aber atomare Einheit. Daher verwenden wir die Menge E zur Beschreibung der *Kanten* (engl. edges), welche die Linien in der Abbildung darstellen. Offensichtlich steht die Kantenmenge *irgendwie in Beziehung* zur Knotenmenge, da eine Kante immer an einem Punkt endet – eine Beziehung, die wir mathematisch ausdrücken müssen.

Wir beginnen mit dem einfachen Fall: keine Schleifen und höchstens eine Kante zwischen je zwei Knoten. Auf parallele Kanten kommen wir bei der Modellierung der Königsberger Brücken zurück.

Definition 1.1. Ein einfacher ungerichteter Graph ist ein Tupel $G = (V, E)$, wobei V die endliche Knotenmenge und $E \subseteq [V]^2$ die Kantenmenge ist.

Notation

Hier bezeichnet $[V]^2 = \{\{u, v\} \mid u, v \in V,\ u \neq v\}$ die Menge aller zweielementigen Teilmengen von V. Eine Kante ist ein ungeordnetes Paar: $\{u, v\} = \{v, u\}$, sie hat also keine Richtung.

Innehalten und nachdenken – Selbst erarbeiten

Sie können sicherlich $[V]^2 = \{\{u, v\} \mid u, v \in V \wedge u \neq v\}$ ausdrücken. Fällt Ihnen ein allgemeiner Ausdruck für $[V]^n$ mit $n \in \mathbb{N}$ ein? Hinweis: Verwenden Sie den Begriff der *Potenzmenge* $\mathcal{P}(V)$.

Hintergrund

Zur Erinnerung: Eine Menge ist eine Sammlung von Objekten, in der jedes Objekt nur einmal vorkommt. In einer Menge gibt es keine Reihenfolge, also beispielsweise kein „erstes Element“. Die Potenzmenge ist die Menge aller möglichen Teilmengen einer Menge. In einem Tupel dürfen Objekte mehrfach vorkommen, und es gibt eine Reihenfolge. Mengen werden in diesen Notizen durch „{“ und „}“ eingeschlossen, Tupel durch „(“ und „)“. In der Programmierung werden geordnete Folgen häufig durch Listen oder Arrays dargestellt.

Sofern nicht anders angegeben, sind unsere ungerichteten Graphen einfach.
Nun haben wir eine Definition des Graphen und können anfangen, über Graphen zu argumentieren. Dazu untersuchen und definieren wir einige Eigenschaften ungerichteter Graphen. Da wir zwei Mengen haben, können wir die „Größe“ eines Graphen auf zwei Arten betrachten. Wir können sagen, dass er viele Knoten hat, oder dass er viele Kanten hat.

Definition 1.2. Die *Ordnung* (engl. order) eines Graphen $G = (V, E)$ ist seine Knotenanzahl, also $|V|$. Die *Größe* (engl. size) eines Graphen ist seine Kantenanzahl, also $|E|$.

Bei einfachen ungerichteten Graphen ist die Größe auch durch die Ordnung beschränkt, da $|E| \leq |[V]^2|$ gilt.
Als Nächstes beobachten wir, dass in unserem einfachen Graphen jede Kante genau zwei Knoten verbindet. Wir wollen daher ausdrücken, dass ein Knoten andere Knoten als *Nachbarn* hat. Manche Knoten besitzen mehr Nachbarn als andere, deshalb wollen wir auch die Anzahl der Nachbarn ausdrücken.

Innehalten und nachdenken – Selbst erarbeiten

Wie könnten Sie ausdrücken, dass zwei Knoten durch eine Kante verbunden sind? Wie würden Sie das mathematisch formulieren? Beim Weiterlesen wird es „offensichtlich“ sein, selbst darauf zu kommen ist aber nicht so einfach. Mathematische Gewandtheit lässt sich durch Übung erwerben – durch Lesen und Schreiben.

Definition 1.3. Zwei Knoten $u, v \in V$ mit $u \neq v$ heißen *benachbart* (engl. adjacent). Wir verwenden das Relationssymbol $\sim$ und schreiben $u \sim v \Leftrightarrow \exists e \in E : u \in e \wedge v \in e$. Wir sagen außerdem, dass eine Kante $e \in E$ und ein Knoten $v \in V$ miteinander *verbunden*, also inzident (engl. incident), sind, wenn $v \in e$ gilt.

Wir können den Operator „ $\in$ “ auf e anwenden, denn E ist eine Menge von Mengen, und e ist ein Element von E und somit selbst eine Menge.
Damit haben wir einen Formalismus, um Nachbarschaft zwischen Knoten auszudrücken. Mit dieser Definition können wir nun die Anzahl der Nachbarn eines Knotens angeben.

Definition 1.4. Der *Grad* eines Knotens v ist die Anzahl der zu ihm inzidenten Kanten, also $\deg(v) = |\{e \in E \mid v \in e\}|$.

Hintergrund

Bei unserem einfachen Graphen macht es für die Graddefinition keinen Unterschied, ob wir die Anzahl „benachbarter Knoten“ oder die Anzahl „inzidenter Kanten“ verwenden. Bei Multigraphen besteht jedoch ein wesentlicher Unterschied.

Nun wollen wir die Nachbarschaft eines Knotens ausdrücken.

Definition 1.5. Die Menge aller zu einem Knoten v benachbarten Knoten heißt *Nachbarschaft* von v und wird mit $N(v) = \{u \in V \mid u \sim v\}$ bezeichnet.

Für einfache Graphen gilt $|N(v)| = \deg(v)$, da jede Kante genau zwei Knoten verbindet. Bei Multigraphen kann der Grad eines Knotens jedoch größer als die Größe seiner Nachbarschaft sein. Ein Knoten ohne Nachbarn hat Grad 0 und heißt *isolierter Knoten* (engl. isolated vertex).

Außerdem gibt es einen maximalen und einen minimalen Grad eines Graphen, also das Maximum beziehungsweise Minimum der Grade aller seiner Knoten.

Definition 1.6. Der *Maximalgrad* (engl. maximum degree) eines Graphen $G = (V, E)$ wird mit $\Delta(G) = \max_{v \in V} \deg(v)$ bezeichnet, der *Minimalgrad* (engl. minimum degree) mit $\delta(G) = \min_{v \in V} \deg(v)$.

Es gibt auch Graphen, für die $\Delta(G) = \delta(G)$ gilt. Diese heißen *reguläre Graphen* (engl. regular graphs).

Definition 1.7. Ein Graph ist regulär, wenn $\exists r \in \mathbb{N} : \forall v \in V : \deg(v) = r \Leftrightarrow \Delta(G) = \delta(G) = r$ gilt. Ein solcher Graph heißt *r-regulärer Graph* (engl. r-regular graph).

Innehalten und nachdenken – Selbst erarbeiten

Denken Sie sich einen 3-regulären Graphen mit 6 Knoten aus. Wie sähe er aus? Können Sie auch einen 3-regulären Graphen mit 5 Knoten zeichnen?

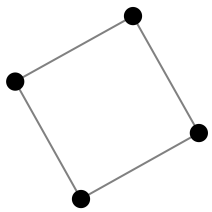


Abbildung 2: Ein 2-regulärer Graph mit 4 Knoten.

Wenn Sie versucht haben, einen 3-regulären Graphen mit 5 Knoten zu zeichnen, stellen Sie fest, dass es offenbar nicht möglich ist, egal was Sie versuchen. Gibt es wirklich keine Möglichkeit, einen solchen Graphen zu konstruieren? Betrachten wir dies mathematisch.

Für einen einfachen ungerichteten Graphen $G = (V, E)$ muss gelten:

Satz 1.8.

$$\sum_{v \in V} deg(v) = 2 \cdot |E|$$

Beweis. Jede Kante wird genau zweimal gezählt: einmal für jedes ihrer Enden, und jede Kante hat genau zwei Enden. Es gibt keine Kante ohne Enden in V. Zählen wir jede Kante genau zweimal, haben wir die doppelte Kantenanzahl gezählt. □

Da diese Zahl ein Vielfaches von zwei ist, muss sie *gerade* sein. Unser hypothetischer Graph mit Grad 3 an jedem seiner 5 Knoten hätte die Gradsumme 15 – eine ungerade Zahl. Ein solcher „Graph“ würde Satz 1.8 verletzen und kann daher nicht existieren. Wir können aber eine noch stärkere Aussage machen:

Satz 1.9 (Handschlaglemma)**.** *In einem einfachen ungerichteten Graphen muss es eine gerade Anzahl von Knoten mit ungeradem Grad geben. Formal sei $G = (V, E)$ ein einfacher ungerichteter Graph. Dann gilt $|\{v \in V \mid deg(v) \notin \mathbb{N}_e\}| \in \mathbb{N}_e$, wobei $\mathbb{N}_e$ die Menge der geraden Zahlen ist (definitionsgemäß gilt $0 \in \mathbb{N}_e$).*

Beweis. Nach Satz 1.8 wissen wir bereits, dass $\sum_{v \in V} \deg(v)$ gerade ist. Wir können diese Summe in zwei Summen aufteilen, eine über alle Knoten mit geradem Grad und eine über alle Knoten mit ungeradem Grad:

$$\sum_{v \in V} \deg(v) = \sum_{v \in V \mid \deg(v) \in \mathbb{N}_e} \deg(v) + \sum_{v \in V \mid \deg(v) \notin \mathbb{N}_e} \deg(v)$$

Wir wissen: Wenn $z = x + y$ und $z \in \mathbb{N}_e$, dann müssen x und y dieselbe Parität haben.
Die erste Summe auf der rechten Seite ergibt eine gerade Zahl, da sie eine Summe gerader Zahlen ist; die Knotengrade sind gerade.
Daher muss auch die zweite Summe eine gerade Zahl ergeben. Der zweite Ausdruck summiert jedoch nur ungerade Zahlen; die Knotengrade sind ungerade.
Eine Summe ungerader Zahlen ist genau dann gerade, wenn sie eine gerade Anzahl ungerader Zahlen enthält.
Somit muss es stets eine gerade Anzahl von Knoten mit ungeradem Grad geben. □

Wieder sehen wir, dass ein 3-regulärer Graph mit 5 Knoten das Handschlaglemma verletzt. Er hätte eine ungerade Anzahl von Knoten (5) mit ungeradem Grad (3).

1.2 Gerichtete Graphen

Wir führen einen gerichteten Graphen formal als 2-Tupel $G = (V, E)$ ein, wobei V die Knotenmenge und E die Kantenmenge ist, ähnlich wie bei ungerichteten Graphen.

Innehalten und nachdenken – Selbst erarbeiten

Überlegen Sie, wie *Sie* die Gerichtetheit einer Kante ausdrücken würden! Eine Kante muss offensichtlich „etwas mit zwei Knoten“ sein, und wir müssen klar ausdrücken, dass einer der Start und der andere das Ende ist.

Um die Gerichtetheit auszudrücken, sagen wir, dass eine Kante ein Tupel, also ein Paar zweier Knoten ist. Wie Sie wissen, spielt in einem Tupel die Reihenfolge eine Rolle, und (u, v) und (v, u) sind verschiedene Tupel. Wir legen fest, dass das erste Element des Paars der Startknoten (engl. start vertex) und das zweite der Endknoten (engl. end vertex) ist.

Definition 1.10. Die Kantenmenge E eines Graphen ist als Menge geordneter Knotenpaare definiert, also $E \subseteq V \times V$. Wir sagen, eine Kante $e = (u, v)$ ist von u nach v gerichtet, und schreiben $u \to v$.

Beachten Sie, dass das kartesische Produkt $V \times V$ auch Paare desselben Knotens (u, u) enthält, also Schleifen. Im Unterschied zu einfachen ungerichteten Graphen schließt der einfache gerichtete Graph Schleifen daher nicht aus.

Innehalten und nachdenken – Kurzcheck

Beantworten Sie selbst: Erlaubt diese Definition gerichteter Graphen auch mehrere Kanten zwischen zwei Knoten? Warum beziehungsweise warum nicht?

Auch die Beziehung zwischen Knoten ist in gerichteten Graphen stärker differenziert. Gegeben seien ein gerichteter Graph $G = (V, E)$ und eine Kante $e = (u, v) \in E$ mit $u \neq v \wedge u, v \in V$. Dann heißt u ein *Vorgänger* (engl. predecessor) von v und v ein *Nachfolger* (engl. successor) von u. Ähnlich zur Nachbarschaft haben wir nun die *Nachfolger* und *Vorgänger* eines Knotens.

Definition 1.11. Die *Nachfolger* eines Knotens v in einem gerichteten Graphen $G = (V, E)$ bilden die Menge aller Knoten, auf die eine von v ausgehende Kante zeigt, also $Suc(v) = \{u \in V \mid (v, u) \in E\}$.

Die *Vorgänger* eines Knotens v in einem gerichteten Graphen $G = (V, E)$ bilden die Menge aller Knoten, von denen eine Kante nach v zeigt, also $Pre(v) = \{u \in V \mid (u, v) \in E\}$.

Ebenso müssen wir den Gradbegriff verfeinern. In gerichteten Graphen gibt es zwei Arten von Graden: den *Eingangsgrad* (engl. in-degree) und den *Ausgangsgrad* (engl. out-degree). Bei ungerichteten Graphen sind Suc und Pre mit der Nachbarschaft N identisch.

Definition 1.12. Der *Eingangsgrad* eines Knotens v ist die Anzahl der in ihn gerichteten Kanten, also $\text{indeg}(v) = |\{(a, b) \in E \mid v = b\}|$.

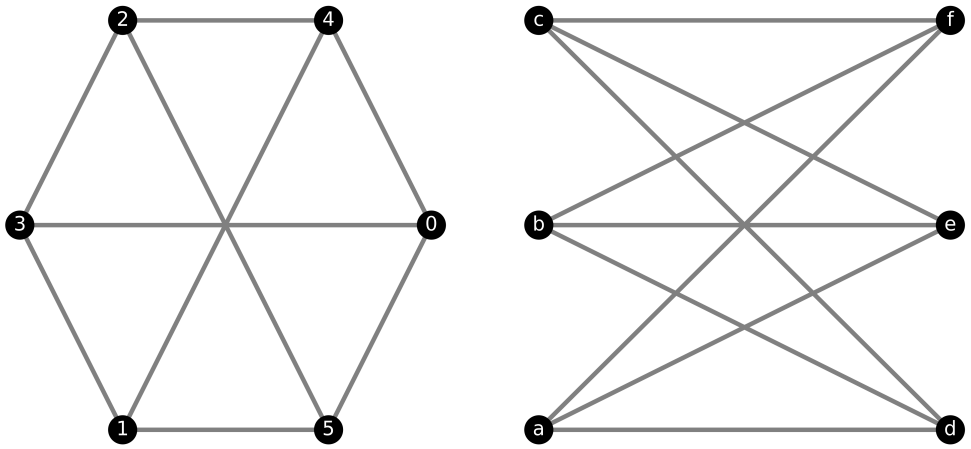
Der *Ausgangsgrad* eines Knotens v ist die Anzahl der aus ihm gerichteten Kanten, also $\text{outdeg}(v) = |\{(a, b) \in E \mid v = a\}|$.

Innehalten und nachdenken – Kurzcheck

Wie beeinflusst eine Schleife nach dieser Definition den Eingangs- und Ausgangsgrad eines Knotens? Die Antwort steht in der Definition; Sie müssen sie nur lesen und aufschlüsseln. Mathematische Definitionen müssen exakt und eindeutig sein.

1.3 Isomorphie von Graphen

Eine bisher umgangene Frage ist, wann zwei Graphen „gleich“ sind. Echte Gleichheit ist natürlich nur möglich, wenn Knotenmenge und Kantenmenge exakt übereinstimmen. Häufig sind zwei Graphen jedoch nur unterschiedlich beschriftet und sehen trotzdem gleich aus. Betrachten Sie die beiden Graphen in Abbildung 3: Sind sie „dieselben“?



(a) (b)

Abbildung 3: Sind diese beiden Graphen „dieselben“?

Wir können zwei Graphen als „dieselben“ ansehen, wenn wir einen von ihnen mit den Bezeichnungen des anderen beschreiben können. In der Mathematik sagen wir, dass es einen Morphismus gibt, der einen Graphen in den anderen überführt.

Definition 1.13. Zwei Graphen $G = (V, E)$ und $H = (U, F)$ heißen *isomorph* (engl. isomorphic), wenn eine bijektive Funktion $f : V \to U$ existiert, sodass für jedes Knotenpaar $a, b \in V$ die Äquivalenz $a \sim b \Leftrightarrow f(a) \sim f(b)$ gilt (offensichtlich muss $|V| = |U| \wedge |E| = |F|$ gelten).

In dieser Definition erhalten isomorphe Graphen die Nachbarschaftsbeziehungen. Sind zwei Knoten in einem Graphen benachbart, dann sind ihre Bilder unter der Abbildung f auch im anderen Graphen benachbart. Daraus folgt zwar auch, dass entsprechende Knoten denselben Grad haben, doch zum Nachweis der Isomorphie reicht es nicht, nur die Gradfolgen zweier Graphen zu vergleichen. Zu bestimmen, ob zwei Graphen isomorph sind, ist leider ein recht schwieriges Problem, das *Graphisomorphieproblem*. Es mag zunächst harmlos und einfach aussehen, seine Komplexitätsklasse ist jedoch tatsächlich weiterhin unbekannt. Für $n = |V| = |U|$ gibt es $n!$ mögliche bijektive Funktionen zwischen den Knotenmengen der beiden Graphen. Diese Zahl wächst extrem schnell: Bereits bei 20 Knoten gibt es $2,432,902,008,176,640,000$ mögliche Abbildungen. Ein Brute-Force-Ansatz, der jede mögliche Abbildung prüft, ist daher schon für mittelgroße Graphen völlig undurchführbar. László Babai kündigte 2015 einen bedeutenden Durchbruch an und stellte ihn 2016 auf der STOC vor: Graphisomorphie lässt sich in quasipolynomieller Zeit $\exp((\log n)^{O(1)})$ entscheiden, wobei n die Knotenanzahl in jedem der beiden Graphen ist [1]. Hier gilt $\exp(x) = e^x$, und $O(1)$ im Exponenten steht für eine feste Konstante. Wenn log den natürlichen Logarithmus bezeichnet, gilt $e^{\log n} = n$, aber eine Potenz von $\log n$ verändert das Wachstum erheblich. Zum Beispiel:

$$e^{(\log n)^2} = (e^{\log n})^{\log n} = n^{\log n}.$$

Dieses Beispiel wächst schneller als jede feste Potenz von n, aber viel langsamer als 2^n. Die quasipolynomielle Schranke liefert daher keinen Algorithmus mit linearer oder auch nur polynomieller Laufzeit. Ob das allgemeine Graphisomorphieproblem einen Polynomialzeitalgorithmus besitzt oder NP-vollständig ist, bleibt unbekannt.

1.4 Graphfamilien

In der Literatur gibt es einige besondere Graphen mit bestimmten Eigenschaften, die ausdrücklich benannt sind. Dazu gehören:

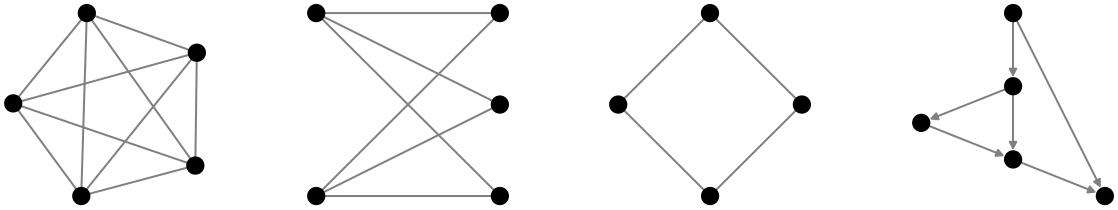
Vollständige Graphen (engl. complete graphs): Ein vollständiger Graph ist ein einfacher ungerichteter Graph, in dem jeder Knoten zu jedem anderen benachbart ist. Da die Kanten dadurch festgelegt sind, bezeichnen wir einen vollständigen Graphen mit n Knoten mit K_n. Der vollständige Graph K_n ist $(n-1)$-regulär. Auch K_0 ohne Knoten und K_1 ohne Kanten existieren als triviale Fälle.

Bipartite Graphen (engl. bipartite graphs): Ein bipartiter Graph ist ein einfacher ungerichteter Graph, dessen Knoten in zwei disjunkte Mengen U und V aufgeteilt werden können, sodass jede Kante einen Knoten in U mit einem Knoten in V verbindet. Innerhalb derselben Menge gibt es keine Kanten. Ein *vollständiger bipartiter Graph* ist ein bipartiter Graph, in dem jeder Knoten in U mit jedem Knoten in V verbunden ist. Wir bezeichnen ihn mit $K_{m,n}$, wobei $|U| = m$ und $|V| = n$ gelten.

Kreise (engl. cycles): Ein Kreis ist ein einfacher ungerichteter Graph, dessen Knoten sich so im Kreis anordnen lassen, dass jeder Knoten genau zu seinen beiden Nachbarknoten im Kreis benachbart ist. Mit C_n bezeichnen wir einen Kreis mit n Knoten. Jeder Kreis C_n ist 2-regulär, und jeder C_n mit $n = 2 \cdot k,\ k \in \mathbb{N}$, also einer geraden Knotenanzahl, ist auch bipartit.

Azyklische gerichtete Graphen (engl. directed acyclic graphs, DAGs): Ein gerichteter azyklischer Graph ist eine besondere Unterfamilie gerichteter Graphen, die *keine Kreise enthalten*. Beginnen wir an einem Knoten v und folgen einer Folge gerichteter Kanten, können wir also nicht schließlich wieder zu v zurückkehren. DAGs treten in vielen Anwendungen auf, etwa zur Darstellung von Abhängigkeiten bei der Aufgabenplanung, in Datenverarbeitungspipelines und Versionsverwaltungssystemen.

Abbildung 4 zeigt Beispiele dieser besonderen Graphen. Betrachten wir die Eigenschaft eines Kreises, bipartit zu sein. Bei C_4 (Abbildung 4c) und allgemein bei geraden Kreisen können wir eine



(a) K_5 (b) $K_{2,3}$ (c) C_4 (d) Ein DAG

Abbildung 4: Einige besondere Graphen.

einfache Färberegel anwenden. Während wir den Kanten folgen, färben wir die Knoten abwechselnd mit zwei Farben, bis alle gefärbt sind. Wenn dabei nie ein Konflikt auftritt, etwa die Färbung eines bereits gefärbten Knotens mit der anderen Farbe, ist der Graph bipartit. Das funktioniert für C_4, aber nicht für C_5, weil wir den Startknoten mit der anderen Farbe erreichen.

Satz 1.14. *Ein ungerichteter Graph ist genau dann bipartit, wenn er keine Kreise ungerader Länge enthält.*

Es gibt viele weitere benannte Graphen, etwa den Petersen-Graphen. Diese untersuchen wir hier jedoch nicht.

1.5 Teilgraphen

Manchmal möchten wir nur über einen Teil eines Graphen nachdenken. Betrachten wir eine globale Lieferkette, in der Knoten Lieferanten, Produzenten, Lager und Verbraucher darstellen, verbunden durch Kanten, die Materialflüsse mit bestimmten Transportmitteln bezeichnen. Wollen wir in diesem Graphen nur diejenigen Knoten und Kanten betrachten, die mit einem bestimmten Material zusammenhängen, möchten wir möglicherweise einen Teil des Graphen „extrahieren“ können. Dies nennen wir einen *Teilgraphen*.

Definition 1.15. Ein Teilgraph (engl. subgraph) $H = (V_H, E_H)$ eines Graphen $G = (V, E)$ ist ein Graph mit $V_H \subseteq V$ und $E_H \subseteq E$.

Da wir sagen, dass H ein Graph ist, muss er auch die oben definierten Eigenschaften eines Graphen erfüllen. Deshalb müssen wir $E_H \subseteq [V_H]^2$ beziehungsweise $E_H \subseteq V_H \times V_H$ für gerichtete Graphen nicht ausdrücklich angeben; dies ist in der Aussage „Das ist ein Graph“ bereits enthalten.

Innehalten und nachdenken – Selbst erarbeiten

Manchmal wollen wir sagen, dass ein Teilgraph nur aus einer Teilmenge der Knoten gebildet wird und dabei alle Kanten des ursprünglichen Graphen zwischen diesen Knoten behält. Ebenso können wir einen Teilgraphen aus einer Teilmenge der Kanten bilden wollen, der alle zu diesen Kanten inzidenten Knoten behält. Versuchen Sie, dies mathematisch auszudrücken, bevor Sie weiterlesen.

Ein aus einer Teilmenge der Knoten gebildeter Teilgraph heißt *knoteninduzierter Teilgraph* (engl. vertex-induced subgraph), ein aus einer Teilmenge der Kanten gebildeter heißt *kanteninduzierter Teilgraph* (engl. edge-induced subgraph).

Definition 1.16. Ein *knoteninduzierter Teilgraph* $H = (V_H, E_H)$ eines Graphen $G = (V, E)$ ist ein Teilgraph mit $V_H \subseteq V$ und $E_H = \{\{u, v\} \in E \mid u \in V_H \wedge v \in V_H\}$.

Ein *kanteninduzierter Teilgraph* $H = (V_H, E_H)$ eines Graphen $G = (V, E)$ ist ein Teilgraph mit $E_H \subseteq E$ und $V_H = \{v \in V \mid \exists e \in E_H : v \in e\}$.

Notation

Wir verwenden die Notation $\langle V \rangle$ für einen knoteninduzierten Teilgraphen und $\langle E \rangle$ für einen kanteninduzierten Teilgraphen. Je nach gewünschtem Teilgraphen können wir also auch $H = \langle V_H \rangle$ oder $H = \langle E_H \rangle$ schreiben. Die Winkelklammern lassen sich somit als Operator, wenn Sie möchten als Funktion, auffassen, der aus einer gegebenen Menge einen Graphen erzeugt.

Wir verwenden denselben Operator für Knoten- und Kantenmengen, daher ist die Bedeutung wichtig. Das ist etwas ungenau. Präziser müssten wir zwei Operatoren einführen, etwa $\langle_v \cdot \rangle_v$ für knoteninduzierte Teilgraphen. Die Notation wäre dann aber schwieriger zu lesen – wäre sie wirklich klarer?
Teilgraphen, die dieselbe Knotenmenge enthalten, haben einen besonderen Namen. Sie heißen *spannende Teilgraphen* (engl. spanning subgraphs). In einem späteren Abschnitt kommen wir darauf zurück.

Definition 1.17. Ein Teilgraph $H = (V_H, E_H)$ eines Graphen $G = (V, E)$ heißt *spannender Teilgraph*, wenn $V_H = V$ und $E_H \subseteq E$ gelten.

Innehalten und nachdenken – Kurzcheck

Ist jeder Graph ein Teilgraph seiner selbst? Warum beziehungsweise warum nicht?

1.6 Gewichtete Graphen

In vielen Anwendungen sind Kanten Zahlen zugeordnet, sogenannte *Gewichte*. In einem Transportnetzwerk können Kanten etwa Straßen zwischen Städten, den Knoten, darstellen, und das Gewicht kann Distanz oder Reisezeit zwischen den Städten bedeuten. In sozialen Netzwerken können Kanten Verbindungen zwischen Personen darstellen und das Gewicht die Anzahl der Interaktionen.

Innehalten und nachdenken – Selbst erarbeiten

Wie würden Sie Gewichte mathematisch ausdrücken? Denken Sie darüber nach, bevor Sie weiterlesen. Sobald man es sieht, ist es trivial, selbst darauf zu kommen aber vielleicht nicht so einfach. Schärfen Sie Ihre mathematische Ausdrucksfähigkeit, indem Sie es selbst versuchen.

Studierende schlugen häufig vor, die Kantendefinition zu erweitern, etwa zu $e = \{u, v, w\}$. Das ist eine typische Programmierlösung – vielleicht war es auch Ihre Wahl? Mathematisch ist sie aber nicht so elegant. Wie identifizieren wir das Gewicht? Eine Formulierung wie „Das Gewicht ist dasjenige Element der Kantenmenge, das kein Knoten ist“ wäre vielleicht möglich, aber recht kompliziert. Bei gerichteten Graphen können wir die Elemente unterscheiden, müssten die Eigenschaften aber über einen Index ansprechen und uns merken, dass Index 3 das Gewicht enthält, wie bei einer tupelbasierten Kantenliste. Eine allgemeinere, einfachere und elegantere Methode, eine Objekteigenschaft mathematisch auszudrücken, ist eine *Funktion*, die von der Objektmenge in die Menge der Eigenschaftswerte abbildet.

Definition 1.18. Ein *gewichteter Graph* ist ein Tupel $G = (V, E, \omega)$, wobei V die Knotenmenge, E die Kantenmenge und $\omega : E \to Z$ eine Gewichtsfunktion ist, die jeder Kante eine Zahl aus der Zahlenmenge Z zuweist, mit $Z \in \{\mathbb{N}, \mathbb{Z}, \mathbb{R}, \ldots\}$.

Für einige Algorithmen sind positive Gewichte ($Z = \mathbb{N}$ oder $Z = \mathbb{R}^+$) Voraussetzung; manchmal ist Ganzzahligkeit aus Effizienzgründen oder für ganzzahlige Ergebnisse notwendig.
Stellt etwa eine Kante $e = \{A, B\}$ eine Straße mit zwölf Minuten Fahrzeit dar, schreiben wir $\omega(e) = 12$. Die Endpunkte bleiben in e; die Funktion liefert die Reisezeit. Eine zweite Funktion könnte derselben Kante eine Maut zuweisen.

Exkurs

In der Programmierung besteht ein Zusammenhang mit objektorientierter Programmierung (OOP) und datenorientiertem Entwurf (DoD). OOP gruppiert die Eigenschaften eines Objekts häufig in einer Struktur; viele solche Objekte bilden ein *Array von Strukturen*. DoD kann stattdessen eine *Struktur von Arrays* verwenden, wobei die Identität eines Objekts als Index in diese Arrays dient. Dies kann die Cache-Effizienz verbessern, wenn dieselbe Eigenschaft vieler Objekte verarbeitet wird. Mathematische Funktionen bieten eine verwandte Möglichkeit, ansonsten atomaren Objekten Eigenschaften zuzuordnen. Auch Entity-Component-System-Architekturen (ECS) in Spielen lassen sich gut mit datenorientierten Techniken kombinieren. Das sind nützliche Zusammenhänge, doch die beste Darstellung hängt von der Anwendung ab.

2 Wanderungen und Zusammenhang

Wir haben bereits einiges an Formalismus entwickelt, um über Graphen zu argumentieren. Einen Aspekt können wir jedoch noch nicht beschreiben. Erinnern Sie sich an die Zeichnung in Abbildung 1? Wir erkannten Knoten, die „abgetrennt“ wirkten und nicht mit den anderen verbunden waren. Mit unseren bisherigen Definitionen können wir diese Eigenschaft nicht ausdrücken. Vielleicht denken Sie, man könnte den Knotengrad verwenden. Dieser ist jedoch eine Eigenschaft eines einzelnen Knotens. Dass einige Knoten nicht mit anderen verbunden sind, erscheint eher als Eigenschaft des gesamten Graphen.
Wir müssen neue Begriffe und Konzepte erfinden und formal definieren, wenn wir über solche Eigenschaften argumentieren wollen.

Innehalten und nachdenken – Selbst erarbeiten

Versuchen Sie erneut, darüber nachzudenken, bevor Sie weiterlesen. Wie würden Sie vorgehen? Welche Konzepte müssen wir einführen? Wie werden sie definiert? Probieren Sie es selbst aus und haben Sie etwas Spaß mit Mathematik. Machen Sie sich keine Sorgen, wenn es nicht korrekt oder richtig wird. Vielleicht hatten Sie die richtige Idee, konnten sie aber schwer beschreiben, vielleicht war die Idee insgesamt falsch. Währenddessen haben Sie aber vermutlich viel gelernt! Vielleicht hatten auch die Begründer der Graphentheorie Schwierigkeiten, bevor sie die richtigen Definitionen fanden. Dies ist ein kreativer Prozess.

Definition 2.1. Eine Wanderung (engl. walk) in einem Graphen ist ein Tupel $W_{a,b} = (S_V, S_E)$, wobei $S_V = (v_0, v_1, \dots, v_n)$ eine Knotenfolge und $S_E = (e_1, e_2, \dots, e_n)$ eine Kantenfolge ist, sodass

$$v_i \in V \quad \forall i \in \{0, 1, \dots, n\} \tag{1}$$
$$e_i \in E \quad \forall i \in \{1, 2, \dots, n\} \tag{2}$$
$$e_i = \{v_{i-1}, v_i\} \quad \forall i \in \{1, 2, \dots, n\} \tag{3}$$
$$v_0 = a \tag{4}$$
$$v_n = b \tag{5}$$

In dieser Definition ist $a = v_0$ der Startknoten und $b = v_n$ der Endknoten der Wanderung. Wanderungen lassen sich auch gerichtet beschreiben; da die Kanten dann Tupel sind, müssen wir in Gleichung (3) die Bedingung $e_i = (v_{i-1}, v_i)$ verwenden. Da eine Wanderung in einfachen Graphen bereits durch S_V oder S_E allein bestimmt ist, können wir in der Notation einer Wanderung außerdem nur eine der Folgen angeben.

Hintergrund

Zwei Aspekte sind in diesem Exkurs zu bedenken. Zunächst sind $v_0, v_1, \dots$ und Ähnliches nicht selbst Knoten, sondern Variablen. Zuvor haben wir Variablen wie u und v verwendet, um Elemente der Menge V darzustellen. Hier verbinden wir zusätzlich jede Variable mit einer Position in der Folge, bezeichnet durch den Index.

Innehalten und nachdenken – Kurzcheck

Nun sind wir fast am Ziel. Wir haben ein neues Objekt und seine Eigenschaften definiert. Wenn Sie sich je fragen, ob etwas eine Wanderung ist, müssen Sie die Definition betrachten. Erkennen Sie schon, wie wir nun Zusammenhang ausdrücken können?

Das Konzept, ob Knoten durch das Folgen von Kanten von anderen aus erreichbar sind, nennen wir *Zusammenhang* (engl. connectedness) und definieren es formal mithilfe der Wanderung.

Definition 2.2. Ein Graph $G = (V, E)$ heißt *zusammenhängend* (engl. connected), wenn für *jedes* Knotenpaar mindestens eine Wanderung existiert. Das heißt: $\forall u, v \in V : u \neq v \; \exists \; W_{u,v} = (S_V, S_E)$ ist eine Wanderung.

Definition 2.3. Gegeben sei eine Wanderung zwischen den Knoten u und v: $W_{u,v} = (S_V, S_E)$. Ihre Länge $\mathcal{L}_1(W_{u,v}) = |S_E|$ ist die Anzahl der Kanten in der Folge. Die *triviale Wanderung* $W = ((u), ())$, deren Start- und Endknoten gleich sind, ist die kürzestmögliche Wanderung und hat Länge 0.

Definition 2.4. Eine Wanderung mit gleichem Start und Ende heißt *geschlossen* (engl. closed). Das Gegenteil, eine Wanderung, die an einem anderen Knoten endet, als sie beginnt, heißt *offen* (engl. open).

Aus der Definition einer Wanderung erkennen wir, dass es keine Einschränkung dafür gibt, ob ein Knoten erneut besucht oder dieselbe Kante mehrfach verwendet werden darf. Durch Hin- und Herspringen zwischen zwei Knoten könnten wir einfach eine sehr lange Wanderung erzeugen. Zum Beispiel hat die geschlossene Wanderung mit $S_V = (a, b, a, b, a, b, a, b, a, b, a)$ Länge 10, falls $a \sim b$ gilt. Manchmal sind Einschränkungen beim Auftreten von Kanten oder Knoten aber erwünscht, daher führen wir Folgendes ein:

Definition 2.5. Ein *Weg* (engl. trail) ist eine Wanderung, in der keine Kante wiederholt wird:
$\forall e_i, e_j \in S_E : \; e_i = e_j \Leftrightarrow i = j.$

Definition 2.6. Ein *Pfad*, auch einfacher Weg (engl. path), ist ein Weg, in dem kein Knoten wiederholt wird:
$\forall v_i, v_j \in S_V : \; v_i = v_j \Leftrightarrow i = j.$

Die triviale Wanderung ist der einzige geschlossene *Pfad*. Über Kreise haben wir bereits gesprochen. Ein geschlossener Weg, bei dem Start- und Endknoten die einzigen doppelt vorkommenden Knoten sind, bildet ebenfalls einen Kreis. Aus dieser Definition ist auch klar, dass jeder Pfad ein Weg und jeder Weg eine Wanderung ist. Existiert zwischen zwei Knoten ein Pfad, existiert daher auch eine Wanderung zwischen ihnen.

Definition 2.7. Wir definieren den Operator $\circ$ für die Verkettung von Tupeln. Betrachten wir $A = (a_0, a_1, \ldots, a_n)$ und $B = (b_0, b_1, \ldots, b_m)$. Ihre Verkettung ist $A \circ B = (a_0, a_1, \ldots, a_n, b_0, b_1, \ldots, b_m)$.

Wir verwenden $\circ$ auch für Wanderungen, aber nur, wenn das Ende der ersten gleich dem Start der zweiten ist. Sind $S_V^1 = (v_0, \ldots, v_n)$ und $S_V^2 = (v_n, u_1, \ldots, u_m)$, dann gilt

$$W_1 \circ W_2 = ((v_0, \ldots, v_n, u_1, \ldots, u_m), \; S_E^1 \circ S_E^2).$$

Der gemeinsame Knoten tritt an der Verbindungsstelle einmal auf; alle Kanten bleiben erhalten. Beispielsweise ergibt das Verbinden der Wanderungen (a, b) und (b, c) die Wanderung (a, b, c). Die gewöhnliche Tupelverkettung, etwa zum Anhängen von Einträgen an algorithmische Listen, behält weiterhin jeden Eintrag.

Definition 2.8. Die Menge aller möglichen Wanderungen in einem Graphen G wird mit $\mathcal{W}(G)$ bezeichnet.

Innehalten und nachdenken – Kurzcheck

Lässt sich für endliche Graphen sagen, ob $\mathcal{W}(G)$ endlich oder unendlich ist? Warum beziehungsweise warum nicht?

2.1 Zusammenhangskomponenten, Brücken, Abstand und Durchmesser

Mit diesen Definitionen können wir nun mathematisch ausdrücken, dass ein Graph in Teile „getrennt“ ist. Wir nennen ihn *unzusammenhängend* (engl. disconnected), im Gegensatz zu einem zusammenhängenden (engl. connected) Graphen.
Zusammenhang kann jedoch mehr als nur eine binäre Eigenschaft sein. Manche Graphen haben mehrere voneinander getrennte Teile, und auch diesen „Teilen“ möchten wir einen Namen geben. Wir können zwischen je zwei Knoten $u, v \in V$ eine Relation R darüber definieren, ob ein Pfad zwischen ihnen existiert:

$$uRv \Leftrightarrow \exists\, W : W \text{ ist ein Pfad von } u \text{ nach } v$$

Eine Relation kann viele Eigenschaften haben. Die Relation R ist

- *reflexiv* (engl. reflexive): uRu für alle $u \in V$,
- *symmetrisch* (engl. symmetric): Aus uRv folgt vRu für alle $u, v \in V$, und
- *transitiv* (engl. transitive): Aus uRv und vRw folgt uRw für alle $u, v, w \in V$.

Sie ist reflexiv, da wir für jeden Knoten $u \in V$ eine triviale Wanderung $W = (u)$ konstruieren können. Sie ist symmetrisch, da wir den Pfad umkehren können; dies gilt nur für ungerichtete Graphen. Schließlich verketten wir die beiden Pfade zu einer Wanderung von u nach w. Trotz korrekter Verbindungsstelle kann sie andernorts Knoten wiederholen. Sobald ein Knoten zweimal vorkommt, entfernen wir den geschlossenen Abschnitt zwischen diesen Vorkommen. Wiederholen wir dies, erhalten wir einen Pfad von u nach w, bei $u = w$ den trivialen Pfad. Damit ist Transitivität bewiesen.
Aufgrund dieser drei Eigenschaften heißt R eine Äquivalenzrelation (engl. equivalence relation). Mit R können wir die Knotenmenge in Äquivalenzklassen (engl. equivalence classes) zerlegen. Jede Äquivalenzklasse ist eine Menge von Knoten, die alle voneinander erreichbar sind: Von jedem Knoten u der Äquivalenzklasse existiert ein Pfad zu jedem anderen Knoten v dieser Klasse.

Definition 2.9. Eine *Zusammenhangskomponente* (engl. connected component) eines Graphen $G = (V, E)$ ist eine Äquivalenzklasse der Relation R.

Innehalten und nachdenken – Selbst erarbeiten

Ist Zusammenhang auch in Teilgraphen garantiert? Können wir etwas über die Anzahl der Zusammenhangskomponenten eines Teilgraphen aussagen?

Ein zusammenhängender Graph hat nur eine Zusammenhangskomponente. Das Entfernen einer Kante kann die Anzahl der Zusammenhangskomponenten erhöhen, das Hinzufügen einer Kante kann sie verringern.

Definition 2.10. Eine *Brücke* (engl. bridge) in einem Graphen $G = (V, E)$ ist eine Kante $e \in E$, deren Entfernen die Anzahl der Zusammenhangskomponenten erhöht, also $|C(G)| < |C(H)|$ mit $H = (V, E \setminus e)$, wobei $C(G)$ die Menge der Zusammenhangskomponenten von G ist.

Warum das wichtig ist

Brücken sind beim Netzwerkentwurf und bei der Netzwerkanalyse wichtig. Sie stellen kritische Verbindungen dar, deren Entfernen das Netzwerk zerlegen kann. Im Internet können Brücken etwa entscheidende Verbindungen zwischen verschiedenen Netzwerken oder Regionen darstellen. Brücken zu identifizieren und zu schützen ist wesentlich für Zuverlässigkeit und Robustheit eines Netzwerks. Auch in Lieferketten ist das Auffinden von Brücken im Liefernetzwerk für das Risikomanagement höchst relevant.

Definition 2.11. Der *Abstand* (engl. distance) $d(u, v)$ zwischen zwei Knoten $u, v \in V$ eines Graphen $G = (V, E)$ ist die Länge des kürzesten Pfades zwischen u und v, also $d(u, v) = \min_{W_{u,v}} \{\mathcal{L}(W_{u,v}) \mid W_{u,v}$ ist ein Pfad von u nach $v\}$.

Definition 2.12. Der *Durchmesser* (engl. diameter) eines Graphen $G = (V, E)$ ist der größte Abstand zwischen je zwei seiner Knoten. Er wird mit $d(G) = \max_{u,v \in V} d(u, v)$ bezeichnet.

Kehren wir zum bisher Gesagten zurück: Wir haben *Wanderungen*, *Wege* und *Pfade* definiert. Nun können wir *Kreise* formaler definieren.

Definition 2.13. Ein *Kreis* ist ein geschlossener Weg $S_V = (v_0, v_1, \ldots, v_{n-1}, v_0)$ mit $\forall_{i,j \in \{1,2,\ldots,n-1\}} : i \neq j \Rightarrow v_i \neq v_j \wedge v_i \neq v_0$.

Die Länge des kürzesten Kreises in einem Graphen G heißt *Taillenweite* (engl. girth) und wird mit $g(G)$ bezeichnet; die Länge des längsten Kreises heißt *Umfang* (engl. circumference) und wird mit $c(G)$ bezeichnet. Für Graphen ohne Kreise gelten definitionsgemäß $g(G) = \infty$ und $c(G) = 0$.

Satz 2.14. *Jeder ungerichtete Graph $G = (V, E)$ besitzt einen Pfad mit mindestens der Länge $\delta(G)$.*

Beweis. Nehmen wir einen *längsten* Pfad $S_V = (v_0, v_1, \ldots, v_n)$ in G an, wobei n seine Länge ist. Da S_V ein längster Pfad ist, muss jeder Nachbar von v_n darin liegen. Andernfalls wäre der Pfad nicht der längste, da wir ihn einfach durch Anhängen dieses Nachbarn verlängern könnten. Daher gilt $n \geq deg(v_n) \geq \delta(G)$. □

Satz 2.15. *Jeder ungerichtete Graph G mit $\delta(G) \geq 2$ besitzt einen Kreis mit mindestens der Länge $\delta(G) + 1$.*

Beweis. Aus Satz 2.14 wissen wir, dass ein Pfad mit mindestens der Länge $\delta(G)$ existiert. Nehmen wir erneut einen längsten Pfad $S_V = (v_0, v_1, \ldots, v_n)$. Der Knoten v_n hat mindestens zwei Nachbarn, da $\delta(G) \geq 2$ vorausgesetzt ist. Wählen wir $i \in \mathbb{N}$ mit $i < n$ als kleinsten Index mit $v_i \sim v_n$. Dann ist v_i der Nachbar von v_n, den wir auf dem Pfad zuerst besuchen. Durch Rückkehr von v_n nach v_i können wir den Kreis $C = (v_i, v_{i+1}, \ldots, v_n, v_i)$ bilden. Dieser enthält *alle* Nachbarn von v_n, da i den ersten Nachbarn bezeichnet. Zusätzlich besitzt C eine weitere Kante, die ihn schließt. Daher muss seine Länge mindestens $\delta(G) + 1$ sein. □

Wie wir bei der Definition von Zusammenhangskomponenten gesehen haben, verlangten wir eine symmetrische Relation. Bei gerichteten Graphen ist Symmetrie jedoch nicht garantiert. Existiert ein Pfad von u nach v, können wir nicht annehmen, dass auch ein $v - u$-Pfad existiert. Daher benötigen wir für gerichtete Graphen eine andere Definition von Zusammenhang.

Definition 2.16. Aus einem gerichteten Graphen $G = (V, E)$ können wir einen ungerichteten Graphen $H = (V, X)$ mit $X = \{\{u, v\} \in [V]^2 \mid (u, v) \in E \vee (v, u) \in E\}$ konstruieren. G heißt *schwach zusammenhängend* (engl. weakly connected), wenn H zusammenhängend ist.

In einem schwach zusammenhängenden gerichteten Graphen können wir nicht von jedem Knoten jeden anderen erreichen. Wollen wir diesen stärkeren Zusammenhangsbegriff ausdrücken, müssen wir auch eine stärkere Definition für gerichtete Graphen einführen.

Definition 2.17. Ein gerichteter Graph $G = (V, E)$ heißt *stark zusammenhängend* (engl. strongly connected), wenn für jedes Knotenpaar $u, v \in V$ sowohl ein Pfad von u nach v als auch ein Pfad von v nach u existiert.

Entsprechend können wir eine Äquivalenzrelation R bilden, indem wir beide Pfade verlangen und damit ausdrücklich Symmetrie festlegen. Die Äquivalenzklassen heißen dann *starke Zusammenhangskomponenten* (engl. strongly connected components).
Beachten Sie, dass die Definition weder sagt, wie wir die Komponenten erhalten, noch, ob wir sie effizient berechnen können. Dafür benötigen wir Algorithmen.

Innehalten und nachdenken – Programmieraufgabe

Fällt Ihnen ein Algorithmus ein, um alle Zusammenhangskomponenten eines gerichteten Graphen zu finden? Versuchen Sie es selbst, implementieren Sie ihn und prüfen Sie, ob er funktioniert und wie schnell er ist. Vielleicht finden Sie einen völlig neuen und hocheffizienten Ansatz, den die Welt noch nicht kennt! Erzählen Sie der Welt davon! Vielleicht haben Sie keinen Algorithmus gefunden oder er arbeitet eher schlecht – das macht nichts. Sie haben trotzdem viel über das Problem und seine Herausforderungen gelernt, und dieses Wissen hilft Ihnen, vorhandene Algorithmen zu verstehen.
Sie sind nicht überzeugt, dass neue Algorithmen erfunden werden können? Graphentheorie ist weiterhin ein aktives Forschungsgebiet, und noch immer werden effizientere Algorithmen entdeckt. Ein Beispiel ist ein 2023 veröffentlichter deterministischer Algorithmus mit nahezu linearer Laufzeit für exakten maximalen und kostenminimalen Fluss in gerichteten Graphen mit polynomiell beschränkten ganzzahligen Nachfragen, Kosten und Kapazitäten [2]. Vielleicht möchten Sie aber zuerst Ihr Verständnis der Graphentheorie vertiefen.

2.1.1 Tarjans Algorithmus zur Bestimmung starker Zusammenhangskomponenten

Tarjans Algorithmus in Algorithmus 1 findet effizient alle starken Zusammenhangskomponenten. Diese helfen uns, Kreise in einem gerichteten Graphen zu lokalisieren. In manchen Anwendungen sind Kreise unerwünscht, etwa bei der Planung von Aufgaben mit Abhängigkeiten. Enthält der Abhängigkeitsgraph einen Kreis, können wir keinen gültigen Ablaufplan finden. Solche Graphen treten etwa bei Datenbanktransaktionen oder Webdiensten auf.

Algorithmus 1 Tarjans Algorithmus zur Bestimmung starker Zusammenhangskomponenten

Eingabe: Ein gerichteter Graph $G = (V, E)$
Ausgabe: Die starken Zusammenhangskomponenten von G

1: $index \leftarrow 0$
2: $S \leftarrow$ ein leerer Stapel
3: $\forall_{v \in V}\ node_{index}[v] \leftarrow$ undefined
4: $\forall_{v \in V}\ node_{link}[v] \leftarrow$ undefined
5: $onStack \leftarrow \{\}$ ▷ Eine leere Menge
6: $SCCs \leftarrow ()$ ▷ Eine leere Liste von Listen – das Ergebnis
7: **für** $v \in V$ **tue**
8: **wenn** $node_{index}[v] =$ undefined **dann**
9: StrongConnect(v)
10: **Ende wenn**
11: **Ende für**
12: **Funktion** StrongConnect(v)
13: $node_{index}[v] \leftarrow index$
14: $node_{link}[v] \leftarrow index$
15: $index \leftarrow index + 1$
16: push(S, v)
17: $onStack \leftarrow onStack \cup \{v\}$
18: **für** $(v, w) \in E$ **tue** ▷ Alle Nachfolger von v
19: **wenn** $node_{index}[w] =$ undefined **dann**
20: StrongConnect(w)
21: $node_{link}[v] \leftarrow \min(node_{link}[v], node_{link}[w])$
22: **sonst wenn** $w \in onStack$ **dann**
23: $node_{link}[v] \leftarrow \min(node_{link}[v], node_{index}[w])$
24: **Ende wenn**
25: **Ende für**
26: **wenn** $node_{link}[v] = node_{index}[v]$ **dann**
27: $scc \leftarrow ()$ ▷ Eine leere Liste
28: **wiederhole**
29: $w \leftarrow \text{pop}(S)$
30: $onStack \leftarrow onStack \setminus \{w\}$
31: $scc \leftarrow scc \circ (w)$
32: **bis** $w = v$
33: $SCCs \leftarrow SCCs \circ (scc)$
34: **Ende wenn**
35: **Ende Funktion**
36: **gib zurück** $SCCs$

Der Algorithmus ist rekursiv und enthält eine Tiefensuche (DFS). Sein Rekursionsstapel verfolgt den aktuellen DFS-Pfad. Der separate Stapel S enthält entdeckte Knoten, die noch keiner Komponente zugewiesen sind; ein Knoten kann nach Abschluss seines rekursiven Aufrufs in S bleiben. Die Arrays speichern Entdeckungsindizes und Lowlink-Werte ($node_{link}$ in Algorithmus 1). Ein Lowlink fasst zusammen, wie weit zurück der DFS-Teilbaum einen noch in S liegenden Knoten erreichen kann, über Baumkanten gefolgt von höchstens einer Kante zu diesem Stapel. Wird ein Knoten gefunden, dessen Index mit dem kleinsten erreichbaren Index übereinstimmt, ist eine starke Zusammenhangskomponente identifiziert, und alle ihre Knoten werden vom Stapel entfernt.

Beispiel: zwei Kreise in einer Komponente. Betrachten wir den Graphen in Abbildung 5. Er enthält die Kreise $A \to B \to C \to D \to A$ und $A \to D \to A$. Die Knoten A und D können einander erreichen, bilden aber keine eigene starke Zusammenhangskomponente: Eine Komponente enthält *alle* Knoten, die mit ihren Mitgliedern wechselseitig erreichbar sind. Hier gehören B und C zur selben Komponente, da wir über den längeren Kreis von jedem der vier Knoten zu jedem anderen gelangen. Die einzige starke Zusammenhangskomponente ist somit $\{A, B, C, D\}$.

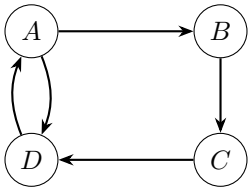


Abbildung 5: Zwei Kreise mit gemeinsamen Knoten A und D. Alle vier Knoten gehören zu einer starken Zusammenhangskomponente.

Starten wir die DFS bei A und erkunden $A \to B$ vor $A \to D$. Die Knoten erhalten in der Reihenfolge A, B, C, D die Indizes $0, 1, 2, 3$. Wenn D die Kante zurück zu A findet, wird sein Lowlink 0. Bei der Rückkehr durch die rekursiven Aufrufe wird dieser Wert an C und dann an B weitergegeben:

Schritt	Lowlink-Aktualisierung	Stapel S (unten nach oben)
Entdecke A, B, C, D	Anfangs $0, 1, 2, 3$	(A, B, C, D)
Untersuche $D \to A$	$node_{link}[D] \leftarrow 0$	(A, B, C, D)
Kehre von D zu C zurück	$node_{link}[C] \leftarrow 0$	(A, B, C, D)
Kehre von C zu B zurück	$node_{link}[B] \leftarrow 0$	(A, B, C, D)
Beende A	Lowlink und Index sind beide 0	$()$

Die Aufrufe für D, C und B enden, ohne diese Knoten aus S zu entfernen: Ihre Lowlinks sind kleiner als ihre eigenen Indizes. Zurück bei A ändert die verbleibende Kante $A \to D$ nichts. Erst wenn A endet, stimmen Lowlink und Index überein; der Algorithmus entfernt daher D, C, B, A gemeinsam vom Stapel und liefert eine Komponente.
Analysieren wir die Zeitkomplexität von Tarjans Algorithmus. Er führt eine Tiefensuche (DFS) im Graphen durch, die $O(V + E)$ Zeit benötigt, wobei V die Knotenanzahl und E die Kantenanzahl ist. Während des DFS-Durchlaufs wird jeder Knoten genau einmal besucht und jede Kante genau einmal verfolgt.

Hinweis zur Implementierung

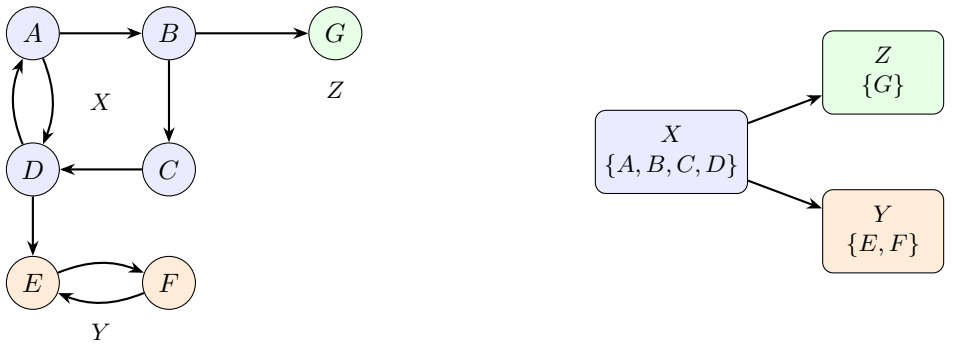
$O(V+E)$ ist linear in der Eingabegröße, wenn der Graph durch Adjazenzlisten dargestellt wird. Für dünn besetzte Graphen (engl. sparse graphs) gilt $E = O(V)$, daher ist die Laufzeit auch linear in der Knotenanzahl. Bei dichten Graphen gilt $E = \Theta(V^2)$; sie ist damit quadratisch in der Knotenanzahl, bleibt aber linear in der Größe der Adjazenzlisteneingabe. Das Durchgehen einer vollständigen Adjazenzmatrix kostet dagegen $O(V^2)$.
Warum verschwindet der V-Anteil in der O-Notation nicht? Warum sagen wir nicht einfach, dass der Algorithmus $O(E)$ benötigt? Weil wir ausdrücken wollen, dass wir selbst bei einem Graphen ganz ohne Kanten alle Knoten mindestens einmal besuchen müssen.

2.1.2 Topologische Sortierung

An der Ausgabe von Tarjans Algorithmus können wir eine weitere Eigenschaft erkennen. Betrachten wir die Folge der zurückgegebenen Komponenten als $(c_1, c_2, \ldots, c_n)$, dann kann jeder Knoten in c_1 nur Kanten zu anderen Knoten in c_1 haben, nicht zu Knoten in c_2 oder höher. Ein Knoten der zweiten zurückgegebenen Komponente c_2 kann nur Kanten zu Knoten in c_1 und c_2 haben, nicht zu c_3 oder höher. Allgemein kann ein Knoten in Komponente c_i mit $1 \leq i \leq n$ nur Kanten zu Knoten einer Komponente c_j mit $j \leq i$ haben.

Um diese Ordnung deutlicher zu sehen, ersetzen wir jede starke Zusammenhangskomponente durch einen Knoten. Zwischen zwei solchen Knoten zeichnen wir eine Kante, wann immer eine Kante im ursprünglichen Graphen von der einen Komponente zur anderen führt. Der entstehende *Kondensationsgraph* ist ein DAG: Ein Kreis zwischen Komponenten würde sie wechselseitig erreichbar machen, sodass sie eine einzige Komponente bildeten. Eine *topologische Ordnung* listet die Knoten eines DAG so auf, dass jede Kante von einem früheren zu einem späteren Knoten zeigt. Tarjan liefert die Komponenten in umgekehrter topologischer Ordnung; die Umkehrung dieser Liste ergibt eine topologische Ordnung des Kondensationsgraphen.

Beispiel: Komponenten ordnen. Erweitern wir den vorherigen Graphen um $D \to E$, $E \to F$, $F \to E$ und $B \to G$, wie in Abbildung 6. Seine Komponenten sind $X = \{A, B, C, D\}$, $Y = \{E, F\}$ und $Z = \{G\}$. Der Kondensationsgraph besitzt nur zwei Kanten, $X \to Y$ und $X \to Z$. Daher muss X zuerst kommen, während Y und Z in beliebiger Reihenfolge auftreten dürfen: Sowohl (X, Y, Z) als auch (X, Z, Y) sind gültige topologische Ordnungen. Der ursprüngliche Graph hat Kreise, daher können wir seine Komponenten so ordnen, aber nicht seine einzelnen Knoten topologisch sortieren.



(a) Ursprünglicher Graph mit Komponenten X, Y, Z. (b) Kondensationsgraph: X kommt zuerst.

Abbildung 6: Das Ersetzen jeder starken Zusammenhangskomponente durch einen Knoten ergibt einen DAG. Sowohl (X, Y, Z) als auch (X, Z, Y) sind topologische Ordnungen.

Startet Tarjan beispielsweise bei A, folgt zuerst dem längeren Kreis und erkundet $D \to E$ vor der Rückkehr zu B, beendet er zuerst Y. Dann besucht er von B aus G und beendet Z, bevor er schließlich X zurückgibt. Seine Ausgabe (Y, Z, X) wird durch Umkehren zur gültigen topologischen Ordnung (X, Z, Y).
Wissen wir jedoch, dass der Graph ein DAG ist, können wir einen einfacheren Algorithmus für die topologische Ordnung verwenden, beziehungsweise für die umgekehrte Ordnung bei Umkehrung aller Kantenrichtungen. Algorithmus 2 zeigt eine Variante von Kahns Algorithmus. Er ist konzeptionell einfach, aber nicht so leistungsfähig wie Tarjans Algorithmus, und seine Komplexität bleibt $O(V + E)$.

Auf dem Kondensationsgraphen in Abbildung 6 beginnt Kahns Algorithmus mit $S = \{X\}$, da nur X Eingangsgrad null besitzt. Nach Entfernen von X und seinen ausgehenden Kanten haben sowohl Y als auch Z Eingangsgrad null. Die nächste Wahl von Y ergibt (X, Y, Z); die nächste Wahl von Z ergibt (X, Z, Y).

Innehalten und nachdenken – Selbst erarbeiten

Was meinen Sie: Ist die topologische Ordnung jedes DAG eindeutig oder gibt es mehrere Ordnungen? Können Sie sich einen nichttrivialen DAG mit eindeutiger topologischer Ordnung vorstellen? Können Sie einen DAG mit mehreren topologischen Ordnungen zeichnen?

Algorithmus 2 Topologische Sortierung (Toposort) für DAGs

Eingabe: Ein gerichteter azyklischer Graph $G = (V, E)$
Ausgabe: Eine topologische Ordnung der Knoten in G

```
1:  L ← ()                                          ▷ Eine leere Liste – das Ergebnis
2:  ∀_{v∈V} id(v) ← indeg(v)                        ▷ Zuordnung der Eingangsgrade
3:  S ← {v ∈ V | id(v) = 0}                         ▷ Alle Knoten ohne eingehende Kanten
4:  solange S ≠ ∅ tue
5:      Wähle ein beliebiges v ∈ S
6:      S ← S \ {v}
7:      L ← L ∘ (v)
8:      für (v, u) ∈ E tue                          ▷ Alle Nachfolger von v
9:          id(u) ← id(u) − 1
10:         wenn id(u) = 0 dann
11:             S ← S ∪ {u}
12:         Ende wenn
13:     Ende für
14: Ende solange
15: wenn |{v ∈ V | id(v) > 0}| > 0 dann
16:     Fehler: Der Graph besitzt mindestens einen Kreis
17: Ende wenn
18: gib zurück L
```

3 Hamiltonsche und eulersche Graphen

3.1 Hamiltonsche Graphen

Hamiltonsche Graphen sind nach dem irischen Mathematiker Sir William Rowan Hamilton benannt, der im 19. Jahrhundert unter anderem das als Icosian Game bekannte Rätsel erfand. Das Spiel zeigte das Dodekaeder (Abbildung 7), einen besonderen Graphen. Die Herausforderung war, darin etwas zu finden, das später Hamiltonkreis genannt wurde.

Definition 3.1. Ein *hamiltonscher Pfad* (engl. Hamiltonian path) in einem Graphen $G = (V, E)$ ist ein Pfad, der jeden Knoten genau einmal besucht.

Definition 3.2. Ein *hamiltonscher Zyklus* (engl. Hamiltonian cycle), auch Hamiltonkreis, in einem Graphen $G = (V, E)$ ist ein Kreis, der jeden Knoten genau einmal besucht.

Ein Graph mit einem Hamiltonkreis heißt *hamiltonsch* (engl. Hamiltonian). Einen solchen Kreis zu finden erscheint möglicherweise harmlos und ist für manche Graphen recht einfach. Bei K_n etwa bildet jede der $n!$ Knotenpermutationen einen Hamiltonkreis. Auch bei bestimmten Gittern sind solche Kreise leicht zu finden. Für allgemeine Graphen gibt es jedoch keinen bekannten effizienten Algorithmus, der entscheidet, ob ein Graph hamiltonsch ist, oder einen solchen Kreis findet. Die Entscheidung, ob in einem allgemeinen Graphen ein hamiltonscher *Pfad* existiert, ist NP-vollständig; einen zu finden ist das entsprechende Suchproblem.
Mehrere bekannte Rätsel und Probleme beruhen auf Hamiltonkreisen, etwa das Rasenmäherproblem oder das Problem des Handlungsreisenden (TSP). Das TSP ist besonders schwierig, da nicht nur ein Hamiltonkreis gefunden werden soll, sondern unter allen solchen Kreisen derjenige mit minimalen Kosten, etwa Distanz, Zeit oder Geld, summiert über alle besuchten Kantengewichte. Die Lösung des TSP gehört zu den am intensivsten untersuchten Optimierungsproblemen. Es gibt viele Heuristiken und Approximationsalgorithmen, dennoch bleibt es bis heute eine Herausforderung.

Es gibt einige *hinreichende Bedingungen*, die die Existenz eines Hamiltonkreises in einem Graphen garantieren. Eine solche Bedingung liefert der Satz von Dirac.

Satz 3.3 (Satz von Dirac)**.** *Sei $G = (V, E)$ ein einfacher Graph mit $|V| = n \geq 3$.*
$\delta(G) \geq \frac{n}{2} \Rightarrow G$ ist hamiltonsch.

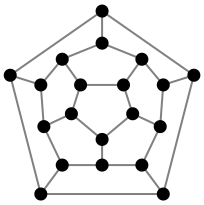


Abbildung 7: Der Dodekaedergraph, der in Hamiltons Icosian Game verwendet wurde.

Hintergrund

Eine Bedingung heißt *hinreichend*, wenn sie die Eigenschaft garantiert, sobald sie erfüllt ist. Die Eigenschaft kann aber auch gelten, wenn die Bedingung nicht erfüllt ist. Beim 10-Kreis gilt etwa $\delta(C_{10}) = 2 \not\geq \frac{10}{2}$, dennoch ist er definitionsgemäß hamiltonsch.
Eine Bedingung heißt *notwendig*, wenn die Eigenschaft ohne sie nicht gelten kann; die Bedingung wird also benötigt. Die Eigenschaft muss aber selbst bei erfüllter Bedingung nicht gelten. Beispielsweise ist $n \geq 3$ eine notwendige Bedingung im Satz von Dirac: Ein Graph mit weniger als drei Knoten kann nicht hamiltonsch sein. Hinreichend und notwendig hängen mit logischen Implikationen zusammen: Bei $P \Rightarrow Q$ ist P eine hinreichende Bedingung für Q, bei $P \Leftarrow Q$ eine notwendige. Gelten beide Richtungen, also $P \Leftrightarrow Q$, ist die Bedingung notwendig und hinreichend.

Innehalten und nachdenken – Kurzcheck

Warum besitzt K_2 keinen Kreis?

Die Existenz eines Hamiltonpfads hängt auch mit topologischer Sortierung zusammen.

Satz 3.4. *Ein gerichteter azyklischer Graph (DAG) besitzt genau dann eine eindeutige topologische Ordnung, wenn er einen Hamiltonpfad enthält.*

Damit ist die Entscheidung, ob ein DAG einen Hamiltonpfad besitzt, recht einfach, denn eine topologische Ordnung lässt sich, wie bereits gesehen, effizient berechnen. Anhand dieser Ordnung prüfen wir, ob zwischen jedem aufeinanderfolgenden Knotenpaar eine Kante existiert. Ist dies der Fall, haben wir einen Hamiltonpfad gefunden und wissen, dass keine andere topologische Ordnung existiert. Andernfalls wissen wir, dass mehrere topologische Ordnungen existieren und wir keinen Hamiltonpfad finden können.

Sei $T = (v_1, v_2, \ldots, v_n)$ die topologische Ordnung eines DAG $G = (V, E)$ mit n Knoten. Gilt $\forall_{i \in \{1,2,\ldots,n-1\}} : (v_i, v_{i+1}) \in E$, dann ist T ein Hamiltonpfad in G und die topologische Ordnung ist eindeutig.

3.1.1 Problem des Handlungsreisenden

Das Problem des Handlungsreisenden (TSP) ist ein bekanntes Optimierungsproblem der Graphentheorie und kombinatorischen Optimierung. Formal lässt es sich wie folgt definieren:

Definition 3.5 (Problem des Handlungsreisenden)**.** Gegeben sei ein gewichteter hamiltonscher Graph $G = (V, E)$ mit Gewichtsfunktion $w : E \to \mathbb{R}^+$. Ziel ist, einen Hamiltonkreis C zu finden, dessen Gesamtgewicht der Kanten minimal ist, also minimiere $\sum_{e \in C} w(e)$.

Wie besprochen kann schon das Finden eines einzigen Hamiltonkreises schwierig sein, insbesondere in einem dünn besetzten Graphen. Den günstigsten zu finden ist noch anspruchsvoller. In einem vollständigen Graphen gibt es $\frac{(n-1)!}{2}$ mögliche Hamiltonkreise, die ein Brute-Force-Algorithmus aufzählen könnte. Dies ist keine Anforderung an jeden exakten Algorithmus. Das TSP ist NP-schwer; es ist kein Polynomialzeitalgorithmus bekannt, der alle Instanzen löst. Schnelle Näherungsalgorithmen zu finden bleibt eine große Herausforderung. Diese oft aus mehreren

Techniken kombinierten Algorithmen sind praktisch sehr nützlich, da das TSP viele Anwendungen in Logistik und Fertigung besitzt.

3.2 Eulersche Graphen

Leonhard Euler (1707-1783) gilt als Begründer der Graphentheorie, und das Problem der „Sieben Brücken von Königsberg“ wird oft als erstes Problem der Graphentheorie genannt. Es beruht auf der Stadt Königsberg, heute Kaliningrad in Russland, die vom Fluss Pregel geteilt wurde und im Jahr 1736 sieben Brücken zwischen ihren Teilen besaß. Die Frage lautete: *Kann man durch die Stadt gehen und dabei jede Brücke genau einmal überqueren*?

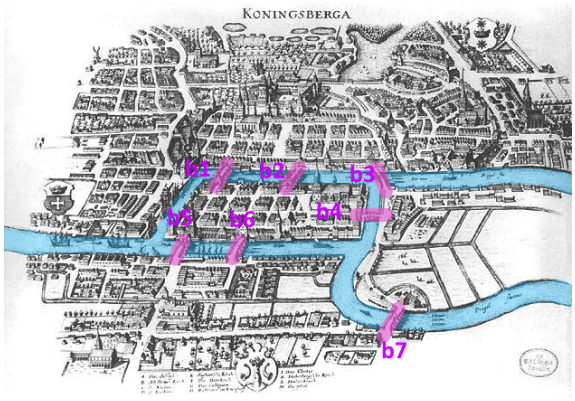


Abbildung 8: Die sieben Brücken von Königsberg (Quelle: Wikimedia Commons, Autor: User:Twotwos, 17. September 2025), verändert durch Hinzufügen von Brückenbezeichnungen.

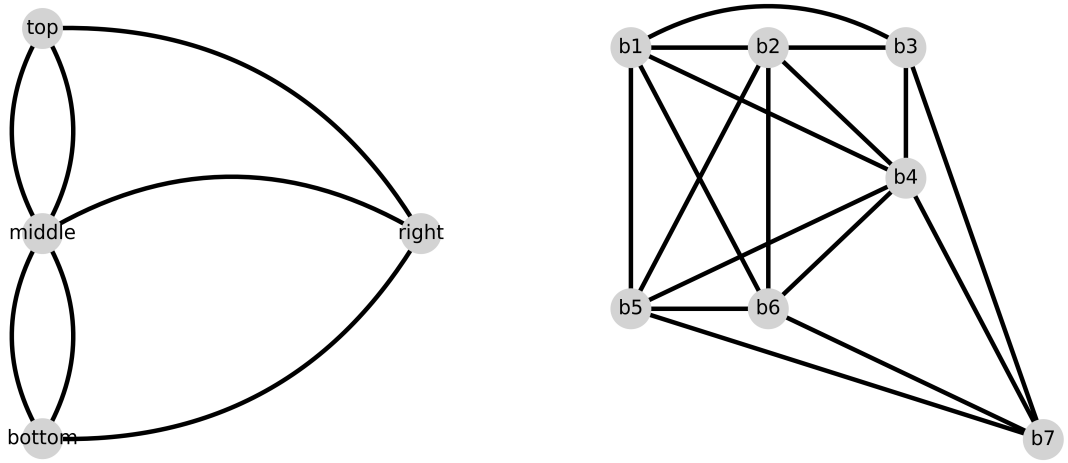
Zunächst musste Euler das Problem abstrahieren und ein mathematisches Modell erstellen. Das war die Geburtsstunde der Graphentheorie. In Abbildung 8 erkennen wir vier Landmassen: mittlere Insel, rechte Insel, oberes und unteres Flussufer. Die sieben Brücken (b_1 bis b_7) verbinden diese Landmassen. Abstraktion bedeutet, für das Problem unwichtige Details zu ignorieren und dann eine mathematische Beschreibung oder ein Modell zu entwickeln. Offensichtlich sind weder Häuser, Stadtmauern und Straßen auf den Landmassen noch Brückenlängen oder Flussform relevant. Dennoch haben wir mehrere Möglichkeiten, und ich möchte sowohl ein korrektes als auch ein falsches Modell dieses Problems zeigen. In der Praxis ist eine sinnvolle und korrekte Abstraktion nicht immer naheliegend oder einfach.

In Abbildung 9a modellieren wir die vier Landmassen als Knoten und die sieben Brücken als Kanten; nennen wir dies *Modell A*. In Abbildung 9b wählen wir die Brücken als Knoten und die Existenz eines Pfades zwischen ihnen, der keine Brücke überquert, als Kanten; nennen wir dies *Modell B*.

Definition 3.6. Ein ungerichteter Multigraph mit möglichen Schleifen ist ein Tripel $G = (V, E, i)$, wobei V und E endliche Mengen sind und $i : E \to [V]^2 \cup [V]^1$ jeder Kante ihre Endpunkte zuweist. Dabei ist $[V]^1$ die Menge der einelementigen Teilmengen von V. Verschiedene Kanten können dieselben Endpunkte haben; eine einelementige Menge beschreibt eine Schleife.

Modell A verwendet diese Multigraphkonvention: Verschiedene Brücken können dieselben Landmassen verbinden. Um stattdessen mit einem einfachen Graphen zu arbeiten, setzen wir in die Mitte jeder Brücke einen neuen Knoten und ersetzen die Brücke durch zwei Kanten. Eine Kante $\{u, v\}$ durch einen Pfad $u - x - v$ über einen neuen Knoten x zu ersetzen heißt, die Kante zu *unterteilen*. Jede Brücke erhält ihren eigenen Mittelpunkt, sodass der entstehende Graph keine parallelen Kanten besitzt. Jeder Mittelpunkt hat Grad zwei, und ein Durchlauf, der ihn betritt, muss ihn über seine andere Kante verlassen. Das Überqueren dieser zwei Kanten entspricht daher dem Überqueren der ursprünglichen Brücke, sodass die folgenden Algorithmen auch auf diesen Graphen angewendet werden können.

Zunächst scheinen beide Modelle gültig zu sein. In Modell A suchen wir eine Möglichkeit, jede Kante genau einmal zu durchlaufen; in Modell B suchen wir scheinbar eine Möglichkeit, jeden Knoten genau einmal zu besuchen. Das haben wir doch gerade gelernt, oder? Das ist ein Hamiltonkreis! Selbst wenn das ein schwieriges Problem ist, erscheint es hier einfach. Versuchen wir diesen Ansatz also zuerst mit Modell B. Dort finden wir leicht einen Hamiltonkreis, zum Beispiel



(a) Modell A

(b) Modell B

Abbildung 9: Verschiedene Graphen für die sieben Brücken von Königsberg

$(b_1, b_5, b_7, b_6, b_4, b_3, b_2, b_1)$. Großartig! Können wir also schließen: Ja, eine Tour über jede Brücke genau einmal ist möglich? Analysieren wir die Tour: Offenbar starten wir am oberen Ufer, überqueren b_1 zur mittleren Insel, dann b_5 zum unteren Ufer, dann b_7 zur rechten Insel und anschließend b_6, welche ... nun ja, die mittlere Insel mit dem unteren Ufer verbindet, aber wir sind auf der rechten Insel. Das geht nicht. Wir können b_6 an dieser Stelle nicht überqueren. Ist also die Kante zwischen b_6 und b_7 falsch? Nein: Wenn wir von der rechten Insel zum unteren Ufer gegangen wären, wäre es in Ordnung. Brauchen wir also einen gerichteten Graphen? Aber die Kante würde weiterhin in beide Richtungen zeigen. Vielleicht möchten Sie nun einfach einen anderen Hamiltonkreis finden – war nur dieser falsch? Sie werden jedoch feststellen, dass jeder Hamiltonkreis in Modell B dasselbe Problem besitzt: Eine bestimmte Brücke lässt sich nicht einfach überqueren. In Modell B haben wir wichtige Informationen über die Bewegungsrichtung verloren, und diese „Richtung“ lässt sich nicht einfach mit einem gerichteten Graphen modellieren. Die Erkenntnis ist: Brücken als Knoten bedeuten, dass wir die Brücke nicht tatsächlich überqueren müssen. Wir haben modelliert, sie nur zu besuchen, etwa sie zu berühren; das heißt aber nicht, sie zu überqueren. Modell B ist daher nicht die richtige Abstraktion des ursprünglichen Problems. Anders gesagt: Auf Grundlage von Modell B können wir die Frage nicht beantworten.

Innehalten und nachdenken – Kurzcheck

Auch wenn Modell B falsch ist, sollten Sie anhand der obigen Argumentation verstehen, *warum* es falsch ist. Aus falschen Modellen können wir viel lernen, wenn wir zumindest erkennen, dass sie falsch sind, und den Grund korrekt herleiten. Andernfalls machen wir dieselben Fehler in einem anderen Kontext erneut.

Modell B reparieren. Wir können den Hamiltonkreis-Ansatz zum Funktionieren bringen, indem wir jede Brücke durch drei neue Knoten statt durch einen darstellen. Für jede ursprüngliche Kante $e = \{u, v\}$ erzeugen wir einen Pfad $u_e - m_e - v_e$. Seine Endpunkte stellen die beiden Brückenenden dar; Endpunkte verschiedener Brücken bleiben separate Knoten. Dann verbinden wir jedes Paar von Brückenendpunkten auf derselben Landmasse. Die ursprünglichen Landmassenknoten werden nicht mehr benötigt.

Der mittlere Knoten m_e hat genau zwei Nachbarn, daher muss ein Hamiltonkreis beide Kanten von $u_e - m_e - v_e$ verwenden. Dies zwingt ihn, die gesamte Brücke zu überqueren. Die Verbindungen zwischen den Brückenendpunkten erlauben anschließend, über eine andere Brücke derselben Landmasse weiterzugehen. Ein Hamiltonkreis im neuen Graphen beschreibt somit einen eulerschen Kreis im ursprünglichen Graphen. Umgekehrt ergibt ein eulerscher Kreis einen Hamiltonkreis, indem man

dem Dreiknotenpfad jeder Brücke folgt und aufeinanderfolgende Brücken an ihrer gemeinsamen Landmasse verbindet.

Dies ist ein korrektes Modell, hilft aber wenig bei der Suche nach einer Tour: Bei m ursprünglichen Brücken besitzt es $3m$ Knoten sowie die Kanten zwischen Endpunkten auf jeder Landmasse. Wir müssten weiterhin ein Hamiltonkreisproblem lösen. Direkt mit Modell A zu arbeiten liefert eine viel einfachere Methode, wie wir gleich sehen werden.

Wenden wir uns also Modell A zu. Vielleicht können wir das Problem dort lösen. In Modell A suchen wir einen Weg, der jede Kante genau einmal besucht. Ist das möglich? Euler bewies anhand von Modell A, dass ein solcher Weg nicht existieren kann.

Satz 3.7. *Ein ungerichteter zusammenhängender Graph $G = (V, E)$ enthält genau dann einen eulerschen Weg, also einen offenen Weg, der jede Kante genau einmal besucht, wenn er genau zwei Knoten ungeraden Grades besitzt. Der Weg muss an diesen beiden Knoten beginnen und enden.*

Satz 3.8. *Ein ungerichteter zusammenhängender Graph $G = (V, E)$ enthält genau dann einen eulerschen Kreis, also einen geschlossenen Weg, der jede Kante genau einmal besucht, wenn alle Knoten von G geraden Grad haben.*

Ein Graph, für den ein eulerscher Kreis existiert, heißt *eulersch*.

Beweis. Notwendigkeit. In einem geschlossenen Weg, der jede Kante verwendet, gehört zu jeder Ankunft an einem Knoten ein Verlassen über eine andere Kante. Daher ist jeder Grad gerade. Bei einem offenen eulerschen Weg hat der Start einen zusätzlichen Abgang und das Ende eine zusätzliche Ankunft; genau diese beiden Knoten haben ungeraden Grad.

Hinlänglichkeit. Angenommen, alle Grade sind gerade. Wir beginnen an einem Knoten mit einer Kante und folgen unbenutzten Kanten, bis keine Fortsetzung mehr möglich ist. Die Parität verhindert, dass wir anderswo als am Start stecken bleiben; so entsteht ein geschlossener Weg. Bleiben unbenutzte Kanten übrig, garantiert der Zusammenhang, dass ein Knoten des aktuellen Weges zu einer unbenutzten Kante inzident ist. Die unbenutzten Grade sind weiterhin gerade. Wir konstruieren dort einen weiteren geschlossenen Weg und fügen ihn in den ersten ein. Jeder Schritt verwendet weitere Kanten, daher endet das Verfahren mit einem geschlossenen eulerschen Weg. Das ist Hierholzers Idee.

Haben genau zwei Knoten a, b ungeraden Grad, fügen wir zwischen ihnen eine Hilfskante hinzu und erlauben dafür eine parallele Kante. Nun sind alle Grade gerade. Wir konstruieren einen geschlossenen eulerschen Weg und löschen die Hilfskante, wobei wir die verbleibende Folge unmittelbar nach ihr beginnen. Dies ergibt einen offenen eulerschen Weg zwischen a und b. □

Im Graphen von Modell A haben alle vier Knoten ungeraden Grad. Nach den Sätzen 3.7 und 3.8 kann daher weder ein eulerscher Kreis noch ein eulerscher Weg existieren. Das Problem hat somit keine Lösung.

Aus diesen Sätzen folgt, dass wir leicht prüfen können, ob sich in einem solchen Graphen ein eulerscher Weg oder Kreis finden lässt. Wir müssen nur alle Knotengrade betrachten und die ungeraden zählen. Gibt es mehr als zwei, existiert kein eulerscher Weg. Gibt es genau zwei, existiert ein eulerscher Weg; gibt es keinen, existiert ein eulerscher Kreis. Im Vergleich zum Hamiltonpfad- oder Hamiltonkreisproblem sind Probleme eulerscher Kreise einfach. Der Satz sagt uns jedoch nur, dass ein solcher Weg oder Kreis existiert. Um ihn tatsächlich zu finden, benötigen wir weiterhin einen Algorithmus.

3.2.1 Fleurys Algorithmus

Die Idee von Fleurys Algorithmus in Algorithmus 3 ist, an einem beliebigen Knoten beziehungsweise einem der beiden Knoten ungeraden Grades zu beginnen und die Kanten nacheinander zu durchlaufen. Wenn Sie das selbst versucht haben, bemerkten Sie vielleicht, dass man dabei stecken bleiben kann: Man wählt eine Kante, von der kein Pfad zu den verbleibenden Kanten zurückführt. Deshalb müssen wir in Fleurys Algorithmus vermeiden, *Brücken* zu wählen, solange eine andere Möglichkeit besteht.
Brücken zu vermeiden verursacht erhebliche Kosten, denn festzustellen, ob eine Kante eine Brücke ist, ist rechnerisch nicht trivial. Für jede betrachtete Kante müssen wir eine Art Zusammenhangstest durchführen, etwa eine Tiefensuche (DFS). Ansonsten entfernen wir in diesem Algorithmus in

Algorithmus 3 Fleurys Algorithmus zur Suche nach einem eulerschen Weg oder Kreis

Eingabe: Ein *zusammenhängender* ungerichteter Graph $G = (V, E)$ mit null oder zwei Knoten ungeraden Grades

Ausgabe: Ein eulerscher Weg oder Kreis in G

wenn $\exists\, v_0 \in V : \deg(v_0)$ ist ungerade **dann**
    $v \leftarrow v_0$
sonst
    $v \leftarrow$ ein beliebiger Knoten in V
Ende wenn
$F \leftarrow E$ ▷ Kopie der Kantenmenge
$K \leftarrow (v)$ ▷ Aktueller Weg, beginnend mit v
solange $F \neq \emptyset$ **tue**
    $e = \{v, u\} \leftarrow$ eine zu v inzidente Kante, die in (V, F) keine Brücke ist, außer sie ist die einzige Kante bei v
    $F \leftarrow F \setminus \{e\}$
    $v \leftarrow$ der andere Endpunkt der Kante e
    $K \leftarrow K \circ (v)$
Ende solange
gib zurück K

jeder Iteration eine Kante. Da DFS in $O(V + E)$ läuft, beträgt die Zeitkomplexität von Fleurys Algorithmus $O(E^2)$; der Anteil $V \cdot E$ wird vom Anteil E^2 dominiert und daher weggelassen.

3.2.2 Hierholzers Algorithmus

Hierholzers Algorithmus wurde 1873 veröffentlicht, nach seinem Tod 1871. Er konstruiert und verbindet Wege wie im obigen Beweis der Hinlänglichkeit, ohne Brücken zu prüfen. Sind alle Grade gerade, schließt sich ein anfänglicher maximaler Weg an seinem Start. Bei zwei Knoten ungeraden Grades beginnen wir an einem und enden am anderen; zusätzliche geschlossene Wege lassen sich an Knoten mit unbenutzten Kanten einfügen.

Algorithmus 4 implementiert dies rekursiv. Die Restkantenmenge F und die Ausgabeliste K werden von allen rekursiven Aufrufen gemeinsam verwendet. Knoten werden angehängt, nachdem ihre verbleibenden Kanten erkundet wurden. Dadurch entsteht der Weg in umgekehrter Reihenfolge, was in einem ungerichteten Graphen gültig ist. Mit Adjazenzlisten und Entfernen oder Markieren von Kanten in konstanter Zeit wird jede Kante konstant oft verarbeitet. Dies ergibt $O(E)$ Zeit für einen zusammenhängenden Graphen mit Kanten. Deshalb wird Hierholzer im Allgemeinen gegenüber Fleury bevorzugt.

3.2.3 Briefträgerproblem

Das chinesische Briefträgerproblem (CPP), auch Routeninspektionsproblem, sucht eine *geschlossene Wanderung* minimalen Gewichts, die jede Kante mindestens einmal durchläuft. Es wurde 1960 von Meigu Guan eingeführt. Postzustellung, Müllabfuhr und Straßenreinigung sind naheliegende Anwendungen: Straßen sind Kanten, Kreuzungen sind Knoten.

Sind alle Grade gerade, verwendet ein geschlossener eulerscher Weg jede Straße einmal. Andernfalls müssen manche Straßen wiederholt werden. Das Wiederholen einer Route ändert die Parität ihrer Endpunkte. Daher sollen zusätzliche Durchläufe gewählt werden, die alle Grade zu minimalen Zusatzkosten gerade machen. Straßen können unterschiedliche Längen besitzen, sodass die Wahl wichtig ist. Anders als das NP-schwere TSP-Optimierungsproblem lässt sich das zusammenhängende ungerichtete CPP mit nichtnegativen Gewichten in polynomieller Zeit lösen. Andere Varianten der Routeninspektion können NP-schwer sein.

Algorithmus 4 Hierholzers Algorithmus zur Suche nach einem eulerschen Kreis

Eingabe: Ein *zusammenhängender* ungerichteter Graph $G = (V, E)$ mit null oder zwei Knoten ungeraden Grades

Ausgabe: Ein eulerscher Weg oder Kreis in G

```
1: v ← ein Knoten ungeraden Grades, falls vorhanden, sonst ein beliebiger Knoten in V
2: F ← E                                              ▷ Kopie der Kantenmenge
3: K ← ()                                             ▷ Aktueller Weg, anfangs leer
4: Visit(V, F, v, K)
5: gib zurück K
6: Prozedur Visit(V, F, v, K)
7:     N ← {u ∈ V : {v, u} ∈ F}
8:     für u ∈ N tue
9:         wenn {v, u} ∈ F dann          ▷ Kante könnte im rekursiven Aufruf entfernt worden sein
10:            F ← F \ {{v, u}}
11:            Visit(V, F, u, K)
12:        Ende wenn
13:    Ende für
14:    K ← K ∘ (v)
15: Ende Prozedur
```

Innehalten und nachdenken – Herausforderung

Wie würden Sie an die Lösung des CPP herangehen? Können Sie selbst einen Algorithmus entwickeln? Können Sie seine Optimalität beweisen?

4 Transitive Hülle

Die transitive Hülle macht aus indirekten Verbindungen direkte: Sie fügt eine Kante von u nach v hinzu, wann immer wir v von u aus erreichen können, indem wir einer oder mehreren Kanten folgen. Dadurch können wir die Erreichbarkeit direkt aus einem Graphen oder seiner Adjazenzmatrix ablesen.

Definition 4.1. Für einen gerichteten Graphen $G = (V, E)$ ist seine *transitive Hülle* $G^+ = (V, E^+)$, wobei

$$E^+ = \{(u, v) \in V \times V \mid \text{eine Wanderung positiver Länge von } u \text{ nach } v \text{ existiert}\}.$$

Seine *reflexive transitive Hülle* erlaubt auch die triviale Wanderung und enthält daher (v, v) für jedes $v \in V$.

In einem ungerichteten Graphen können alle Knoten derselben Zusammenhangskomponente einander erreichen. Seine transitive Hülle verbindet daher jedes Paar verschiedener Knoten dieser Komponente und macht die Komponente zu einer Clique. Für die Matrixberechnung tragen wir jede ungerichtete Kante in beiden Richtungen ein. Einer Kante zu folgen und zurückzukehren ergibt ebenfalls eine Wanderung positiver Länge von einem Knoten zu sich selbst, daher ist der entsprechende Diagonaleintrag 1. In den folgenden Zeichnungen lassen wir diese Schleifen weg, damit die Verbindungen zwischen den Knoten gut erkennbar bleiben.

Hintergrund

Eine Clique ist eine Menge paarweise benachbarter Knoten. Eine größte Clique in einem beliebigen Graphen zu berechnen ist NP-schwer; zu entscheiden, ob eine Clique mit mindestens einer vorgegebenen Größe existiert, ist NP-vollständig. In dem ungerichteten Graphen, der entsteht, wenn man die Erreichbarkeitsrelation ohne Schleifen zeichnet, sind die größten Komponenten seine größten Cliquen. Dieser Spezialfall ist daher einfach. Eine Clique in diesem Graphen muss im ursprünglichen Graphen keine Clique gewesen sein.

Es gibt mehrere Algorithmen zur Berechnung der transitiven Hülle eines Graphen. Wir werden sowohl eine Lösung mittels Matrixmultiplikationen als auch den Floyd-Warshall-Algorithmus untersuchen.

4.1 Transitive Hülle mittels Matrixmultiplikation

Matrixmultiplikation haben Sie bereits im Rahmen der linearen Algebra kennengelernt. Eine Adjazenzmatrix A eines Graphen $G = (V, E)$ ist eine quadratische Matrix der Größe $|V| \times |V|$, die die Kanten des Graphen darstellt.

$$a_{ij} = \begin{cases} 1 & \text{falls } v_i \sim v_j \\ 0 & \text{sonst} \end{cases}$$

Definition 4.2. Das boolesche Produkt $C = A \odot B$ zweier Adjazenzmatrizen A und B ist definiert als $c_{ij} = \bigvee_{k=1}^{n} (a_{ik} \wedge b_{kj})$, wobei $\wedge$ die logische UND-Operation und $\vee$ die logische ODER-Operation ist.

Die boolesche Potenz A^i beschreibt Wanderungen mit genau i Kanten, einschließlich solcher, die Knoten wiederholen. Für $n = |V| \geq 1$ ergibt sich die Erreichbarkeit durch Wanderungen positiver Länge als

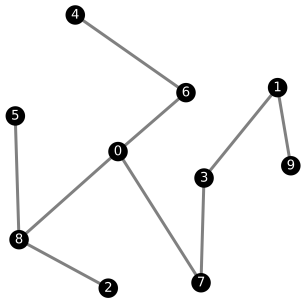
$$T^+ = A \vee A^2 \vee \cdots \vee A^n.$$

Warum schließen wir die n-te Potenz ein? Bei verschiedenen Endpunkten bleibt nach dem Entfernen geschlossener Abschnitte ein Pfad mit höchstens $n-1$ Kanten übrig. Bei gleichen Endpunkten kann eine kürzeste nichtleere Rückkehrwanderung ein Kreis der Länge n sein. Sind triviale Wanderungen erlaubt, liefert die Einheitsmatrix bereits die Diagonale, sodass gilt:

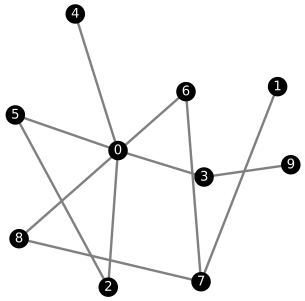
$$T^* = I \vee A \vee A^2 \vee \cdots \vee A^{n-1}.$$

Zum Beispiel besitzt im gerichteten Kreis $1 \to 2 \to 3 \to 1$ weder A noch A^2 einen von null verschiedenen Diagonaleintrag, aber es gilt $A^3 = I$. Somit fehlen in $A \vee A^2$ alle drei Rückkehrmöglichkeiten positiver Länge.

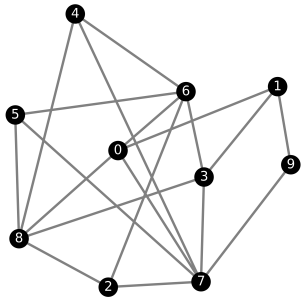
Abbildung 10 zeigt einen Beispielgraphen und seine Adjazenzmatrizen A^1 bis A^6 sowie B^1 bis B^3 eines gerichteten Graphen. Beachten Sie: Während die Knoten v_1 und v_3 im ursprünglichen ungerichteten Graphen verbunden sind, kann v_1 den Knoten v_7 und sich selbst in genau zwei Schritten erreichen. Die Zeichnungen lassen Schleifen weg, daher wird in A^2 nur die Verbindung zu v_7 dargestellt. Da v_1 und v_3 über die drei Kanten lange Wanderung (v_1, v_3, v_1, v_3) verbunden werden können, sind sie in A^3 wieder verbunden. In diesem Graphen gibt es keine Wanderung gerader Länge zwischen v_1 und v_3, daher sind sie in geraden Potenzen der Adjazenzmatrix nicht verbunden. Da in diesem ungerichteten Graphen außerdem jeder Knoten jeden anderen erreichen kann, ergibt die Darstellung seiner Erreichbarkeitsrelation ohne Schleifen einen vollständigen Graphen. Im gerichteten Graphen gibt es keine Kreise, daher wäre B^4 bereits die Nullmatrix.



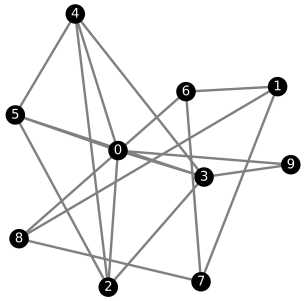
(a) Ausgangsgraph (A)



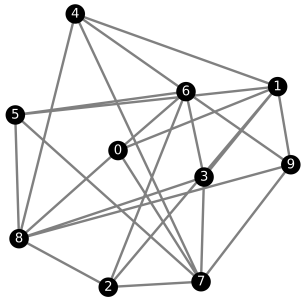
(b) Verbindungen in 2 Schritten (A^2)



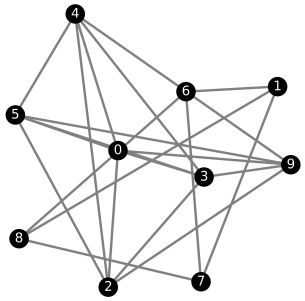
(c) Verbindungen in 3 Schritten (A^3)



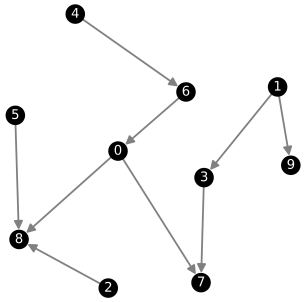
(d) Verbindungen in 4 Schritten (A^4)



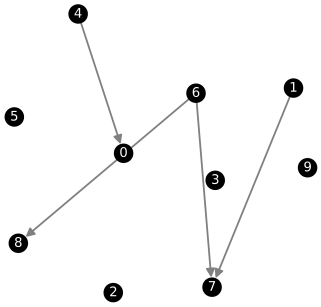
(e) Verbindungen in 5 Schritten (A^5)



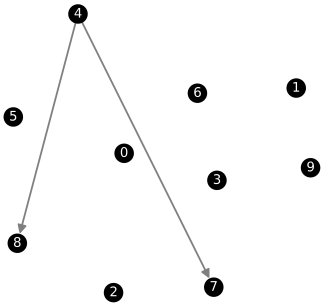
(f) Verbindungen in 6 Schritten (A^6)



(g) Ausgangsgraph (B)



(h) Verbindungen in 2 Schritten (B^2)



(i) Verbindungen in 3 Schritten (B^3)

Abbildung 10: Verbindungen außerhalb der Diagonale in den booleschen Potenzen A^1 bis A^6 und B^1 bis B^3. Es handelt sich um Graphzeichnungen; Diagonaleinträge werden nicht dargestellt.

In der Praxis ist die Berechnung der transitiven Hülle mittels Matrixmultiplikationen oft nicht effizient, insbesondere bei großen Graphen. Häufig werden effizientere Algorithmen wie der Floyd-Warshall-Algorithmus bevorzugt.

4.2 Warshall-Algorithmus

Algorithmus 5 berechnet die Adjazenzmatrix der transitiven Hülle mit drei verschachtelten Schleifen und hat damit eine Zeitkomplexität von $O(V^3)$. Seine zentrale Idee ist, längere Verbindungen zu entdecken, indem jeweils ein weiterer Zwischenknoten berücksichtigt wird. In Abschnitt 5 verwenden wir dieselbe Idee im *Floyd-Algorithmus* für kürzeste Pfade zwischen allen Knotenpaaren.

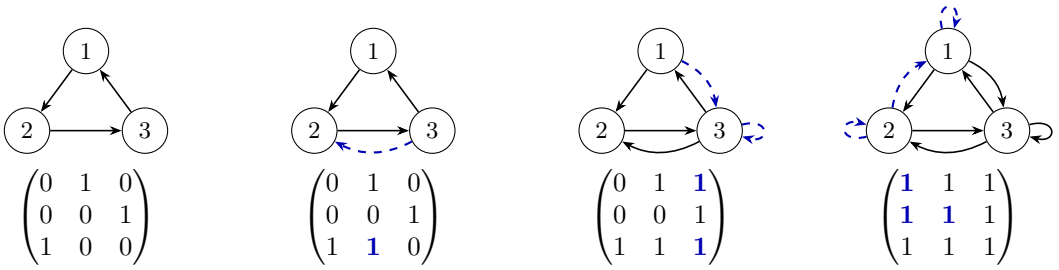
Nummerieren wir die Knoten als $v_1, \ldots, v_n$. Anfangs ist $d^{(0)}$ die Adjazenzmatrix: Wir kennen nur die direkten Kanten. Nach Stufe k gilt $d^{(k)}[i][j] = 1$, wenn es eine Wanderung positiver Länge von v_i nach v_j gibt, deren Zwischenknoten alle zu $\{v_1, \ldots, v_k\}$ gehören. Die Endpunkte dürfen beliebige Knoten sein.

In Stufe k prüfen wir, ob eine Verbindung von v_i nach v_j über v_k führen kann. Wenn wir bereits wissen, wie wir von v_i nach v_k und von v_k nach v_j gelangen, ergibt das Aneinanderhängen dieser beiden Wanderungen eine Verbindung von v_i nach v_j. Alle zuvor gefundenen Verbindungen behalten wir ebenfalls bei. Daher gilt

$$d^{(k)}[i][j] = d^{(k-1)}[i][j] \vee \big(d^{(k-1)}[i][k] \wedge d^{(k-1)}[k][j]\big).$$

Die äußere Schleife über k macht nacheinander jeden Knoten als Zwischenknoten verfügbar. Algorithmus 5 speichert die aufeinanderfolgenden Stufen in einer einzigen Matrix.

Abbildung 11 verfolgt den Algorithmus am Kreis $1 \to 2 \to 3 \to 1$. Bei $k = 1$ fügt die Wanderung $3 \to 1 \to 2$ die Verbindung $3 \to 2$ hinzu. Bei $k = 2$ finden wir $1 \to 3$ und $3 \to 3$; Letztere kombiniert die bekannte Verbindung $3 \to 2$ mit der Kante $2 \to 3$. Schließlich fügt $k = 3$ die Verbindungen $2 \to 1$, $1 \to 1$ und $2 \to 2$ hinzu. Nun kann jeder Knoten jeden Knoten einschließlich sich selbst erreichen, sodass die endgültige Matrix ausschließlich aus Einsen besteht.



(a) Ausgangsmatrix $d^{(0)}$. (b) Nach $k = 1$. (c) Nach $k = 2$. (d) Nach $k = 3$.

Abbildung 11: Warshalls Algorithmus auf einem gerichteten Dreierkreis. Gestrichelte blaue Pfeile und fette blaue Matrixeinträge zeigen die in jeder Stufe hinzugefügten Verbindungen. Zeilen und Spalten sind in der Reihenfolge $1, 2, 3$ angeordnet.

Algorithmus 5 Algorithmus zur Berechnung der transitiven Hülle

Eingabe: Ein Graph $G = (V, E)$
Ausgabe: Die Adjazenzmatrix der transitiven Hülle von G

```
1:  d[·][·] ← 0                          ▷ Adjazenzmatrix mit Nullen initialisieren
2:  für (u, v) ∈ E tue
3:      d[u][v] ← 1                      ▷ Bei ungerichteten Graphen Symmetrie sicherstellen
4:  Ende für
5:  für k ∈ V tue
6:      für i ∈ V tue
7:          für j ∈ V tue
8:              d[i][j] ← d[i][j] ∨ (d[i][k] ∧ d[k][j])
9:          Ende für
10:     Ende für
11: Ende für
12: gib zurück d
```

5 Kürzeste Pfade

Die Suche nach kürzesten Pfaden in Graphen ist ein grundlegendes Problem der Informatik und Graphentheorie. Es hat zahlreiche Anwendungen in verschiedenen Bereichen, darunter auch Computerspiele. In diesem Abschnitt untersuchen wir verschiedene Algorithmen zur Suche nach kürzesten Pfaden und einige ihrer Grenzen. Ein wesentlicher Unterschied zwischen dem in Abschnitt 3.1 besprochenen TSP und kürzesten Pfaden ist, dass wir nicht *alle* Knoten des Graphen besuchen müssen. Wir werden sehen, dass das Kürzeste-Pfade-Problem effizient lösbar ist, sodass wir es mit vertretbaren Rechenressourcen auf sehr große Graphen anwenden können.

Um das Kürzeste-Pfade-Problem formal zu definieren, müssen wir zunächst die *Länge* eines Pfades in einem Graphen definieren. Die Länge einer Wanderung haben wir bereits in Definition 2.3 eingeführt, dabei aber nur die Anzahl ihrer Kanten verwendet. Typischerweise treten Kürzeste-Pfade-Probleme in *gewichteten Graphen* auf, und wir wollen das Gesamtgewicht aller Kanten des Pfades minimieren, nicht die Kantenanzahl. Daher führen wir eine Variante unserer Längenfunktion ein.

Definition 5.1. Gegeben sei eine (u, v)-Wanderung zwischen den Knoten u und v: $W_{u,v} = (S_V, S_E)$ in einem gewichteten Graphen mit Gewichtsfunktion $\omega : E \to Z$. Die Länge der Wanderung ist definiert als $\mathcal{L}_2(W_{u,v}) = \sum_{e \in S_E} \omega(e)$. Wieder haben die leere und die triviale Wanderung Länge 0.

Für einen gegebenen Start- und Endknoten in einem gewichteten Graphen besteht das *Kürzeste-Pfade-Problem* darin, eine Wanderung zwischen beiden Knoten zu finden, deren Länge minimal ist. Gibt es mehrere minimale Wanderungen, ist jede davon eine gültige Lösung.
Nicht alle Kürzeste-Pfade-Probleme sind jedoch einfach zu lösen. Kanten mit negativem Gewicht bringen eine Schwierigkeit mit sich, mit der die effizientesten Algorithmen nicht umgehen können. Ein Hindernis für jedes Kürzeste-Pfade-Problem sind negative Kreise, die von der Quelle erreichbar sind und von denen auch das Ziel erreichbar ist. In einem negativen Kreis verringert jede Runde die Gesamtlänge des Pfades. Abbildung 12 zeigt einen gerichteten Graphen mit einem solchen Kreis. In einem ungerichteten Graphen erlaubt bereits eine einzelne negative Kante eine negative geschlossene Wanderung, indem sie in beiden Richtungen durchlaufen wird; nach unserer früheren Definition ist dies jedoch nicht notwendigerweise ein Kreis.

Hintergrund

Beachten Sie, dass wir ausdrücklich die Definition einer *Wanderung* verwenden, um das Kürzeste-*Pfade*-Problem zu definieren. Das ist seltsam! Erklären wir es so: Bei positiven Kantengewichten kann jeder Teil einer Wanderung, der zum selben Knoten zurückkehrt, *entfernt werden*, denn dadurch wird ihre Länge tatsächlich kleiner. Der Umweg ist immer teurer. Folglich ist jede kürzeste Wanderung in einem solchen Graphen *auch ein Pfad*. Bei nichtnegativen Gewichten oder gerichteten negativen Kanten ohne negative Kreise erhöht das Entfernen geschlossener Abschnitte die Kosten nicht. Daher ist mindestens eine kürzeste Wanderung ein Pfad.
Die Unterscheidung ist wichtig, wenn negative Kreise erlaubt sind. Kann auf dem Weg vom Start zum Ziel ein negativer Kreis durchlaufen werden, *existiert keine kürzeste Wanderung*. Der Vorschlag, dann „einfach“ wiederholte Knoten zu verbieten und das zu erzwingen, was wir sonst erhalten – einen Pfad –, führt jedoch zu einem ganz anderen Problem. Natürlich existiert ein kürzester Pfad, also ein tatsächlicher Pfad, sobald das Ziel erreichbar ist, ABER ihn zu finden ist im Allgemeinen NP-schwer. Warum? Betrachten wir einen Graphen mit Kreisen, in dem alle Gewichte -1 sind. Einen kürzesten $s-t$-Pfad darin zu finden ist tatsächlich das Längste-Pfade-Problem: Möglichst viele Knoten zu besuchen bedeutet auch, möglichst viele Kanten zu besuchen, und jede Kante verringert die Pfadlänge. Wieder erkennen Sie den Zusammenhang mit der Suche nach einem Hamiltonpfad.

Algorithmen für kürzeste Pfade beruhen auf dem folgenden *Optimalitätsprinzip* (engl. optimality principle):

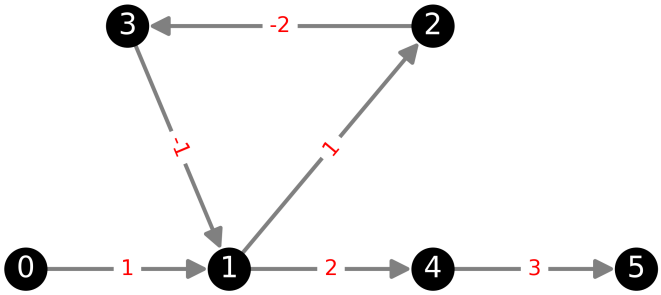


Abbildung 12: Graph mit negativem Kreis

Satz 5.2 (Optimalitätsprinzip für kürzeste Pfade)**.** *Sei $W_{s,t}$ eine kürzeste Wanderung von s nach t in einem gewichteten Graphen. Teilen wir diese Wanderung an einem beliebigen Vorkommen eines Knotens v in ein Präfix $W_{s,v}$ und ein Suffix $W_{v,t}$. Dann ist $W_{s,v}$ eine kürzeste Wanderung von s nach v und $W_{v,t}$ eine kürzeste Wanderung von v nach t. Insbesondere gilt*

$$\mathcal{L}_2(W_{s,t}) = \mathcal{L}_2(W_{s,v}) + \mathcal{L}_2(W_{v,t}).$$

Beweis. Könnte einer der Teile durch eine kürzere Wanderung zwischen denselben Endpunkten ersetzt werden, würde diese Ersetzung auch die gesamte Wanderung von s nach t verkürzen, im Widerspruch zu ihrer Optimalität. □

Das Prinzip gilt immer dann, wenn eine kürzeste Wanderung existiert. Kann auf einer Wanderung von s nach t ein negativer Kreis besucht werden, erzeugt dessen wiederholtes Durchlaufen Wanderungen mit immer kleinerem Gesamtgewicht; eine kürzeste Wanderung existiert dann nicht.

Hintergrund

Diese Eigenschaft heißt optimale Teilstruktur: Eine optimale Lösung enthält optimale Lösungen ihrer Teilprobleme. Sie ermöglicht es Kürzeste-Pfade-Algorithmen, Lösungen aus optimalen Routen zwischen Zwischenknoten aufzubauen.

5.1 Kürzeste Pfade mittels Breitensuche

Die Breitensuche findet Pfade mit den wenigsten Kanten. In ungewichteten Graphen oder Graphen, deren Kanten alle dasselbe nichtnegative Gewicht besitzen, sind dies auch kürzeste Pfade nach Gewicht. Algorithmus 6 speichert die Anzahl der Kanten vom Startknoten aus.

Eine einfache Breitensuche (BFS) findet mit Adjazenzlisten alle ungewichteten Distanzen in linearer Zeit. Ihr Pseudocode steht in Algorithmus 6. Die Zeitkomplexität ist $O(V + E)$, da jeder Knoten und jede Kante höchstens einmal verarbeitet wird. Der Algorithmus verwaltet eine Liste A von Knoten, die alle dieselbe Distanz zum Startknoten haben. Für jeden Knoten der aktuellen Ebene werden alle Nachbarn untersucht. Wurden sie noch nicht besucht, also $\text{dist}(u) = \infty$, wird ihre Distanz aktualisiert und sie werden der Liste B hinzugefügt, den Knoten der nächsten Ebene. Nachdem alle Knoten der aktuellen Ebene verarbeitet wurden, wird die aktuelle Ebene A durch die nächste Ebene B ersetzt. Der Vorgang läuft weiter, bis keine Knoten mehr zu erkunden sind. Der Algorithmus liefert die Distanz vom Startknoten zu allen anderen Knoten sowie eine Vorgängerliste W, mit der die tatsächlichen kürzesten Pfade rekonstruiert werden können.

Algorithmus 6 Kürzeste Pfade mittels Breitensuche

Eingabe: Ein ungewichteter Graph $G = (V, E)$ und ein Startknoten s
Ausgabe: Distanzen als Kantenanzahl von s aus und Vorgänger zur Pfadrekonstruktion

```
1:  lvl ← 0
2:  A ← (s)
3:  W(s) ← (s)
4:  dist(s) ← 0
5:  für alle v ∈ V \ {s} tue
6:      dist(v) ← ∞
7:      W(v) ← ()
8:  Ende für
9:  solange |A| > 0 tue
10:     lvl ← lvl + 1
11:     B ← ()
12:     für alle v ∈ A tue
13:         für alle u ∈ Suc(v) tue                ▷ für alle Nachfolger von v
14:             wenn dist(u) = ∞ dann
15:                 dist(u) ← lvl
16:                 W(u) ← (v)
17:                 B ← B ∘ (u)                    ▷ hänge u an B an
18:             Ende wenn
19:         Ende für
20:     Ende für
21:     A ← B
22: Ende solange
23: gib zurück dist, W
```

Wir wenden Algorithmus 6 auf den gerichteten Graphen in Abbildung 13 an und starten bei s. Vorerst ignorieren wir die Kantengewichte und zählen nur die durchlaufenen Kanten. Bei Wahlmöglichkeiten verarbeiten wir Nachfolger in alphabetischer Reihenfolge.

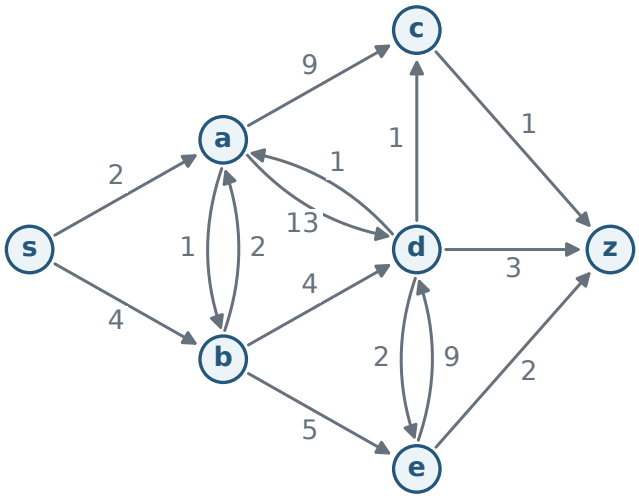


Abbildung 13: Ein gerichteter Graph zum Vergleich von Kürzeste-Pfade-Algorithmen. Die Zahlen neben den Kanten sind ihre Gewichte.

Distanz von s	0	1	2	3
Erstmals erreichte Knoten	s	a, b	c, d, e	z

Nach der Verarbeitung von s lautet die aktuelle Liste $A = (a, b)$. Die Verarbeitung von a entdeckt c und d; die von b fügt e hinzu. Die nächste Liste ist daher $A = (c, d, e)$. Knoten c entdeckt dann z und liefert den Pfad (s, a, c, z) mit drei Kanten. Jeder Knoten erhält seine Distanz bei seiner ersten Entdeckung.
Betrachten wir nun die Gewichte. Der Pfad (s, a, c, z) kostet $2+9+1 = 12$, während (s, a, b, d, c, z) trotz seiner fünf Kanten nur $2+1+4+1+1 = 9$ kostet. Bei ungleichen Gewichten muss die Minimierung der Kantenanzahl nicht das Gesamtgewicht minimieren. Wir benötigen daher Algorithmen, die eine Distanz nach der ersten Entdeckung eines Knotens verbessern können.

Exkurs

Ein besonderer Graph, für den BFS am effizientesten ist, ist ein Baum. Bäume behandeln wir in Kapitel 6.1. Nur weil ein Algorithmus nicht zur Lösung eines allgemeinen Problems geeignet ist, kann er trotzdem der beste für ein spezielles Problem sein – selbst wenn es sich um einen allgemeinen Algorithmus wie BFS handelt.

5.2 Floyd-Algorithmus

Der bereits aus Abschnitt 4.2 bekannte Warshall-Algorithmus lässt sich zur Lösung des Kürzeste-Pfade-Problems für alle Paare (APSP) anpassen. Dieses sucht nicht nur einen einzelnen kürzesten Pfad zwischen zwei Knoten, sondern kürzeste Pfade zwischen *allen Knotenpaaren* eines gewichteten Graphen. Im angepassten Code in Algorithmus 7 verwenden wir die Operationen min und + statt logischem Oder und logischem Und. Die Zeitkomplexität bleibt $O(V^3)$. Der Algorithmus kann auf Graphen mit negativen Gewichten angewendet werden.

Die Interpretation über Zwischenknoten ist genau dieselbe wie bei Warshall: Wir behalten die beste bisherige Route oder kombinieren eine Route nach k mit einer von k aus. Sind etwa die aktuellen Einträge $d[A][C] = 8$, $d[A][B] = 2$ und $d[B][C] = 3$, ändert das Zulassen von B als Zwischenknoten $d[A][C]$ zu $\min(8, 2 + 3) = 5$. Die Endpunkte A, C müssen noch nicht als Zwischenknoten verarbeitet worden sein.

Algorithmus 7 Algorithmus zur Berechnung kürzester Pfade zwischen allen Paaren

Eingabe: Ein gewichteter Graph $G = (V, E, \omega)$ ohne negative Kreise
Ausgabe: Eine Matrix kürzester Distanzen zwischen allen Knotenpaaren in G

```
d[·][·] ← ∞                                   ▷ Distanzmatrix initialisieren
für (u, v) ∈ E tue
    d[u][v] ← ω((u, v))                       ▷ Bei ungerichteten Graphen Symmetrie sicherstellen
Ende für
für v ∈ V tue
    d[v][v] ← 0                               ▷ Distanz zu sich selbst ist null
Ende für
für k ∈ V tue
    für i ∈ V tue
        für j ∈ V tue
            d[i][j] ← min(d[i][j], d[i][k] + d[k][j])
        Ende für
    Ende für
Ende für
gib zurück d
```

Außerdem liefert der Algorithmus nur die kürzesten Distanzen zwischen Knotenpaaren, nicht die tatsächlichen Pfade. Um die Pfade zu erhalten, müssen wir Vorgänger speichern. Wir können eine Vorgängermatrix $pred[\cdot][\cdot]$ initialisieren, indem wir für jede Kante $(u, v) \in E$ den Wert $pred[u][v] = u$ setzen. Wann immer wir eine Distanz $d[i][j]$ über einen Zwischenknoten k aktualisieren, setzen wir auch den Vorgänger $pred[i][j] = pred[k][j]$. Nach Abschluss des Algorithmus können wir den kürzesten Pfad von u nach v rekonstruieren, indem wir von v aus durch die Vorgängermatrix zurückgehen, bis wir u erreichen.

5.3 Bellman-Moore-Ford-Algorithmus

Bellman–Moore–Ford berechnet kürzeste Pfade von einem Startknoten s zu allen anderen Knoten. Wie bei BFS setzen wir anfangs $\text{dist}(s) = 0$ und alle anderen Distanzen auf ∞. Bei ungleichen Kantengewichten kann eine neu entdeckte Route jedoch eine bereits endliche Distanz verbessern. Falls

$$\text{dist}(u) + \omega((u, v)) < \text{dist}(v),$$

ersetzen wir $\text{dist}(v)$ durch diesen kleineren Wert und merken uns u als Vorgänger von v. Diese Operation heißt *Entspannung* (engl. relaxation), auch Relaxation. Die bekannte Wanderung nach u muss noch nicht die kürzeste sein; jede Verbesserung kann helfen, eine bessere Route nach v zu entdecken.

Hintergrund

Oben haben wir gesagt, dass Bellman–Moore–Ford kürzeste Pfade von s zu *allen anderen* Knoten findet. Tatsächlich können wir den Algorithmus nicht vorzeitig anhalten. Wir wissen erst dann, ob wir den kürzesten Pfad zu einem bestimmten Knoten t gefunden haben, wenn bestätigt ist, dass keine weitere Relaxation möglich ist und kein weiterer Knoten zu besuchen bleibt, sei es durch erstmaliges Entdecken oder durch die Relaxation eines Nachbarn. Zu diesem Zeitpunkt hat der Algorithmus kürzeste Pfade zu allen anderen Knoten gefunden.

Verbessert sich die Distanz zu einem Knoten, müssen wir seine ausgehenden Kanten untersuchen, um zu sehen, ob die Verbesserung auch kürzere Routen zu seinen Nachfolgern liefert. Die Knoten, deren ausgehende Kanten noch untersucht werden müssen, halten wir in einer Warteschlange Q. Sie folgt der First-in-first-out-Regel (FIFO): Wir entfernen den am längsten wartenden Knoten und hängen neu verbesserte Knoten am Ende an. Ein bereits wartender Knoten benötigt keinen zweiten Eintrag. Die Menge *onQueue* hält fest, welche Knoten warten. Nachdem ein Knoten entfernt und verarbeitet wurde, kann ihn eine spätere Verbesserung erneut in die Warteschlange aufnehmen.

Hinweis zur Implementierung

In der Praxis benötigen Sie nicht immer eine ungeordnete, hashbasierte oder geordnete Menge. Besitzt jeder Knoten einen bei 0 beginnenden Index, können Sie Bitsets verwenden, die deutlich speichereffizienter sind.

Algorithmus 8 gruppiert diese Arbeit in Iterationen der äußeren Schleife. Zu Beginn jeder Iteration speichert er die Anzahl q der aktuell wartenden Knoten und verarbeitet dann genau q Einträge. Neu angehängte Einträge warten bis zur nächsten Iteration der äußeren Schleife.

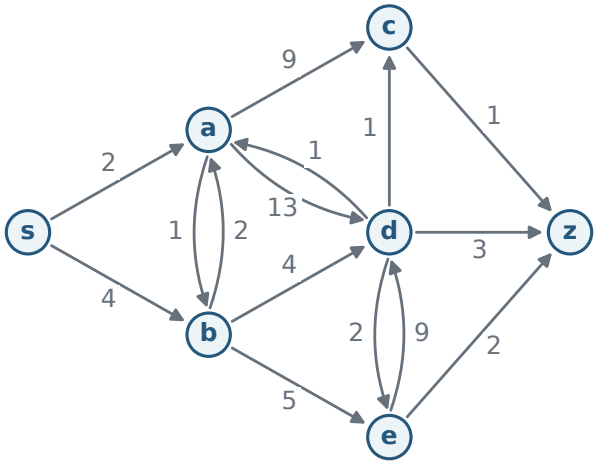
Algorithmus 8 Bellman–Moore–Ford mit einer FIFO-Warteschlange

Eingabe: Ein gewichteter Graph $G = (V, E, \omega)$ und ein Startknoten s
Ausgabe: Distanzen und Vorgänger oder Meldung, dass ein negativer Kreis von s aus erreichbar ist

```
 1: dist(s) ← 0; W(s) ← (s)
 2: für alle v ∈ V \ {s} tue
 3:     dist(v) ← ∞; W(v) ← ()
 4: Ende für
 5: Q ← (s); onQueue ← {s}
 6: für k = 1 bis |V| tue
 7:     q ← |Q|                                  ▷ Anzahl der aktuell wartenden Knoten
 8:     für j = 1 bis q tue
 9:         u ← dequeue(Q); entferne u aus onQueue
10:         für alle v ∈ Suc(u) tue
11:             wenn dist(u) + ω((u, v)) < dist(v) dann
12:                 wenn k = |V| dann
13:                     gib zurück „negativer Kreis von s aus erreichbar“
14:                 Ende wenn
15:                 dist(v) ← dist(u) + ω((u, v)); W(v) ← (u)
16:                 wenn v ∉ onQueue dann
17:                     enqueue(Q, v); füge v zu onQueue hinzu
18:                 Ende wenn
19:             Ende wenn
20:         Ende für
21:     Ende für
22:     wenn Q leer ist dann
23:         gib zurück dist, W
24:     Ende wenn
25: Ende für
26: gib zurück dist, W
```

Wie viele Iterationen der äußeren Schleife sind nötig? Am Ende von Iteration k erfüllt jeder Knoten v die Ungleichung $\text{dist}(v) \leq \mathcal{L}_2(W)$ für jede Wanderung W von s nach v mit höchstens k Kanten. Um dies zu sehen, teilen wir eine solche Wanderung an ihrer letzten Kante (u, v). Nach der vorhergehenden Iteration ist die Distanz zu u höchstens so groß wie die Kosten des Präfixes. Sobald diese Distanz verfügbar ist, hat u sie entweder bereits weitergegeben oder wartet darauf, dies in der aktuellen Iteration zu tun. Die Relaxation von (u, v) liefert daher eine Distanz zu v, die nicht größer als die Kosten von W ist.
Ohne erreichbaren negativen Kreis verwendet ein kürzester Pfad höchstens $|V| - 1$ Kanten. Daher sind alle erreichbaren Distanzen nach höchstens $|V| - 1$ Iterationen der äußeren Schleife korrekt. Eine Verbesserung in Iteration $|V|$ impliziert, dass ein negativer Kreis von s aus erreichbar ist. Jeder Knoten wird pro Iteration der äußeren Schleife höchstens einmal verarbeitet. Mit Adjazenzlisten und Mitgliedschaftstests der Warteschlange in konstanter Zeit beträgt die gesamte Laufzeit einschließlich Initialisierung $O(VE + V)$. Negative Kantengewichte sind erlaubt; sie machen das Relaxationsargument nicht ungültig.
Algorithmus 8 erkennt nur negative Kreise, die vom Startknoten s erreichbar sind. Ein von s aus unerreichbarer Kreis kann dort beginnende Wanderungen nicht beeinflussen. Um negative Kreise irgendwo im Graphen zu erkennen, initialisieren wir jede Distanz mit null und legen jeden Knoten in Q, mit $onQueue = V$. Das ist gleichwertig dazu, zuerst eine künstliche Quelle mit Kanten der Kosten null zu jedem Knoten zu verarbeiten. Eine solche Suche verwenden wir bei der Verbesserung kostenminimaler Flüsse in Abschnitt 10.

Wir kehren zum Graphen in Abbildung 13 zurück, diesmal mit seinen Kantengewichten. Nachfolger werden erneut alphabetisch verarbeitet. Die Tabelle zeigt Warteschlange und Distanzen nach der Untersuchung aller ausgehenden Kanten des angegebenen Knotens u. Der als Nächstes zu entfernende Knoten steht links in Q, und durchgehend gilt $\text{dist}(s) = 0$.



Äußere Iteration k	u	Q danach	a	b	c	d	e	z
–	–	(s)	∞	∞	∞	∞	∞	∞
1	s	(a, b)	2	4	∞	∞	∞	∞
2	a	(b, c, d)	2	3	11	15	∞	∞
2	b	(c, d, e)	2	3	11	7	8	∞
3	c	(d, e, z)	2	3	11	7	8	12
3	d	(e, z, c)	2	3	8	7	8	10
3	e	(z, c)	2	3	8	7	8	10
4	z	(c)	2	3	8	7	8	10
4	c	(z)	2	3	8	7	8	9
5	z	$()$	2	3	8	7	8	9

Wenn a die Distanz zu b von 4 auf 3 verbessert, wartet b noch, sodass sein Warteschlangeneintrag genügt. Später wird c mit Distanz 11 verarbeitet und liefert eine Route nach z mit Kosten 12. Die

Verarbeitung von d verbessert dann die Distanz zu c auf $7 + 1 = 8$. Da c bereits entfernt wurde, muss es erneut angehängt werden. Seine zweite Verarbeitung verbessert die Distanz zu z von 10 auf 9, was auch eine erneute Verarbeitung von z erfordert. Insgesamt gibt es neun Entnahmen aus der Warteschlange: $s, a, b, c, d, e, z, c, z$. Die wiederholte Verarbeitung ist zur Weitergabe dieser Verbesserungen nötig, obwohl alle Gewichte in diesem Beispiel positiv sind.

5.4 Dijkstras Algorithmus

Dijkstras Algorithmus, einer der bekanntesten Algorithmen, findet kürzeste Pfade in Graphen mit nichtnegativen Gewichten. Im Bellman–Moore–Ford-Beispiel mussten c und z erneut verarbeitet werden, weil später eine kürzere Route gefunden wurde. Dijkstra wählt den nächsten Knoten nach seiner kleinsten aktuell bekannten Distanz von s aus. Bei nichtnegativen Kantengewichten garantiert diese Wahl, dass die Distanz bereits die kürzeste ist. Wir nennen einen solchen Knoten *endgültig festgelegt* und fügen ihn der Menge F hinzu. Seine ausgehenden Kanten müssen dann nur einmal untersucht werden. Diese Garantie beweisen wir weiter unten.

Die Zeitkomplexität von Dijkstras Algorithmus hängt stark von zwei Operationen ab:

1. $|V|$-mal müssen wir in Zeile 9 den Knoten v^* mit der kleinsten Distanz bestimmen.
2. $|E|$-mal müssen wir in Zeile 13 die Distanzen aktualisieren (decrease-key-Operation).

Mit einer einfachen Liste benötigt argmin die Zeit $O(V)$, decrease-key dagegen $O(1)$, sodass die Gesamtkomplexität $O(V^2)$ beträgt. Mit einem binären Heap benötigen sowohl argmin als auch decrease-key $O(\log V)$, insgesamt also $O((V + E) \log V)$. Mit einem Fibonacci-Heap lässt sich dies auf $O(E + V \log V)$ verbessern, da decrease-key dort amortisiert $O(1)$ benötigt. Der Pseudocode steht in Algorithmus 9. Somit ist die Lösung des APSP-Problems mit Dijkstra für dünn besetzte Graphen effizienter als mit Floyd-Warshall. Ein großer Teil der tatsächlichen Effizienz hängt allerdings von Implementierung, Rechnerarchitektur und tatsächlicher Graphgröße ab. Vor allem aber lässt sich die Lösung des APSP-Problems mit Dijkstra einfach parallelisieren.

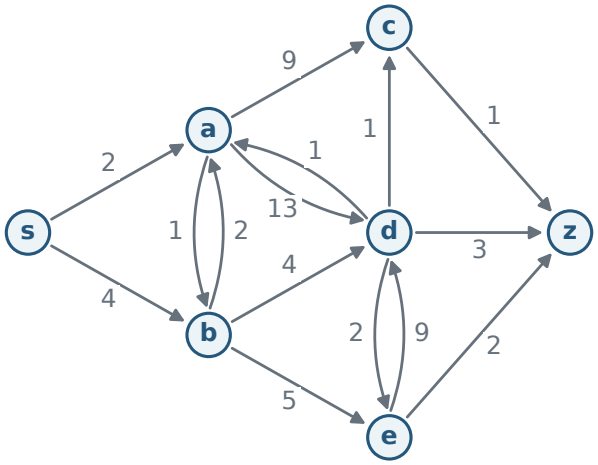
Algorithmus 9 Dijkstras Algorithmus

Eingabe: Ein gewichteter Graph $G = (V, E, \omega)$ mit nichtnegativen Gewichten und ein Startknoten s

Ausgabe: Kürzeste Distanzen von s zu anderen Knoten und eine Vorgängerliste zur Rekonstruktion kürzester Pfade

```
1:  F ← {}
2:  dist(s) ← 0
3:  W(s) ← (s)
4:  für alle v ∈ V \ {s} tue
5:      dist(v) ← ∞
6:      W(v) ← ()
7:  Ende für
8:  solange V ≠ F tue                          ▷ Alternativ anhalten, sobald Zielknoten z ∈ F
9:      v* ← argmin_{v∈V\F} dist(v)            ▷ Knoten mit kleinster Distanz bestimmen
10:     F ← F ∪ {v*}
11:     für alle v ∈ Suc(v*) \ F tue           ▷ alle Nachfolger von v* außerhalb von F
12:         wenn dist(v*) + ω((v*, v)) < dist(v) dann
13:             dist(v) ← dist(v*) + ω((v*, v))        ▷ decrease-key-Operation
14:             W(v) ← (v*)
15:         Ende wenn
16:     Ende für
17: Ende solange
18: gib zurück dist, W
```

Wir wenden Algorithmus 9 auf denselben Graphen in Abbildung 13 an und starten bei s. Jeder endliche Wert $\text{dist}(v)$ ist die Länge einer gefundenen Wanderung; ∞ bedeutet, dass noch keine Wanderung nach v bekannt ist. Die Tabelle zeigt die Distanzen nach der endgültigen Festlegung jedes Knotens und der Untersuchung seiner ausgehenden Kanten. Bei gleichen Distanzen wählen wir alphabetisch.



Nach Festlegung von	s	a	b	c	d	e	z
Anfangs	0	∞	∞	∞	∞	∞	∞
s	0	2	4	∞	∞	∞	∞
a	0	2	3	11	15	∞	∞
b	0	2	3	11	7	8	∞
d	0	2	3	8	7	8	10
c	0	2	3	8	7	8	9
e	0	2	3	8	7	8	9
z	0	2	3	8	7	8	9

Wenn a endgültig festgelegt wird, ändert die Relaxation den Vorgänger von b von s zu a, da $2 + 1 < 4$. Nach b wählt Dijkstra d mit Distanz 7 vor c mit Distanz 11. Die Verarbeitung von d verbessert die Distanz zu c auf 8, bevor c endgültig festgelegt wird. Daher werden die ausgehenden Kanten von c nur mit seiner endgültigen Distanz untersucht. Die kürzeste Route nach z ist (s, a, b, d, c, z) mit Kosten 9. Alle sieben Knoten werden einmal verarbeitet, gegenüber neun Warteschlangenentnahmen im BMF-Beispiel. Die Entdeckung von z mit Kosten 10 ist kein Grund anzuhalten; seine endgültige Festlegung mit Kosten 9 dagegen schon.
Stellen die Gewichte Reisezeiten dar, dauert die Route neun Zeiteinheiten. Ihre aufeinanderfolgenden Knoten folgen Kanten des Graphen, sie ist also zulässig. Um zu zeigen, dass keine schnellere Route existiert, benötigen wir das Korrektheitsargument für Dijkstras Algorithmus und dessen Annahme nichtnegativer Gewichte.
Der in Algorithmus 9 dargestellte Algorithmus kann beendet werden, sobald ein bestimmter Zielknoten z zur Menge endgültig festgelegter Knoten F hinzugefügt wurde, da dann der kürzeste Pfad von s nach z gefunden ist.

Exkurs

Der Vorteil eines frühen Abbruchs zeigt sich weniger in theoretischen Ergebnissen, kann praktisch aber große Auswirkungen haben. Bei einem riesigen Graphen, in dem kürzeste Pfade nur zwischen nahen Knoten berechnet werden, ist frühes Anhalten besonders wichtig. Theoretische Laufzeitschranken gehen von hinreichend großen beliebigen Graphen mit beliebigen Endpunkten aus. Mein Rat lautet, immer zu messen: Auch ein Fibonacci-Heap ist theoretisch

sehr effizient, kann aber in der Praxis für bestimmte Anwendungen und Graphen langsamer sein als ein binärer Heap. Es hängt vom Einzelfall ab.

Innehalten und nachdenken – Kurzcheck

Der tatsächliche Zielknoten wird in Dijkstras Algorithmus nirgends verwendet. Was bedeutet das für die Suche von s zu einem Zielknoten z?

Dijkstras Algorithmus ist die Grundlage der A*-Suche, die wir als Nächstes betrachten, sowie von Prims Algorithmus für minimale Spannbäume, den wir später untersuchen. Beherrschen wir Dijkstras Algorithmus auswendig, können wir somit mit einer kleinen Änderung auch Spannbaumprobleme lösen.

5.4.1 Korrektheitsbeweis

Innehalten und nachdenken – Herausforderung

Wie würden Sie die Korrektheit von Dijkstras Algorithmus beweisen? Welche Beweistechniken haben Sie gelernt?

Beweis. Sei $\text{dist}(v)$ die von Dijkstras Algorithmus berechnete Distanz vom Startknoten s zum Knoten v, und sei $\delta(s, v)$ die tatsächliche kürzeste Pfaddistanz von s nach v. Wir wollen zeigen, dass für alle Knoten $v \in V$ der Wert $\text{dist}(v) = \delta(s, v)$ gilt, wenn v zur Menge F der endgültig festgelegten Knoten hinzugefügt wird.

Wir beweisen die Korrektheit von Dijkstras Algorithmus durch Induktion. Wir nehmen einen Graphen $G = (V, E, \omega)$ mit nichtnegativen Gewichten und einen Startknoten s an.

Induktionsanfang: Der erste endgültig festgelegte Knoten ist s, also gelten nach dieser ersten Auswahl $F = \{s\}$ und $\text{dist}(s) = 0$. Das ist korrekt, da die triviale Wanderung Länge 0 hat; somit gilt $\text{dist}(s) = \delta(s, s) = 0$.

Jeder endliche Wert $\text{dist}(v)$ entspricht den Kosten einer tatsächlichen Wanderung von s aus und kann daher nicht kleiner als die kürzeste Distanz sein. Ist die kleinste Distanz unter den Knoten außerhalb von F gleich ∞, ist kein verbleibender Knoten erreichbar: Eine erste Kante aus den erreichbaren endgültig festgelegten Knoten heraus hätte bereits eine endliche Distanz geliefert.

Induktionsschritt: Ist die kleinste Distanz unter den Knoten außerhalb von F endlich, können wir gemäß unserer Induktionshypothese (IH) annehmen, dass alle Knoten in F korrekte kürzeste Distanzen von s aus besitzen: $\forall_{v \in F}\ \text{dist}(v) = \delta(s, v)$.
Wir müssen zeigen, dass beim Hinzufügen von v^* zu F die Gleichheit $\text{dist}(v^*) = \delta(s, v^*)$ gilt.
Wir wollen zeigen, dass kein kürzerer Pfad P_{s,v^*} existiert. Nehmen wir zum Widerspruch an, der Pfad P sei tatsächlich kürzer. Dann muss gelten:

$$\mathcal{L}_2(P_{s,v^*}) < \text{dist}(v^*)$$

Der Pfad P muss bei $s \in F$ beginnen und bei $v^* \notin F$ enden. Sei u der erste Knoten auf P außerhalb von F, und sei x der Knoten unmittelbar vor u auf P. Dann gelten $x \in F$, $u \notin F$, $(x, u) \in E$, $P_{s,v^*} = (s, \ldots, x, u, \ldots, v^*)$.
Sei $P_{s,u}$ das bei u endende Präfix von P und $P_{s,x}$ das bei x endende Präfix. Nach IH wissen wir aufgrund der nichtnegativen Gewichte:

$$\text{dist}(x) = \delta(s, x) \leq \mathcal{L}_2(P_{s,x}), \text{ also } \text{dist}(x) + \omega((x, u)) \leq \mathcal{L}_2(P_{s,u}) \leq \mathcal{L}_2(P_{s,v^*})$$

Als x zu F hinzugefügt wurde, muss der Algorithmus die Distanz zu u aktualisiert haben, daher:

$$\text{dist}(u) \leq \text{dist}(x) + \omega((x, u))$$

Wir wissen, dass v^* die kleinste Distanz unter allen Knoten außerhalb von F besitzt.
Da $u \notin F$ und wir v^* gewählt haben, wissen wir, dass

$$\text{dist}(v^*) \leq \text{dist}(u)$$

Durch Kombination der Ungleichungen erhalten wir folgenden Widerspruch:

$$\text{dist}(v^*) \leq \text{dist}(u) \leq \text{dist}(x) + \omega((x,u)) \leq \mathcal{L}_2(P_{s,v^*}) < \text{dist}(v^*)$$

Unsere Annahme, dass ein kürzerer Pfad nach v^* existiert, muss daher falsch sein, und wir schließen, dass $\text{dist}(v^*)$ tatsächlich die kürzeste Distanz von s nach v^* ist. □

Der Nachteil von Dijkstras Algorithmus ist, dass er Knoten in alle Richtungen gleichermaßen expandiert, statt sich auf die Richtung zum Zielknoten zu konzentrieren. Sind Start und Ziel weit voneinander entfernt, muss daher weiterhin ein recht großer Teil des Graphen erkundet werden. Dies zeigt Abbildung 14. Im nächsten Abschnitt untersuchen wir den A*-Algorithmus, der diese Einschränkung durch eine Heuristik überwindet, welche die Suche zum Zielknoten lenkt.

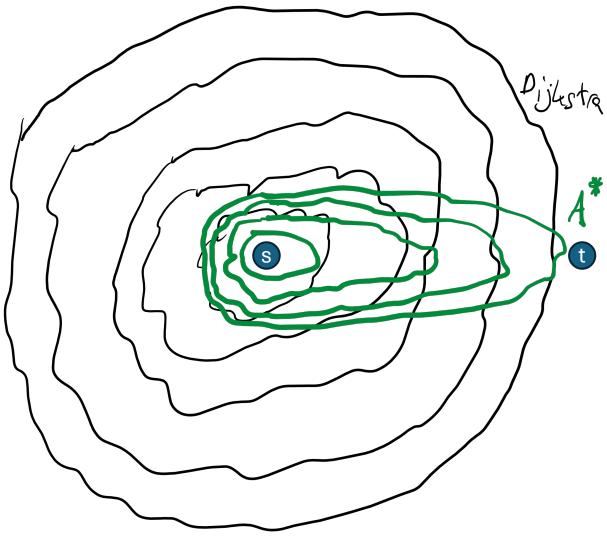


Abbildung 14: Schematische Darstellung der Knotenexpansion bei Dijkstra im Vergleich zu A*

5.5 A*-Algorithmus

Die Idee des A*-Algorithmus ist, die Erkundung jener Knoten zu bevorzugen, die eher zum Zielknoten führen. Dazu müssen wir eine Quantifizierung der „Richtung“ zum Ziel in den Algorithmus einbauen. Das geschieht über eine *heuristische Funktion* $h : V \to Z_0^+$ (Z ist die Zahlenmenge der Kantengewichte, hier aber auf den nichtnegativen Bereich eingeschränkt).

Wir können für h keine beliebigen Werte verwenden, da diese die Suche fehlleiten könnten; wir benötigen aber auch nicht die tatsächlichen kürzesten Distanzen. Die heuristische Funktion schätzt die Kosten von einem Knoten v zum Zielknoten z. Wie bei Dijkstra können wir weiterhin eine optimale gierige Entscheidung garantieren, wenn wir beweisen können, dass h *monoton*, auch *konsistent* genannt, ist (engl. monotonic bzw. consistent).

Definition 5.3. Eine heuristische Funktion $h : V \to Z_0^+$ heißt *monoton* oder *konsistent*, wenn für jede Kante $(u,v) \in E$ die Ungleichung $h(u) \leq \omega((u,v)) + h(v)$ gilt und $h(z) = 0$ ist.

Vielleicht haben Sie in anderen Lehrveranstaltungen oder Quellen bereits gehört, dass die Heuristik h die tatsächliche kürzeste Pfaddistanz von v nach z nicht *überschätzen* darf. Diese Eigenschaft heißt *Zulässigkeit*. Man kann zeigen, dass Monotonie Zulässigkeit impliziert, aber die Umkehrung gilt nicht notwendigerweise. Monotonie ist eine stärkere Bedingung als Zulässigkeit und ist erforderlich, damit der Algorithmus mit einer „abgeschlossenen Menge“ F wie bei Dijkstra optimal ist.

Der Pseudocode des A*-Algorithmus mit abgeschlossener Menge, der eine konsistente Heuristik h erfordert, steht in Algorithmus 10.

Innehalten und nachdenken – Kurzcheck

Erkennen Sie den Unterschied zwischen Dijkstras Algorithmus und A*?

Algorithmus 10 A*-Algorithmus

Eingabe: Ein gewichteter Graph $G = (V, E, \omega)$ mit nichtnegativen Gewichten, ein Startknoten s, ein Zielknoten z, eine konsistente heuristische Funktion h

Ausgabe: Kürzeste Distanz von s nach z und eine Vorgängerliste zur Rekonstruktion des kürzesten Pfades

1: $F \leftarrow \{\}$ ▷ Oft „abgeschlossene Menge“ genannt
2: $\text{dist}(s) \leftarrow 0$
3: $W(s) \leftarrow (s)$
4: **für alle** $v \in V \setminus \{s\}$ **tue**
5: $\quad \text{dist}(v) \leftarrow \infty$
6: $\quad W(v) \leftarrow ()$
7: **Ende für**
8: **solange** $z \notin F$ **tue**
9: $\quad v^* \leftarrow \text{argmin}_{v \in V \setminus F} \text{dist}(v) + h(v)$
10: $\quad F \leftarrow F \cup \{v^*\}$
11: $\quad$ **für alle** $v \in Suc(v^*) \setminus F$ **tue** ▷ alle Nachfolger von v^* außerhalb von F
12: $\quad\quad$ **wenn** $\text{dist}(v^*) + \omega((v^*, v)) < \text{dist}(v)$ **dann**
13: $\quad\quad\quad \text{dist}(v) \leftarrow \text{dist}(v^*) + \omega((v^*, v))$ ▷ h wird hier nicht verwendet
14: $\quad\quad\quad W(v) \leftarrow (v^*)$
15: $\quad\quad$ **Ende wenn**
16: $\quad$ **Ende für**
17: **Ende solange**
18: **gib zurück** dist, W

In Abbildung 15 sehen wir einen pathologischen Fall, in dem A* mit abgeschlossener Menge den kürzesten Pfad von s nach z nicht findet. Wir können überprüfen, dass h zulässig ist, stellen jedoch fest, dass h nicht konsistent ist:

$$\begin{aligned} h(c) &\not\leq h(b) + \omega((b, c)) \\ 6 &\not\leq 2 \quad\quad + 1 \end{aligned}$$

Innehalten und nachdenken – Selbst erarbeiten

Warum scheitert A* in diesem Beispiel?

Was geschieht, nachdem die Suche a abgeschlossen hat? Es gelten $\text{dist}(b) = 5$ und $h(b) = 2$, also ist die geschätzte Gesamtdistanz für b gleich 7. Andererseits gelten $\text{dist}(c) = 3$ und $h(c) = 6$, also ist die geschätzte Gesamtdistanz für c gleich 9. Da der Wert von b kleiner ist, wird es als Nächstes gewählt und zur abgeschlossenen Liste F hinzugefügt. Wenn c in der nächsten Iteration gewählt wird, ist b bereits in F, sodass die kürzere Distanz dorthin nicht erkannt wird: Der Pfad (s, a, c, b) hat Distanz 4. In diesem Beispiel findet A* gemäß Algorithmus 10 den falschen Pfad (s, a, b, z).

Bei einer zulässigen, aber nicht konsistenten Heuristik können wir jedoch eine Variante von A* verwenden. In dieser Variante nutzen wir keine abgeschlossene Menge F, da wir nicht ausschließen können, später einen kürzeren Pfad zu finden. Stattdessen geben wir kürzere Distanzen ähnlich wie beim Bellman-Ford-Algorithmus weiter, allerdings mit einer Prioritätswarteschlange statt einer FIFO-Warteschlange. Somit können Knoten mehrfach expandiert werden, doch solange die Heuristik zulässig ist, wird weiterhin der optimale Pfad gefunden. Der Pseudocode dieser A*-Version

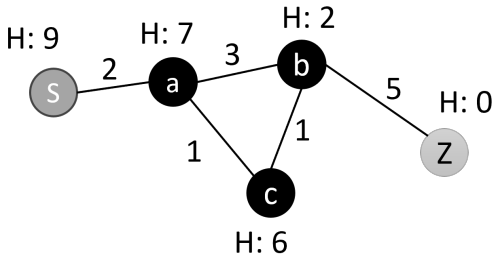


Abbildung 15: Pathologischer Fall, in dem Algorithmus 10 scheitert, da die Heuristik nicht konsistent ist.

steht in Algorithmus 11. Sie ist allerdings nicht so effizient wie die Version mit abgeschlossener Menge, da Knoten mehrfach expandiert werden können.

Algorithmus 11 A*-Algorithmus ohne abgeschlossene Menge

Eingabe: Ein gewichteter Graph $G = (V, E, \omega)$ mit nichtnegativen Gewichten, ein Startknoten s, ein Zielknoten z, eine zulässige heuristische Funktion h

Ausgabe: Kürzeste Distanz von s nach z und eine Vorgängerliste zur Rekonstruktion des kürzesten Pfades

1: $\text{dist}(s) \leftarrow 0$
2: $W(s) \leftarrow (s)$
3: **für alle** $v \in V \setminus \{s\}$ **tue**
4: $\quad \text{dist}(v) \leftarrow \infty$
5: $\quad W(v) \leftarrow ()$
6: **Ende für**
7: $Q \leftarrow$ Prioritätswarteschlange, initialisiert mit $(s, \text{dist}(s) + h(s))$
8: **solange** Q nicht leer ist **tue**
9: $\quad v^* \leftarrow \text{extract-min}(Q)$
10: $\quad$ **wenn** $v^* = z$ **dann**
11: $\quad\quad$ **abbrechen** ▷ Suche anhalten, wenn das Ziel erreicht ist
12: $\quad$ **Ende wenn**
13: $\quad$ **für alle** $v \in Suc(v^*)$ **tue** ▷ für *alle* Nachfolger von v^*
14: $\quad\quad$ **wenn** $\text{dist}(v^*) + \omega((v^*, v)) < \text{dist}(v)$ **dann**
15: $\quad\quad\quad \text{dist}(v) \leftarrow \text{dist}(v^*) + \omega((v^*, v))$
16: $\quad\quad\quad W(v) \leftarrow (v^*)$
17: $\quad\quad\quad$ Füge $(v, \text{dist}(v) + h(v))$ in Q ein bzw. aktualisiere den Eintrag
18: $\quad\quad$ **Ende wenn**
19: $\quad$ **Ende für**
20: **Ende solange**
21: **gib zurück** dist, W

Um die Anzahl erneuter Expansionen in dieser zweiten A*-Variante zu reduzieren, kann man eine Technik namens *pathmax* verwenden. Sie aktualisiert während der Suche die Heuristikwerte, um sicherzustellen, dass sie entlang jedes Pfades nicht abnehmen. Dies kann die Anzahl erneuter Expansionen eines Knotens verringern und so die Effizienz verbessern. Die pathmax-Aktualisierung erfolgt wie folgt: Wann immer ein Knoten v expandiert wird, aktualisieren wir für jeden Nachbarn u von v dessen Heuristikwert so:

$$h(u) \leftarrow \max(h(u), h(v) - \omega((v, u)))$$

In unserem Beispiel hätte dies teilweise geholfen: Beim Abschließen von a würde $h(b) \leftarrow \max(2, 7 - 3) = 4$ gelten. Damit ist die geschätzte Distanz von b gleich der von c, und die Reihenfolge in der Prioritätswarteschlange entscheidet, ob als Nächstes b (falsche Wahl) oder c (richtige Wahl) expandiert wird.

Bei einer konsistenten Heuristik, Heuristikauswertungen in konstanter Zeit und denselben Datenstrukturen besitzt A* ohne erneute Expansion dieselbe Laufzeitschranke für den schlechtesten Fall wie Dijkstra. Dies deckt das erneute Öffnen von Knoten bei einer inkonsistenten Heuristik nicht ab. In der Praxis kann eine nützliche Heuristik die Erkundung erheblich reduzieren; steht keine zur Verfügung, bleibt Dijkstra relevant.

5.6 Bidirektionale Suche

Auch ohne geeignete Heuristik können wir gleichzeitig von beiden Endpunkten aus suchen. Abbildung 16 skizziert die mögliche Einsparung, wenn nahe Knoten ungefähr kreisförmige Gebiete bilden: Zwei Kreisscheiben mit halbem Radius haben zusammen die halbe Fläche einer Kreisscheibe mit vollem Radius. Zwei kleine Pizzen können weniger sein als eine große! Diese geometrische Anschauung kann weniger Expansionen bei räumlichen Graphen erklären; sie ist keine Garantie für beliebige Graphstrukturen.

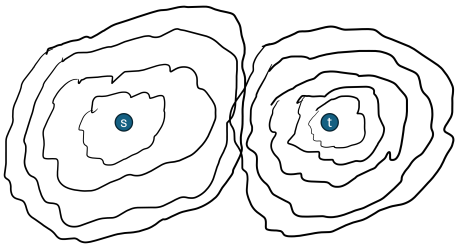


Abbildung 16: Schematische Darstellung der Knotenexpansion bei bidirektionaler Suche

Innehalten und nachdenken – Selbst erarbeiten

Sie haben Dijkstras Algorithmus kennengelernt. Denken Sie kurz selbst über folgende Frage nach: Wann sollte die bidirektionale Suche anhalten, was ist die richtige Bedingung für „in der Mitte getroffen“?

Es zeigt sich, dass die optimale Abbruchbedingung für die bidirektionale Suche etwas knifflig ist. Der naive Ansatz hält an, sobald derselbe Knoten in den Mengen F der endgültig festgelegten Knoten beider Richtungen enthalten ist. Dies liefert zwar einen möglichen Pfad von s nach t, aber nicht notwendigerweise den kürzesten. Abbildung 17 zeigt ein Beispiel, in dem die naive Abbruchbedingung den kürzesten Pfad verfehlt. In beiden Richtungen würde der kürzeste Pfad zum Knoten „c“ gefunden, bevor (s, a, b) oder (t, b, a) gefunden wird. Der Pfad über (a, b) mit Kosten 20 ist jedoch deutlich kürzer als der über c mit Kosten 26.

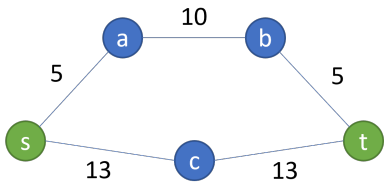


Abbildung 17: Pathologisches Beispiel, in dem die naive Abbruchbedingung den kürzesten Pfad nicht findet

Eine vollständige Route liefert eine obere Schranke S. Seien a, b die kleinsten aktuell bekannten Distanzen unter den noch nicht endgültig festgelegten Knoten der Vorwärts- beziehungsweise Rückwärtssuche. Sobald $a+b \geq S$ gilt, kann Algorithmus 12 anhalten, sofern seine Aktualisierungen der oberen Schranke die Verbindungen zwischen den Suchen erfasst haben. Eine Route zu finden kann dem Nachweis ihrer Optimalität vorausgehen. Jede Suche schließt nur ihre eigene Menge endgültig festgelegter Knoten aus. Die Rückwärtssuche folgt Vorgängern und misst Distanzen nach t.

Hier entfernt join den doppelten Treffknoten, löscht andere geschlossene Abschnitte und speichert eine Kopie der entstehenden Route; S sind deren Kosten. Nichtnegative Gewichte machen das

Entfernen von Kreisen sicher. Wir verwenden $\infty + x = \infty$ und $\infty \geq \infty$, sodass auch unerreichbare Ziele behandelt werden.

Algorithmus 12 Bidirektionaler Dijkstra

Eingabe: Ein gewichteter Graph $G = (V, E, \omega)$ mit nichtnegativen Gewichten, ein Startknoten s und ein Zielknoten t

Ausgabe: Kürzeste Distanz von s nach t und der kürzeste Pfad

wenn $s = t$ **dann**
  gib zurück $0, (s)$
Ende wenn
$F_{s\to t} \leftarrow \{\}$
$F_{t\to s} \leftarrow \{\}$
für alle $v \in V$ **tue**
  $\text{dist}_{s\to t}(v) \leftarrow \infty$
  $W_{s\to t}(v) \leftarrow ()$
  $\text{dist}_{t\to s}(v) \leftarrow \infty$
  $W_{t\to s}(v) \leftarrow ()$
Ende für
$\text{dist}_{s\to t}(s) \leftarrow 0$
$W_{s\to t}(s) \leftarrow (s)$
$\text{dist}_{t\to s}(t) \leftarrow 0$
$W_{t\to s}(t) \leftarrow (t)$
$S \leftarrow \infty$ ▷ Obere Schranke initialisieren
$P \leftarrow ()$
solange $F_{s\to t} \neq V \wedge F_{t\to s} \neq V$ **tue**
  $v^*_{s\to t} \leftarrow \text{argmin}_{v\in V\setminus F_{s\to t}} \text{dist}_{s\to t}(v)$
  $u^*_{t\to s} \leftarrow \text{argmin}_{u\in V\setminus F_{t\to s}} \text{dist}_{t\to s}(u)$
  wenn $\text{dist}_{s\to t}(v^*_{s\to t}) + \text{dist}_{t\to s}(u^*_{t\to s}) \geq S$ **dann** ▷ Abbruchbedingung bei gefundenem kürzesten Pfad
    abbrechen
  Ende wenn
  $F_{s\to t} \leftarrow F_{s\to t} \cup \{v^*_{s\to t}\}$
  $F_{t\to s} \leftarrow F_{t\to s} \cup \{u^*_{t\to s}\}$
  für alle $v \in Suc(v^*_{s\to t}) \setminus F_{s\to t}$ **tue**
    wenn $\text{dist}_{s\to t}(v^*_{s\to t}) + \omega(v^*_{s\to t}, v) < \text{dist}_{s\to t}(v)$ **dann**
      $\text{dist}_{s\to t}(v) \leftarrow \text{dist}_{s\to t}(v^*_{s\to t}) + \omega(v^*_{s\to t}, v)$
      $W_{s\to t}(v) \leftarrow W_{s\to t}(v^*_{s\to t}) \circ (v)$
      wenn $v \in F_{t\to s} \wedge \text{dist}_{s\to t}(v) + \text{dist}_{t\to s}(v) < S$ **dann**
        $P \leftarrow \text{join}(W_{s\to t}(v), W_{t\to s}(v))$
        $S \leftarrow \mathcal{L}_2(P)$
      Ende wenn
    Ende wenn
  Ende für
  für alle $u \in Pre(u^*_{t\to s}) \setminus F_{t\to s}$ **tue** ▷ Vorgänger von $u^*_{t\to s}$
    wenn $\text{dist}_{t\to s}(u^*_{t\to s}) + \omega(u, u^*_{t\to s}) < \text{dist}_{t\to s}(u)$ **dann** ▷ Länge $u \to u^*_{t\to s}$!
      $\text{dist}_{t\to s}(u) \leftarrow \text{dist}_{t\to s}(u^*_{t\to s}) + \omega(u, u^*_{t\to s})$
      $W_{t\to s}(u) \leftarrow (u) \circ W_{t\to s}(u^*_{t\to s})$ ▷ u vorne anfügen
      wenn $u \in F_{s\to t} \wedge \text{dist}_{t\to s}(u) + \text{dist}_{s\to t}(u) < S$ **dann**
        $P \leftarrow \text{join}(W_{s\to t}(u), W_{t\to s}(u))$
        $S \leftarrow \mathcal{L}_2(P)$
      Ende wenn
    Ende wenn
  Ende für
Ende solange
wenn $S = \infty$ **dann**
  gib zurück $\infty, ()$ ▷ Kein Pfad vorhanden
Ende wenn
gib zurück S, P

5.7 A* mit iterativer Vertiefung

Die Tiefensuche (DFS) ist ein verbreiteter Graphalgorithmus. Sie benötigt weniger Speicher als die Breitensuche (BFS) oder eine Bestensuche wie Dijkstra oder A*, muss aber bei der Suche nach kürzesten Pfaden eine exponentielle Anzahl von Pfaden durchsuchen. DFS ist daher für Kürzeste-Pfade-Probleme eher ungeeignet. Wir kennen auch eine DFS-Variante namens Tiefensuche mit iterativer Vertiefung (IDDFS), die eine Folge *tiefenbegrenzter* DFS-Suchen mit wachsenden Tiefengrenzen durchführt, bis der Zielknoten gefunden ist. Algorithmus 13 zeigt den Pseudocode von IDDFS, der wegen der Verwaltung des aktuellen Pfades etwas komplexer als gewöhnliche DFS aussieht.

Algorithmus 13 Tiefensuche mit iterativer Vertiefung (IDDFS)

Eingabe: Ein Graph $G = (V, E)$, ein Startknoten s, ein Zielknoten z
Ausgabe: Ein Pfad von s nach z, sofern einer existiert

```
1: für dlim = 0 bis ∞ tue
2:     found, remaining, path ← DepthLimitedDFS(G, s, z, dlim)
3:     wenn found dann
4:         gib zurück path
5:     Ende wenn
6:     wenn nicht remaining dann
7:         gib zurück null                          ▷ Kein Pfad vorhanden
8:     Ende wenn
9: Ende für
10: Prozedur DepthLimitedDFS(G, s, z, dlim)
11:     remaining ← False
12:     depth ← 0
13:     stack ← ((s, depth))
14:     path ← ()
15:     solange |stack| > 0 tue
16:         (v, d) ← pop(stack)
17:         solange |path| > d tue        ▷ Vorherige Knoten einschließlich Geschwistern entfernen
18:             Entferne das letzte Element aus path
19:             depth ← depth − 1
20:         Ende solange
21:         path ← path ∘ (v)
22:         depth ← d
23:         wenn v = z dann
24:             gib zurück True, *, path
25:         Ende wenn
26:         für alle u ∈ Suc(v) \ {x | x kommt in path vor} tue
27:             wenn depth < dlim dann
28:                 stack ← stack ∘ ((u, depth + 1))
29:             sonst
30:                 remaining ← True
31:             Ende wenn
32:         Ende für
33:     Ende solange
34:     gib zurück False, remaining, ()        ▷ Ziel innerhalb der Tiefengrenze nicht gefunden
35: Ende Prozedur
```

Die Kombination von DFS mit der Optimalität von A* heißt A* mit iterativer Vertiefung (IDA*). Dieser Algorithmus ähnelt IDDFS, verwendet aber statt einer Tiefengrenze eine Distanzgrenze auf Grundlage der geschätzten Gesamtdistanz $f(v) = \text{dist}(v) + h(v)$. Der Pseudocode von IDA* steht in Algorithmus 14. Sein Hauptvorteil ist der geringere Speicherbedarf gegenüber A*. Wieder erhöht eine konsistente Heuristik die Effizienz.

Algorithmus 14 A* mit iterativer Vertiefung (IDA*)

Eingabe: Ein gewichteter Graph $G = (V, E, \omega)$ mit nichtnegativen Gewichten, ein Startknoten s, ein Zielknoten z, eine zulässige oder konsistente heuristische Funktion h
Ausgabe: Kürzeste Distanz von s nach z und der kürzeste Pfad

```
1: limit ← h(s)
2: solange True tue
3:     found, l, path ← DepthLimitedAStar(G, s, z, limit, h)
4:     wenn found dann
5:         gib zurück path
6:     Ende wenn
7:     wenn l = ∞ dann
8:         gib zurück null                          ▷ Kein Pfad vorhanden
9:     Ende wenn
10:    limit ← l
11: Ende solange
12: Prozedur DepthLimitedAStar(G, s, z, limit, h)
13:    min_exceed ← ∞
14:    stack ← ((s, 0, (s)))                        ▷ (Knoten, g-Wert, Pfad)
15:    solange |stack| > 0 tue
16:        (v, g, path) ← pop(stack)
17:        f ← g + h(v)
18:        wenn f > limit dann
19:            min_exceed ← min(min_exceed, f)
20:            fortsetzen
21:        Ende wenn
22:        wenn v = z dann
23:            gib zurück True, *, path             ▷ Ziel gefunden
24:        Ende wenn
25:        für alle u ∈ Suc(v) tue
26:            wenn u ∉ path dann                   ▷ Kreise vermeiden
27:                stack ← stack ∘ ((u, g + ω((v, u)), path ∘ (u)))
28:            Ende wenn
29:        Ende für
30:    Ende solange
31:    gib zurück False, min_exceed, ()             ▷ Ziel innerhalb der Grenze nicht gefunden
32: Ende Prozedur
```

6 Minimale Spannbäume

In Abschnitt 1.5 haben wir bereits kennengelernt, was ein spannender Teilgraph ist. Ein Spannbaum ist ein besonderer Teilgraph, der zusätzliche Bedingungen erfüllt. Spannbäume haben viele Anwendungen in der Informatik. Das Spanning Tree Protocol (STP) wird in Ethernet-Netzwerken verwendet, um Broadcast-Schleifen und endlos zirkulierende Pakete zu verhindern. In verteilten Systemen dienen Spannbäume der effizienten Verbreitung und Zusammenführung von Informationen. Sie spielen auch eine entscheidende Rolle beim Netzwerkentwurf, bei dem eine Menge von Knoten mit minimalem Gesamtgewicht der Kanten verbunden werden soll. Dieses Problem des minimalen Spannbaums behandeln wir in diesem Abschnitt.

6.1 Bäume und Wurzelbäume

Definition 6.1. Ein Baum (engl. tree) ist ein Graph, der zusammenhängend, azyklisch und ungerichtet ist.

Aufgrund dieser Anforderungen besitzt er mehrere interessante Eigenschaften:

- Zwischen je zwei Knoten des Baums gibt es genau einen eindeutigen Pfad.
- Ein Baum mit n Knoten besitzt genau $n-1$ Kanten: $|E| = |V| - 1$.
- Das Hinzufügen einer beliebigen Kante zu einem Baum erzeugt einen Kreis.
- Das Entfernen einer beliebigen Kante aus einem Baum macht den Graphen unzusammenhängend.

Was ist, wenn der Graph nicht zusammenhängend ist? Dafür gibt es den Begriff des Waldes, den wir nun definieren.

Definition 6.2. Ein Wald (engl. forest) ist ein ungerichteter azyklischer Graph, in dem jede Zusammenhangskomponente ein Baum ist.

Innehalten und nachdenken – Kurzcheck

Beantworten Sie anhand der Definitionen für sich selbst: Ist ein Baum ein Wald?

Nun können wir definieren, was ein Spannbaum ist, und das Problem des minimalen Spannbaums einführen.

Definition 6.3. Sei $G = (V, E)$ ein zusammenhängender ungerichteter Graph. Ein **Spannbaum** (engl. spanning tree) von G ist ein Teilgraph $T = (V, E_T)$ mit $E_T \subseteq E$, sodass T ein Baum ist.

Definition 6.4. Ist G ein gewichteter Graph mit einer Gewichtsfunktion $\omega : E \to \mathbb{R}$, dann ist ein **minimaler Spannbaum** (engl. minimum spanning tree, MST) ein Spannbaum $T = (V, E_T)$, der $\sum_{e \in E_T} \omega(e)$ unter allen Spannbäumen von G minimiert.

Hintergrund

Wir haben bereits das TSP als bekanntes Problem der Suche nach dem kürzesten Hamiltonkreis kennengelernt. Ein Kreis ist natürlich kein Baum, doch wenn wir ihn an einer beliebigen Kante auftrennen, etwa an der teuersten, entsteht ein Pfad. Ein Pfad in einem ungerichteten Graphen erfüllt alle Kriterien eines Baums. Ein Hamiltonpfad ist auch ein Spannbaum, da Hamiltonpfade definitionsgemäß alle Knoten enthalten. Hamiltonpfade bilden jedoch nur eine Teilmenge aller Spannbäume, denn sie verlangen zusätzlich, dass der Baum auch ein Pfad ist. Dennoch können wir argumentieren, dass jeder Hamiltonpfad mindestens so lang sein muss wie ein minimaler Spannbaum. Die Kosten eines minimalen Spannbaums sind daher eine untere Schranke für die Kosten eines Hamiltonpfads. Wie wir später sehen werden, sind MSTs recht einfach zu berechnen, insbesondere im Vergleich zum TSP. Sie können uns also etwas darüber sagen, wie kurz ein Hamiltonpfad überhaupt werden kann. Das klingt vielleicht wenig hilfreich,

ist aber eine sehr nützliche Eigenschaft beim Entwurf von Algorithmen zur Lösung schwieriger Probleme wie des TSP.

Abbildung 18 zeigt ein Beispiel für einen Baum und einen Graphen, der kein Baum ist. Der linke Graph ist ein Baum, da er zusammenhängend und azyklisch ist. Der rechte Graph ist kein Baum, da er den Kreis (a, b, e, d, a) enthält.

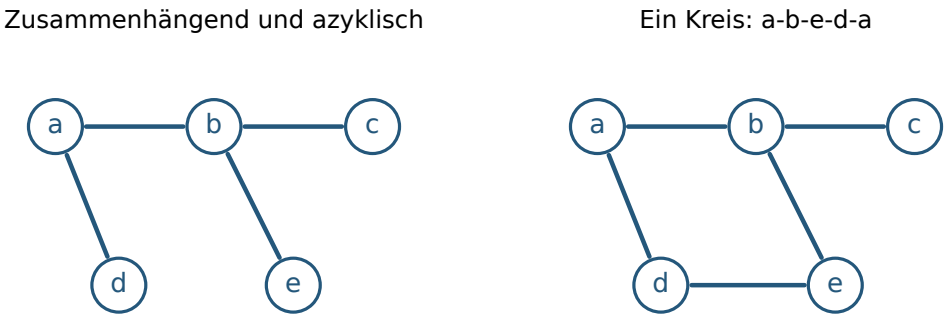


Abbildung 18: Beispiel eines Baums (links) und eines Graphen, der kein Baum ist (rechts)

Bei gerichteten Graphen ist das Problem etwas anders. Ein gerichteter Graph kann selbst dann kreisfrei sein, wenn er mehr als $|V| - 1$ Kanten besitzt. Auch kann bei einem nichttrivialen gerichteten Graphen die Eigenschaft, dass von jedem Knoten ein Pfad zu jedem anderen existiert, nicht erhalten bleiben, wenn der Graph azyklisch bleiben muss. Eine Kantenmenge minimaler Kosten zu finden, mit der wir alle Knoten erreichen können, bleibt dennoch eine interessante Herausforderung. Allerdings müssen wir einen einzelnen sogenannten „Wurzelknoten“ einführen, von dem aus wir alle anderen Knoten erreichen wollen.

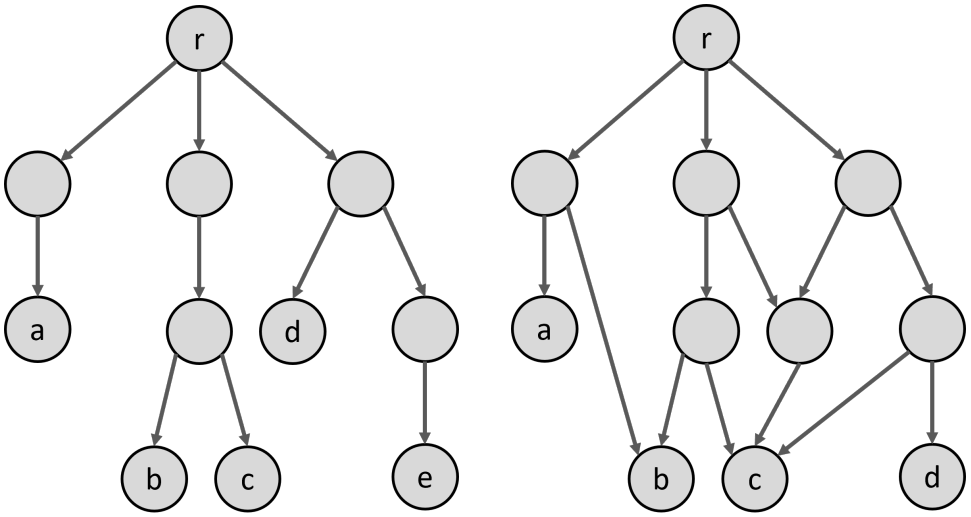
Definition 6.5. Ein zusammenhängender gerichteter Graph $G = (V, E)$ heißt **Wurzelbaum** (engl. arborescence, rooted tree), wenn es einen Knoten $r \in V$ (Wurzel) gibt, sodass folgende Eigenschaften gelten:

1. Jeder Knoten $v \in V$ ist von r aus erreichbar.
2. Die Wurzel hat Eingangsgrad null.
3. Jeder andere Knoten hat Eingangsgrad eins.

Diese Bedingungen implizieren einen eindeutigen gerichteten Pfad von der Wurzel zu jedem Knoten und schließen Kreise aus. Eindeutige einfache Pfade von der Wurzel zu jedem Knoten allein wären nicht ausreichend: $r \to a \to b \to a$ besitzt solche eindeutigen Pfade, aber auch einen Kreis. Daraus folgt, dass jeder Wurzelbaum ein gerichteter azyklischer Graph (DAG, engl. directed acyclic graph) ist, aber nicht jeder DAG ein Wurzelbaum. Abbildung 19 vergleicht einen Wurzelbaum mit einem allgemeineren gerichteten azyklischen Graphen. Die Wurzel trägt die Bezeichnung r, die Blätter sind mit $a, b, c, ...$ bezeichnet. Im DAG ist erkennbar, dass der Pfad von r nach b nicht eindeutig ist. Beide Graphen sind kreisfrei. Auffällig ist, dass in einem Wurzelbaum jeder Knoten beliebig viele Nachfolger, aber höchstens einen Vorgänger haben kann, während in einem DAG jeder Knoten auch beliebig viele Vorgänger haben kann.

Hintergrund

Der Begriff „Baum“ in der Informatik ist nicht derselbe wie der Begriff „Baum“ in der Mathematik. Während wir uns in der Informatik einen Baum häufig als gerichteten Graphen vorstellen, bezeichnet „Baum“ in der Mathematik streng genommen diese besondere Klasse *ungerichteter* Graphen. Verschiedene Fachgemeinschaften können unterschiedliche Wörter für dieselbe Sache verwenden, ähnlich wie natürliche Sprachen; schließlich ist auch Mathematik



(a) Arboreszenz (b) DAG

Abbildung 19: Vergleich von Wurzelbaum und gerichtetem azyklischem Graphen

eine Sprache. Wir können nur die Unterschiede kennen und sie berücksichtigen, wenn wir miteinander sprechen.

6.2 Algorithmus von Prim

Prims Algorithmus konstruiert einen minimalen Spannbaum, indem er von einem beliebigen Startknoten aus einen Baum wachsen lässt. In jedem Schritt fügt er die günstigste Kante hinzu, die einen neuen Knoten mit dem bisherigen Baum verbindet. Konzeptionell ist dies nur eine kleine Variante von Dijkstras Algorithmus: Wir behalten dieselbe algorithmische Struktur bei, ändern aber die Aktualisierung von

$$\text{dist}(v) \leftarrow \text{dist}(v^*) + \omega((v^*, v))$$

zu

$$\text{dist}(v) \leftarrow \omega((v^*, v)).$$

Mit anderen Worten speichert $\text{dist}(v)$ die günstigste *Kante*, die v mit dem bisherigen Baum verbindet, nicht eine Pfadlänge.
Die Datenstrukturen sind entscheidend für die Leistung: Prims Algorithmus verwendet dieselben beiden Operationen wie Dijkstra (extract-min und decrease-key) und besitzt daher dieselbe Zeitkomplexität. Mit einem binären Heap läuft er in $\mathcal{O}((V + E) \log V)$, mit einem Fibonacci-Heap in $\mathcal{O}(E + V \log V)$ und mit einer einfachen Liste in $\mathcal{O}(V^2)$.
Wenn Dijkstras Algorithmus vollständig ausgeführt wird, also nicht nach dem Erreichen eines Zielknotens z vorzeitig endet, definiert die Vorgängerliste ebenfalls einen Spannbaum mit Wurzel s (den *Kürzeste-Pfade-Baum*). Dieser ist jedoch nicht notwendigerweise ein *minimaler* Spannbaum, denn er minimiert die Distanzen von s aus, nicht das Gesamtgewicht der Kanten.

Kürzeste Pfade und günstigste Verbindungen

Angenommen, wir möchten drei Standorte A, B, C durch Kabel verbinden. Die ungerichteten Kanten haben die Gewichte $\omega(\{A, B\}) = 2$, $\omega(\{A, C\}) = 2$ und $\omega(\{B, C\}) = 1$. Wenn eine Reise in A beginnt, kosten die direkten Routen nach B und C jeweils 2; über den anderen Standort zu gehen kostet 3. Dijkstra wählt daher die beiden zu A inzidenten Kanten mit einem Gesamtgewicht des Baums von 4.

Algorithmus 15 Algorithmus von Prim (minimaler Spannbaum)

Eingabe: Ein zusammenhängender ungerichteter gewichteter Graph $G = (V, E, \omega)$ und ein Startknoten s
Ausgabe: Ein minimaler Spannbaum als Vorgängerliste W

$F \leftarrow \{\}$
$\text{dist}(s) \leftarrow 0$
$W(s) \leftarrow (s)$
für alle $v \in V \setminus \{s\}$ **tue**
    $\text{dist}(v) \leftarrow \infty$
    $W(v) \leftarrow ()$
Ende für
solange $V \neq F$ **tue**
    $v^* \leftarrow \text{argmin}_{v \in V \setminus F} \text{dist}(v)$
    $F \leftarrow F \cup \{v^*\}$
    für alle $v \in N(v^*) \setminus F$ **tue** ▷ Nachbarn von v^*
        wenn $\omega((v^*, v)) < \text{dist}(v)$ **dann**
            $\text{dist}(v) \leftarrow \omega((v^*, v))$
            $W(v) \leftarrow (v^*)$
        Ende wenn
    Ende für
Ende solange
gib zurück W

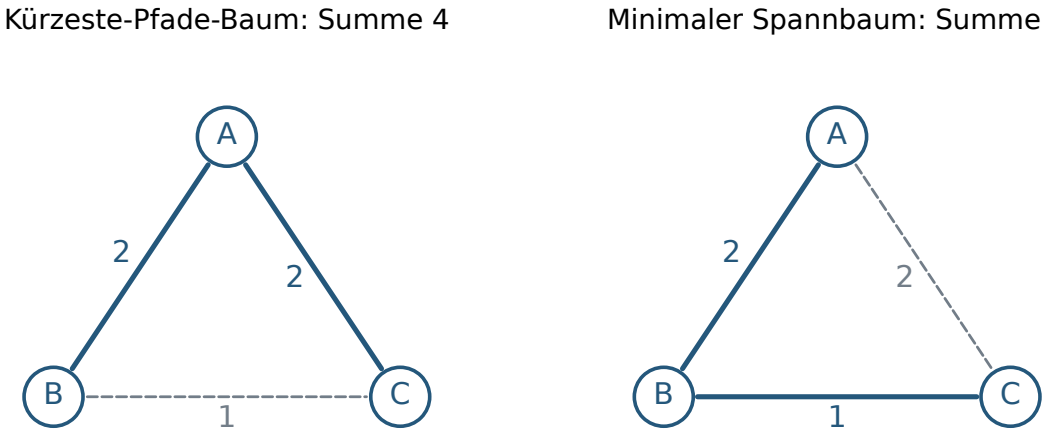


Abbildung 20: Dasselbe ungerichtete Dreieck: ein Kürzeste-Pfade-Baum von A aus (links) und ein minimaler Spannbaum (rechts). Durchgezogene farbige Kanten sind ausgewählt; die gestrichelte Kante wird weggelassen.

Für die Kabelverlegung wählen wir hingegen $\{A, B\}$ und $\{B, C\}$. Ihr Gesamtgewicht ist 3, die Summe der beiden günstigsten Kanten, sodass kein Spannbaum weniger kosten kann. Dies ist ein MST, obwohl seine Route von A nach C den Wert 3 statt der kürzesten Distanz 2 im Graphen hat. Die Wahl von $\{A, C\}$ und $\{B, C\}$ ergibt einen zweiten MST. Ein MST muss nicht alle kürzesten Pfade erhalten.

Prim kann beim Start in A den anfänglichen Gleichstand zugunsten von B auflösen. Anschließend ersetzt er die Verbindungskosten 2 von C durch die Kantenkosten 1 von B aus. Dijkstra vergleicht dagegen 2 mit den *Pfadkosten* $2 + 1 = 3$ und ändert nichts. Die Aktualisierung beschreibt ein anderes Ziel, obwohl die Algorithmen fast identisch aussehen.

Das symbolische Argument am Dreieck macht den Unterschied präzise. Schreiben wir

$$x = \omega(\{A, B\}), \qquad y = \omega(\{A, C\}), \qquad z = \omega(\{B, C\}).$$

Angenommen, der Rest des Graphen ist nur bei A an dieses Dreieck angeschlossen, und das Erreichen von A kostet d_0. Das Ersetzen von $\{A, C\}$ durch $\{B, C\}$ senkt die Baumkosten genau dann, wenn

$$x + z < x + y \quad \Longleftrightarrow \quad z < y.$$

Die direkte Route nach C bleibt jedoch genau dann kürzer als die Route über B, wenn

$$d_0 + y < d_0 + x + z \quad \Longleftrightarrow \quad y < x + z.$$

Beides kann gelten: $z < y < x + z$, wie bei $1 < 2 < 2 + 1$. Damit der gesamte Stern bei A ein Kürzeste-Pfade-Baum ist, muss zusätzlich $x \leq y + z$ gelten; damit $\{A, B\}, \{B, C\}$ ein MST des Dreiecks ist, müssen $x \leq y$ und $z \leq y$ gelten. Unsere Werte $x = y = 2, z = 1$ erfüllen diese Bedingungen.

Beide Bäume in Abbildung 20 verbinden alle drei Standorte ohne Kreis. Bei der Kabelverlegung minimieren wir das Gesamtgewicht der ausgewählten Kanten; bei Reisen ab A minimieren wir die Distanzen von A aus. Die Wahl des Ziels bestimmt, welcher Baum optimal ist.

6.3 Algorithmus von Kruskal

Kruskals Algorithmus nimmt die entgegengesetzte Perspektive ein: Statt einen Baum von einer einzigen Wurzel aus wachsen zu lassen, beginnt er mit einem Wald aus $|V|$ isolierten Knoten und verbindet wiederholt Komponenten durch die günstigste verfügbare Kante. Er verarbeitet die Kanten in aufsteigender Reihenfolge und fügt eine Kante hinzu, wenn sie im aktuellen Wald keinen Kreis erzeugt.

Algorithmus 16 Algorithmus von Kruskal (minimaler Spannbaum)

Eingabe: Ein zusammenhängender ungerichteter gewichteter Graph $G = (V, E, \omega)$
Ausgabe: Ein minimaler Spannbaum T

```
1: T ← (V, ∅)
2: F ← Kanten in E, nach nicht abnehmendem Gewicht sortiert
3: für alle e = (u, v) ∈ F tue
4:     wenn das Hinzufügen von e keinen Kreis in T erzeugt dann
5:         T ← T ∪ {e}
6:     Ende wenn
7: Ende für
8: gib zurück T
```

In einer naiven Implementierung kann der Kreistest für jede Kante durch einen Graphdurchlauf erfolgen, was aufwendig ist. Effiziente Implementierungen verwenden eine Datenstruktur für disjunkte Mengen (Union-Find), um Komponenten zu verwalten und in nahezu konstanter Zeit zu prüfen, ob u und v bereits verbunden sind. Die gesamte Zeitkomplexität wird dann durch das Sortieren der Kanten bestimmt: $\mathcal{O}(E \log E)$.
Zum Vergleich der Zeitkomplexitäten: Kruskal läuft mit Union-Find in $\mathcal{O}(E \log E)$, während Prim dieselbe Komplexität wie Dijkstra besitzt: $\mathcal{O}((V + E) \log V)$ mit einem binären Heap (oder $\mathcal{O}(V^2)$ mit einer Adjazenzmatrix). Für dünn besetzte Graphen sind beide ähnlich, während bei dichten Graphen Prims Implementierung mit Adjazenzmatrix schneller sein kann.
Am selben Dreieck wählt Kruskal zuerst $\{B, C\}$ mit Gewicht 1, dann eine der beiden Kanten mit Gewicht 2, um A anzuschließen. Die verbleibende Kante verwirft er, weil sie einen Kreis schließen würde. Prim und Kruskal stimmen somit bei den minimalen Kosten 3 überein, obwohl Gleichstände unterschiedliche Bäume ergeben können.

6.4 Warum die gierigen Entscheidungen funktionieren: ein Austauschargument

Die nützliche Beweisidee besteht darin, eine Invariante zu erhalten: *Die bereits gewählten Kanten lassen sich zu einem MST ergänzen.* Wir müssen diesen MST nicht im Voraus erraten.

Satz 6.6 (Sichere Kante über einen Schnitt)**.** *Sei F eine Kantenmenge, die in einem MST eines zusammenhängenden ungerichteten gewichteten Graphen enthalten ist. Wähle eine nichtleere echte Knotenmenge S, sodass keine Kante aus F zwischen S und $V \setminus S$ verläuft. Ist e eine Kante minimalen Gewichts über diesen Schnitt, dann ist $F \cup \{e\}$ in einem MST enthalten.*

Beweis. Nehmen wir einen MST T, der F enthält. Enthält er bereits e, sind wir fertig. Andernfalls erzeugt das Hinzufügen von e genau einen Kreis. Was muss dieser Kreis tun, nachdem er entlang e von S nach $V \setminus S$ gewechselt ist? Er muss zurückkehren. Sei f eine weitere Kante des Kreises, die den Schnitt überquert. Da F keine solchen Kanten enthält, gilt $f \notin F$.

Entfernen wir f. Der entstehende Graph $T' = T + e - f$ ist zusammenhängend und azyklisch, also weiterhin ein Spannbaum, und enthält $F \cup \{e\}$. Da e eine günstigste Kante über den Schnitt war, gilt

$$\omega(T') = \omega(T) + \omega(e) - \omega(f) \leq \omega(T).$$

Doch T war bereits minimal, also gilt Gleichheit und T' ist ein MST. Wir haben eine Kante ausgetauscht, ohne die Lösung zu verschlechtern. □

Bei *Prim* wählen wir für S die Knoten des wachsenden Baums. Seine nächste Kante ist eine günstigste Kante über den Schnitt. Bei *Kruskal* wählen wir für S die Knotenmenge einer aktuellen Komponente, die zu seiner nächsten akzeptierten Kante inzident ist. Eine günstigere Kante über den Schnitt wäre bereits verarbeitet worden und hätte die Komponenten verbunden; daher ist auch die akzeptierte Kante eine günstigste Kante über diesen Schnitt. Beide erhalten somit die Invariante. Nach $|V| - 1$ akzeptierten Kanten ist ihr Spannbaum selbst ein MST.

Gleichstände sind relevant: „Eine günstigste Kante ist sicher“ bedeutet, dass sie zu *einem* MST gehört, der F erweitert, nicht notwendigerweise zu jedem MST. Das Dreieck liefert zwei MSTs. Paarweise verschiedene Kantengewichte garantieren Eindeutigkeit: Gäbe es zwei verschiedene MSTs, nähme man die günstigste Kante ihrer symmetrischen Differenz, fügte sie dem Baum hinzu, in dem sie fehlt, und entfernte eine schwerere Kante des entstehenden Kreises, die im anderen Baum fehlt. Das widerspricht der Minimalität. Verschiedene Gewichte sind hinreichend, aber nicht notwendig; ein Graph, der bereits ein Baum ist, besitzt einen eindeutigen Spannbaum, selbst wenn alle Kanten dasselbe Gewicht haben.

Dieses Argument erklärt auch, warum MST-Algorithmen negative Gewichte erlauben. Sie wählen eine azyklische Kantenmenge und bezahlen jedes Gewicht einmal; das wiederholte Hin- und Zurückgehen, das bei kürzesten Wanderungen Schwierigkeiten verursacht, tritt nicht auf.

6.5 Chu-Liu/Edmonds-Algorithmus

Weiterführende Zusammenhänge

Dieser Abschnitt erweitert die Diskussion minimaler Spannbäume auf gerichtete Graphen und ist konzeptionell anspruchsvoller, weil eine lokal optimale Auswahl eingehender Kanten Kreise erzeugen kann, die kontrahiert und rekursiv aufgelöst werden müssen.

Ähnlich wie beim minimalen Spannbaum wollen wir nun einen spannenden Wurzelbaum finden, dessen Kantengewichte eine minimale Summe haben. Kruskals und Prims Algorithmen lassen sich jedoch nicht direkt auf gerichtete Graphen anwenden. In einem minimalen Spannbaum kann jeder Knoten als Wurzel betrachtet werden; der Startpunkt spielt daher keine Rolle. Bei gerichteten Kanten muss hingegen eine Wurzel festgelegt werden. Außerdem haben alle Knoten außer der Wurzel genau eine eingehende Kante. Dies können weder Kruskal noch Prim garantieren; es wäre eine Änderung nötig, die entsprechende Kanten verbietet.

Will man dennoch einen Algorithmus für ungerichtete Graphen, etwa einen angepassten Kruskal-Algorithmus, auf gerichtete Graphen anwenden, ist das Ergebnis nicht optimal. Der Graph in Abbildung 21 zeigt, dass die gierige Kantenauswahl scheitert. Wählt man zuerst die Kante mit Gewicht 1, bleiben nur die beiden Kanten mit Gewicht 9 verfügbar, denn die Kanten mit Gewicht 2 würden einerseits einen Kreis erzeugen und jedenfalls den Eingangsgrad eines Knotens auf 2 erhöhen. Der Wurzelbaum minimaler Kosten enthält dagegen genau die beiden Kanten mit Gewicht 2: $\{(c, a), (a, b)\}$.

Der Chu-Liu/Edmonds-Algorithmus kann auf gerichtete Graphen angewendet werden, um einen Wurzelbaum minimaler Kosten zu finden, sofern einer existiert. Der Algorithmus wurde 1965 von Chu-Liu und 1967 unabhängig davon von Edmonds entwickelt.

Es gibt Formulierungen für minimales und maximales Gewicht; das Negieren der Gewichte überführt die eine in die andere. Die Arbeit von Chu und Liu aus dem Jahr 1965 behandelt ausdrücklich kürzeste Wurzelbäume. Hier minimieren wir Kosten.[1]

Edmonds-Algorithmus

Bei einer *vorgegebenen Wurzel* r suchen wir einen günstigsten spannenden Wurzelbaum, dessen Kanten von r weg gerichtet sind. Ein solcher Wurzelbaum existiert genau dann, wenn jeder Knoten von r aus erreichbar ist. Eingehende Kanten zu r und Schleifen können nicht dazugehören, daher entfernen wir sie vor dem Start des Algorithmus.

Soll die Wurzel *durch die Optimierung gewählt* werden, fügen wir eine künstliche Wurzel R und eine Kante (R, v) zu jedem ursprünglichen Knoten hinzu. Das ist in unserem Beispiel nützlich:

[1] `https://www.cs.cmu.edu/~15850/handouts/chu-liu_1965.pdf`

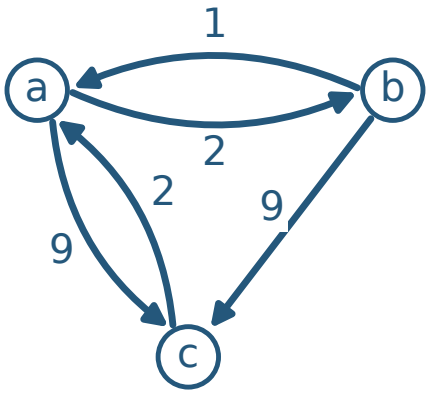


Abbildung 21: Beispiel eines gerichteten Graphen, bei dem die gierige Kantenauswahl scheitert

Eine willkürliche Festlegung einer ursprünglichen Wurzel kann das Optimum ändern. Sei $n = |V|$, sei $B = \max_{e \in E} |\omega(e)|$ (mit $B = 0$, falls E leer ist), und geben wir jeder künstlichen Kante das Gewicht $M > 2nB$.

Warum reicht das aus? Ein erweiterter spannender Wurzelbaum hat n Kanten. Der Beitrag der ursprünglichen Kanten zu jedem solchen Baum liegt zwischen $-nB$ und nB; zwei solche Beiträge unterscheiden sich also um höchstens $2nB$. Eine zusätzliche künstliche Kante kostet mehr als jede mögliche Einsparung bei den ursprünglichen Kanten. Das Optimum minimiert somit zuerst die Anzahl künstlicher Kanten und dann die ursprünglichen Kosten. Besitzt der ursprüngliche Graph einen spannenden Wurzelbaum für irgendeine Wurzel, wird genau eine künstliche Kante gewählt. Entfernen wir R und diese Kante; ihr Endknoten ist die gewählte Wurzel. Benötigt das Optimum mehr als eine künstliche Kante, existiert im ursprünglichen Graphen kein spannender Wurzelbaum mit einer einzigen Wurzel. Gleichwertig kann man das Paar (Anzahl künstlicher Kanten, ursprüngliche Kosten) lexikographisch minimieren.

Es reicht nicht, M lediglich größer als jedes einzelne Kantengewicht zu wählen: Die Einsparung durch das Ändern mehrerer Kanten kann größer sein. In unserem Beispiel mit drei Knoten erfüllen $B = 9$ und $M = 55$ die Schranke. Wir behalten R in der Kontraktionsfolge bei, damit dieses Wurzelauswahlproblem in jedem Rekursionsschritt ein Problem mit vorgegebener Wurzel bleibt.

Der Algorithmus beginnt damit, für jeden Knoten die *eingehende Kante mit dem kleinsten Gewicht* zu wählen. Die Wahl der günstigsten eingehenden Kante liefert eine untere Schranke, da jeder Nichtwurzelknoten genau eine eingehende Kante benötigt. Noch ist nicht garantiert, dass die ausgewählten Kanten einen zulässigen Wurzelbaum bilden. Diese Kantenmenge kann Kreise enthalten; eine künstliche Wurzel allein erzwingt keinen Kreis. Hat ein ursprünglicher Knoten keine eingehende ursprüngliche Kante, kann sofort eine künstliche Kante ausgewählt werden.

Im Beispiel in Abbildung 21 sind die Kanten mit dem kleinsten eingehenden Gewicht für jeden Knoten die Kanten $\{(b, a), (a, b), (a, c)\}$. Man erkennt leicht, dass hier ein Kreis vorliegt. Da wir nur eine eingehende Kante in diesen Kreis erlauben, müssen wir eine Kante des Kreises ersetzen, ihn also an einem Knoten aufbrechen. Dadurch entsteht ein neues Problem desselben Typs wie das ursprüngliche. Wir behandeln den Kreis als einen einzigen Knoten und erhalten so einen Graphen mit weniger Knoten. Gibt es mehrere Kreise, wählen wir einen beliebig aus. Im Chu-Liu/Edmonds-Algorithmus konstruieren wir nun einen neuen Graphen, in dem wir die Knoten des Kreises $C = (V_C, E_C)$ durch einen einzigen Knoten k_C ersetzen. Um den Kreis in den Wurzelbaum einzubinden, müssen die Kosten eingehender Kanten in den Kreis, also in den Knoten k_C, angepasst werden, sodass die Kosten des Aufbrechens berücksichtigt werden. Mit $\text{pre}_c \in V_C$ bezeichnen wir den Vorgänger von $c \in V_C$ im Kreis. Die Gewichte werden wie folgt zugewiesen:

- Kanten, deren beide Endpunkte außerhalb des Kreises liegen, behalten ihre Gewichte. Kanten innerhalb des Kreises verschwinden bei der Kontraktion.
- Ausgehende Kanten behalten ihre Gewichte. Für jedes außerhalb liegende Ziel behalten wir

getrennt einen günstigsten Repräsentanten und merken uns seine ursprünglichen Endpunkte für die Expansion.

- Die Gewichte aller in den Kreis eingehenden Kanten werden angepasst. Mit $X_u = \{(u, c) \in E \mid c \in V_C\}$ bezeichnen wir die Menge aller Kanten, die mit Vorgänger $u \notin V_C$ in den Kreis führen. Die Kosten von $e = (u, k_C)$ sind somit $\omega(e) = \min_{(u,c)\in X_u}\big(\omega((u, c)) - \omega((\text{pre}_c, c))\big)$.

Wir reduzieren die Kosten eingehender Kanten also um die Kosten der Kante im Kreis, die wir ersetzen könnten. Abbildung 22 zeigt einen Ausschnitt davon, wie der Kreis in unserem Beispiel zu einem neuen Knoten führt. In diesem Fall besteht der Kreis aus $V_C = \{a, b\}$, $E_C = \{(a, b), (b, a)\}$. Es gibt eine eingehende Kante von c: (c, a). Da diese Kante die Kante (b, a) ersetzen würde, weil a nur einen Vorgänger haben darf, werden die Kosten um die Kosten dieser Kante reduziert.

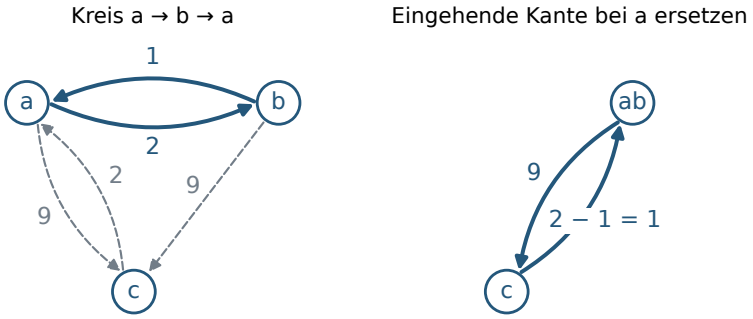
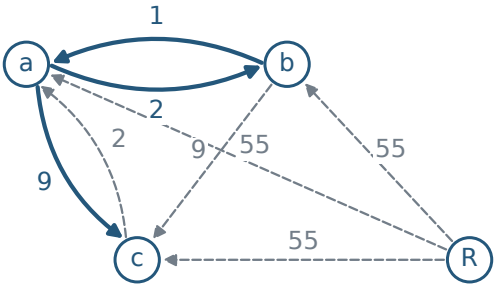


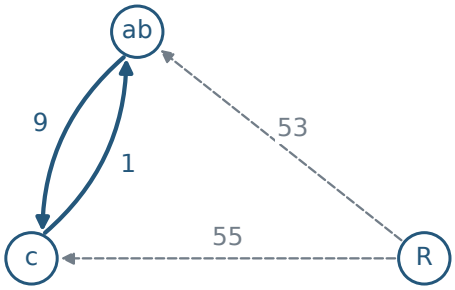
Abbildung 22: Beispiel für die Kontraktion eines Kreises

Der Algorithmus wird nun rekursiv mit dem veränderten Graphen aufgerufen. Wenn kein Kreis mehr verbleibt, ist der Rekursionsbasisfall erreicht, und beim Rücklauf können wir den Wurzelbaum rekonstruieren. Der Chu-Liu/Edmonds-Algorithmus findet somit den spannenden Wurzelbaum minimaler Kosten. Bei der Expansion eines Kreises stellen wir die ursprüngliche eingehende Kante (u, c) und alle ausgewählten Kreiskanten außer (pre_c, c) wieder her. Auch die gespeicherten ausgehenden Repräsentanten werden wiederhergestellt. Die vollständige Folge des Beispiels mit künstlicher Wurzel ist in Abbildung 23 dargestellt.

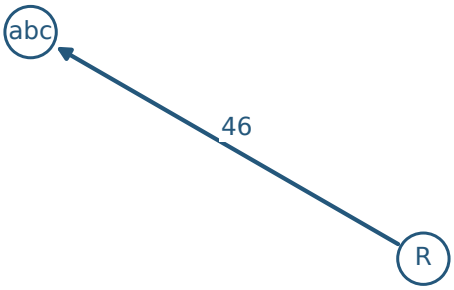
1. Günstigste eingehende Kanten wählen



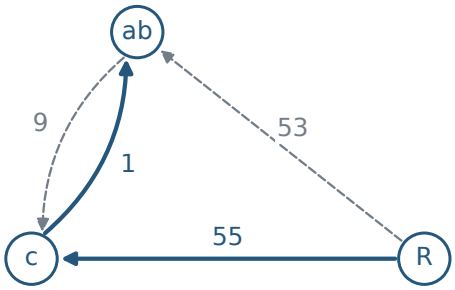
2. {a,b} kontrahieren; weiterer Kreis



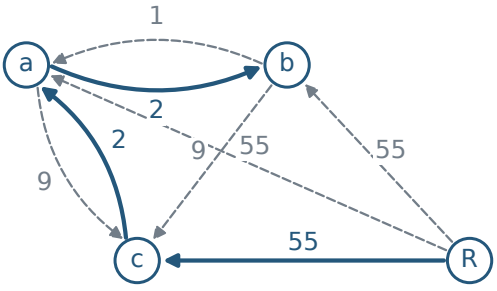
3. {ab,c} kontrahieren; R → abc wählen



4. Expandieren: Eintritt bei c



5. {a,b} expandieren: Eintritt bei a



6. R entfernen: Wurzel c, Kosten 4

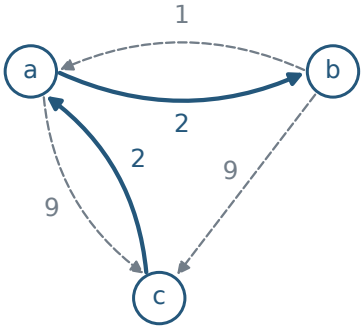


Abbildung 23: Wahl der Wurzel mit $M = 55$: Zuerst $\{a, b\}$, dann $\{ab, c\}$ kontrahieren; bei der Expansion zuerst bei c, dann bei a eintreten. Nach Entfernen von R bleibt $c \to a \to b$ mit Kosten 4.

7 Suche mit Gegenspielern und der Minimax-Algorithmus

Die bisher betrachteten Pfad- und Spannbaumprobleme lassen sich als „Finde eine Auswahl von Kanten, die unter bestimmten Nebenbedingungen die Summe der Kantengewichte minimiert“ verallgemeinern. Die Suche mit Gegenspielern ist grundlegend anders. Es gibt zwei entscheidende Akteure, deren Interessen im Konflikt stehen können. Der Graph der Spielzustände muss kein Baum sein: Verschiedene Zugfolgen können dieselbe Stellung erreichen, und Züge können manchmal zu einer früheren Stellung zurückführen. Für die Suche entfalten wir mögliche Spielverläufe zu einem Wurzelbaum. Ein Zustand, der durch verschiedene Spielverläufe erreicht wird, kann daher in mehreren Baumknoten vorkommen. Wir müssen berücksichtigen, dass eine Menge von Kanten genau entgegen unseren Interessen gewählt wird. Typischerweise ist dies bei Zweipersonenspielen wie „Tic-Tac-Toe“, Vier gewinnt, Schach, Dame, Othello/Reversi und vielen anderen der Fall. Eine verwandte Kategorie sind Spiele wie 2048, bei denen ein zufälliger, unparteiischer Prozess einen neuen Block auf das Spielfeld setzt. Das ist nicht genau ein Gegner, da er nicht optimal spielt, aber die Problemstellung ist ähnlich. Ziel einer Minimax-Anwendung ist, eine Entscheidung zu treffen, bei der das schlechtestmögliche Ergebnis bestmöglich ist.

Innehalten und nachdenken – Herausforderung

Im Webkontext lässt sich ein Angreifer-Verteidiger-Szenario als Spiel modellieren, in dem beide abwechselnd auf die Entscheidungen des anderen reagieren. Ein automatisiertes Verteidigungssystem könnte bei einem gegebenen Angreifermodell mithilfe eines Minimax-Ansatzes über eine Reihe von Gegenmaßnahmen entscheiden.

7.1 Problemstellung

Definition 7.1. Betrachten wir ein deterministisches Zweipersonenspiel mit vollständiger Information, abwechselnden Zügen und einem Nullsummennutzen, gemessen aus der Perspektive von MAX (der Nutzen von MIN ist dessen Negatives). Nehmen wir an, seine Spielverläufe bilden einen endlichen gewurzelten Spielbaum G. Spiele mit Wiederholungen benötigen Beendigungsregeln oder einen endlichen Suchhorizont; eine tiefenbegrenzte Bewertung wird weiter unten eingeführt. Knoten werden typischerweise als Zustände bezeichnet, daher verwenden wir hier s. Endknoten besitzen Nutzenwerte, die durch eine Wertfunktion val gegeben sind. Knoten derselben Tiefe folgen einem gemeinsamen Prinzip: An einem MAX-Knoten wählen wir einen Nachfolger mit maximalem Wert; an einem MIN-Knoten wählt der Gegner einen Nachfolger mit minimalem Wert. Diese Prinzipien wechseln sich mit zunehmender Tiefe ab. Der Minimax-Wert ist rekursiv definiert als

$$\text{value}(s) = \begin{cases} \text{val}(s) & \text{falls } s \text{ ein Endzustand ist,} \\ \max\limits_{w \in Suc(s)} \text{value}(w) & \text{falls } s \text{ ein MAX-Knoten ist,} \\ \min\limits_{w \in Suc(s)} \text{value}(w) & \text{falls } s \text{ ein MIN-Knoten ist.} \end{cases}$$

Innehalten und nachdenken – Kurzcheck

Warum wechseln Maximum und Minimum zwischen aufeinanderfolgenden Ebenen des Spielbaums? Welche Annahmen über den Gegner stecken in diesem Wechsel?

7.2 Ein kleines Gegenbeispiel zum gierigen Spielen

Betrachten wir den Wurzelbaum in Abbildung 24. Dieser Baum besitzt eine MAX-Ebene, eine MIN-Ebene und eine Ebene mit Endzuständen. Die Kanten sind die Aktionen, die ausgeführt werden können.

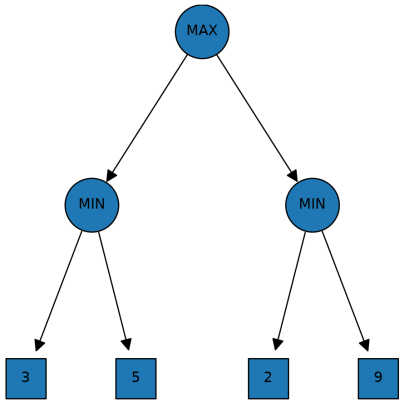


Abbildung 24: Minimax-Beispiel

Innehalten und nachdenken – Selbst erarbeiten

Führen Sie die Wertweitergabe von Hand durch. Welches Ergebnis ergibt sich, wenn beide Parteien optimal spielen? Wie könnte sich Ihr Ergebnis verbessern, wenn Sie erwarten, dass Ihr Gegner suboptimal spielt?

Der Wert beschreibt den Vorteil eines der beiden Spieler. Ein positiver Wert ist ein Vorteil für den MAX-Spieler, ein negativer Wert ein Vorteil für den MIN-Spieler. In einem typischen Nullsummenspiel erhält ein Endknoten, an dem MAX gewinnt, den Wert 1, während ein Sieg von MIN den Wert -1 erhält. Einem Unentschieden wird 0 zugeordnet.

Innehalten und nachdenken – Kurzcheck

Erstellen Sie eine kurze Liste aller Herausforderungen, die Ihnen bei einer praktischen Anwendung in einem geeigneten Szenario einfallen.

7.3 Algorithmus

Der Minimax-Algorithmus ist in Algorithmus 17 angegeben. Es handelt sich um eine rekursive Implementierung, deren Ebenen zwischen MIN und MAX wechseln; die Spieler sind also abwechselnd am Zug. Dieses Modell kann auch gelten, wenn ein Spieler mehrere Züge hat: Diese lassen sich weiterhin zu einer Ebene zusammenfassen, allerdings mit einer größeren Anzahl von Nachfolgezuständen.
Die Laufzeit dieses Algorithmus ist $\Theta(|V| + |E|) = \Theta(|V|)$, da die Anzahl der Kanten eines Wurzelbaums ähnlich groß ist wie die Anzahl seiner Knoten. Das mag sehr effizient erscheinen – linear in der Knotenzahl –, doch die Anzahl solcher Knoten kann extrem groß sein. Bei einem Verzweigungsfaktor von höchstens b und einer Suchtiefe d beträgt die Komplexität $\mathcal{O}(b^d)$. Die rekursive Implementierung benötigt zusätzlich $\Theta(d)$ Hilfsspeicher für den Aufrufstapel, ohne den Speicher für den Spielbaum selbst mitzuzählen.
Eine große Herausforderung ist also die Größe des Wurzelbaums. Beim Schach bezieht sich eine Schätzung von ungefähr $4.8 \cdot 10^{44}$ auf legale *Stellungen*, nicht auf Knoten des vollständigen Spielverlaufsbaums[2]. Zum Vergleich: Der größte Wert einer 64-Bit-Ganzzahl liegt ungefähr bei $1.8 * 10^{19}$. Eine vollständige Auswertung des Baums scheint ausgeschlossen. Hier sind zwei verbreitete Ergänzungen zum grundlegenden Minimax-Algorithmus:

[2] `https://github.com/tromp/ChessPositionRanking`

Algorithmus 17 Rekursiver Minimax-Algorithmus

Eingabe: Ein gewurzelter Spielbaum $G = (V, E)$ mit Wurzel $r \in V$ und Endzuständen $T \subset V$, eine Funktion val : $T \to \mathbb{Z}$
Ausgabe: Der Wert von r bei optimalen Entscheidungen

```
1: Funktion MINIMAX(s, maximizingPlayer)
2:     wenn s ∈ T dann                                   ▷ s ist ein Endzustand
3:         gib zurück val(s)
4:     Ende wenn
5:     wenn maximizingPlayer dann
6:         best ← −∞
7:         für alle w ∈ Suc(s) tue
8:             best ← max(best, MINIMAX(w, False))
9:         Ende für
10:    sonst
11:        best ← ∞
12:        für alle w ∈ Suc(s) tue
13:            best ← min(best, MINIMAX(w, True))
14:        Ende für
15:    Ende wenn
16:    gib zurück best
17: Ende Funktion
18: gib zurück MINIMAX(r, True)
```

- Beschneiden des Baums, d. h. nicht zu jedem Nachfolger hinabsteigen
- Begrenzen der Suchtiefe und Berechnen eines Nutzens für Knoten, die keine Endzustände sind

7.4 Alpha-Beta-Beschneidung

Der rekursive Minimax-Algorithmus berechnet den Wert jedes Knotens im Wurzelbaum. Manchmal wissen wir jedoch, dass dies die Suche nicht mehr beeinflussen würde. Warum? Angenommen, wir bewerten gerade einen MIN-Knoten, dessen Vorgänger naturgemäß ein MAX-Knoten ist. Der MAX-Knoten hat nach der Untersuchung seines ersten Nachfolgers (und damit des darunterliegenden Teilbaums) bereits den Wert 5 festgestellt. Wir können sicher sagen, dass MAX keinen anderen Nachfolger mit einem Wert unter 5 wählen wird. Der Wert von MAX ist nun auf ≥ 5 *beschränkt*. Unser MIN-Knoten hat ebenfalls seinen ersten Nachfolger (und den darunterliegenden Teilbaum) ausgewertet und dabei den Wert 3 erhalten. Sein Wert ist nun auf ≤ 3 beschränkt. Wir können sagen, dass 3 eine obere Schranke für den Wert des MIN-Knotens ist, während 5 eine untere Schranke ist. Welcher Wert x mit $5 \leq x \leq 3$ könnte unsere Entscheidung noch beeinflussen? Genau: Es gibt keinen. Daher müssen die restlichen Nachfolger dieses MIN-Knotens nicht mehr ausgewertet werden. Keiner seiner verbleibenden Nachfolger würde die Entscheidung des vorausgehenden MAX-Knotens noch beeinflussen. Warum sollten wir sie also noch auswerten, wenn die Auswertung so aufwendig ist? Dasselbe Beispiel lässt sich natürlich für ein MIN-MAX-Paar formulieren, und es gilt in jeder Tiefe des Wurzelbaums.
Zusammengefasst verfolgt die Alpha-Beta-Beschneidung beim Durchlaufen des Baums zwei Schranken:

- α ist der größte Wert, den MAX auf dem aktuellen Pfad bereits garantieren kann.
- β ist der kleinste Wert, den MIN auf dem aktuellen Pfad bereits garantieren kann.

Beide Schranken werden an der Wurzel mit $-\infty$ beziehungsweise $+\infty$ initialisiert. An einem MAX-Knoten wird α aktualisiert, sobald ein besseres Kind gefunden wird; an einem MIN-Knoten wird β aktualisiert, sobald ein Kind mit kleinerem Wert gefunden wird. Sobald $\alpha \geq \beta$, können die verbleibenden Nachfolger übersprungen werden.

Innehalten und nachdenken – Kurzcheck

Betrachten Sie Algorithmus 18 und überlegen Sie: Wie könnten Sie die Anzahl der abgeschnittenen Knoten erhöhen? Je mehr Knoten Sie abschneiden können, desto schneller lässt sich die Auswertung des gesamten Wurzelbaums abschließen.

Algorithmus 18 Alpha-Beta-Minimax

Eingabe: Ein Zustand s, Schranken α und β
Ausgabe: Der Minimax-Wert von s

```
1: Funktion AlphaBeta(s, α, β, maximizingPlayer)
2:     wenn s ein Endzustand ist dann
3:         gib zurück val(s)
4:     Ende wenn
5:     wenn maximizingPlayer dann
6:         best ← −∞
7:         für alle w ∈ Suc(s) tue
8:             best ← max(best, AlphaBeta(w, α, β, False))
9:             α ← max(α, best)
10:            wenn α ≥ β dann
11:                gib zurück best
12:            Ende wenn
13:        Ende für
14:    sonst
15:        best ← ∞
16:        für alle w ∈ Suc(s) tue
17:            best ← min(best, AlphaBeta(w, α, β, True))
18:            β ← min(β, best)
19:            wenn β ≤ α dann
20:                gib zurück best
21:            Ende wenn
22:        Ende für
23:    Ende wenn
24:    gib zurück best
25: Ende Funktion
26: gib zurück AlphaBeta(r, −∞, +∞, True)
```

Die Alpha-Beta-Beschneidung liefert genau denselben Wert wie Minimax. Ihre Laufzeit im *schlechtesten Fall* ist weiterhin dieselbe wie beim grundlegenden Minimax-Algorithmus. In der Praxis funktioniert sie jedoch deutlich besser. Die Anzahl der abgeschnittenen Knoten hängt stark von der Reihenfolge ab, in der die Nachfolger untersucht werden. Bei idealer *Zugreihenfolge* kann die Anzahl untersuchter Knoten auf $\mathcal{O}(b^{d/2})$ reduziert werden. Die Zugreihenfolge ist daher eine wichtige praktische Optimierung, obwohl sie das Ergebnis nicht verändert.

Innehalten und nachdenken – Kurzcheck

Was bedeutet hier *ideal*, und wie würde das aussehen?

Im Beispielbaum in Abbildung 24 würde die Alpha-Beta-Beschneidung den letzten Knoten überspringen. Nach Abschluss des linken Zweigs des MAX-Knotens gilt $\alpha = 3$. Nach Betrachtung des ersten Kindes des zweiten MIN-Knotens gilt $\beta = 2 \leq \alpha$. Daher überspringt dieser MIN-Aufruf sein verbleibendes Kind; die Wurzel beendet ihre Arbeit, nachdem die übrigen erforderlichen Schritte abgeschlossen sind.

7.5 Tiefenbegrenzte Suche

Obwohl das Beschneiden bei der Auswertung großer Bäume hilft, gibt es weiterhin Bäume, deren vollständiger Durchlauf nicht machbar ist, zumindest nicht innerhalb eines Menschenlebens. Tiefenbegrenztes Minimax sieht vor, nach einer festen Anzahl L von Zügen anzuhalten. Welchen Nutzen oder Wert sollten wir dann verwenden? Beim Erreichen der Tiefengrenze ist der Zustand meist kein Endzustand, sein Nutzen ist also unbekannt. Dafür führen wir eine Bewertungsfunktion h ein, die den Wert des Zustands schätzt. Sie sollte nur Informationen verwenden, die im Zustand verfügbar sind, und größere Werte für Stellungen liefern, die für MAX besser sind, sowie kleinere Werte für Zustände, die für MIN besser sind.

Algorithmus 19 Tiefenbegrenztes Minimax

Eingabe: Ein Zustand s, verbleibende Tiefe ℓ
Ausgabe: Ein geschätzter Minimax-Wert von s

```
1: Funktion DepthLimitedMinimax(s, ℓ, maximizingPlayer)
2:     wenn s ein Endzustand ist dann
3:         gib zurück val(s)
4:     Ende wenn
5:     wenn ℓ = 0 dann
6:         gib zurück h(s)
7:     Ende wenn
8:     wenn maximizingPlayer dann
9:         best ← −∞
10:        für alle w ∈ Suc(s) tue
11:            best ← max(best, DepthLimitedMinimax(w, ℓ − 1, False))
12:        Ende für
13:    sonst
14:        best ← ∞
15:        für alle w ∈ Suc(s) tue
16:            best ← min(best, DepthLimitedMinimax(w, ℓ − 1, True))
17:        Ende für
18:    Ende wenn
19:    gib zurück best
20: Ende Funktion
21: gib zurück DepthLimitedMinimax(r, L, True)
```

Bei einem Verzweigungsfaktor von höchstens b hat die tiefenbegrenzte Suche eine Laufzeit von $\mathcal{O}(b^L)$, sofern die Berechnung von h konstante Zeit benötigt. Alpha-Beta-Beschneidung lässt sich mit einer Tiefenbegrenzung kombinieren, ändert aber die Laufzeitschranke $\mathcal{O}(b^L)$ für den schlechtesten Fall nicht. Dennoch reduzieren Beschneidung und Zugreihenfolge die Anzahl ausgewerteter Zustände in der Praxis erheblich.
Außerdem sehen wir in Algorithmus 19, dass die Funktion sowohl val als auch h zurückgeben kann und diese Werte anschließend mit min und max verglichen werden. Die beiden Funktionen müssen daher dieselbe Perspektive und Skala verwenden; h muss den Wert eines Zustands bei optimalem Spiel annähern.
Das Hauptrisiko der tiefenbegrenzten Variante ist der *Horizonteffekt*. Ein entscheidungsrelevantes Ereignis kann knapp jenseits der Tiefengrenze liegen, sodass die Bewertungsfunktion es übersieht oder künstlich aufschiebt. Eine verbreitete Gegenmaßnahme beim Schach ist die Ruhesuche: Nach Erreichen der nominellen Tiefengrenze werden erzwingende oder instabile Züge, etwa Schlagzüge oder Schachgebote, weiter untersucht, bis die Stellung ruhig genug ist, damit h eine verlässlichere Schätzung liefern kann.

7.6 Expectimax

Oben haben wir das Spiel 2048[3] erwähnt. Was ist, wenn der Gegner nicht gegen uns spielt, sondern einfach zufällig handelt? Expectimax ersetzt die MIN-Knoten durch Zufallsknoten. Anstatt das Minimum zu nehmen, berechnen wir den mit den Wahrscheinlichkeiten gewichteten Mittelwert der Nachfolgerwerte. An MAX-Knoten wählen wir weiterhin das Maximum. Wir wählen somit den Zug mit dem *besten erwarteten Ergebnis* statt dem besten Ergebnis im schlechtesten Fall.
Die Tiefenbegrenzung funktioniert wie zuvor, doch h sollte nun das erwartete Ergebnis unter den gegebenen Wahrscheinlichkeiten schätzen. Alpha-Beta-Beschneidung ist nicht direkt anwendbar: An einem Zufallsknoten trägt jeder Nachfolger mit positiver Wahrscheinlichkeit zum Mittelwert bei. Beschneidung ist dennoch möglich, wenn Schranken für die verbleibenden Nachfolgerwerte garantieren, dass diese unsere Entscheidung nicht ändern können.

[3] `https://en.wikipedia.org/wiki/2048_(video_game)`

8 PageRank und Zentralität

Bisher haben wir Knoten überwiegend als atomare Objekte behandelt: Wir können sie identifizieren und verbinden, haben aber noch nicht gefragt, was einen Knoten wichtiger als einen anderen macht. Wir haben Pfade gesucht, Kanten für Spannbäume gewählt und Erreichbarkeit berechnet. Natürlich haben wir einzelnen Knoten bereits eine Eigenschaft zugeordnet, nämlich ihren Grad. Nun werden jedoch die Eigenschaften der Knoten selbst zu unserem Hauptinteresse. Welcher Knoten nimmt eine wichtige Position in einem Graphen ein? Und was bedeutet „wichtig“ überhaupt?

In diesem Kapitel untersuchen wir diese Frage zuerst für Webseiten. Wir überführen Hyperlinks in ein mathematisches Modell, erklären, warum eine geeignete Rangfolge existiert, und berechnen sie mit einem einfachen iterativen Algorithmus. Anschließend betrachten wir andere Bedeutungen von Wichtigkeit, sogenannte *Zentralitätsmaße* (engl. centrality measures). Diese führen uns zu Knotengraden und kürzesten Pfaden zurück. Unterwegs benötigen wir etwas lineare Algebra, doch unser Ziel ist zu verstehen, was die mathematischen Ergebnisse ermöglichen, nicht diese Ergebnisse zu beweisen.

8.1 Von Webseiten zu einem mathematischen Modell

In der zweiten Hälfte der 1990er-Jahre entwickelten Larry Page und Sergey Brin in Stanford die Ideen hinter PageRank. Ihre Arbeit von 1998 beschreibt Google als Suchmaschine, die sowohl Seiteninhalte als auch die Struktur der Hyperlinks nutzt [3]. Ein separater Bericht von Page, Brin, Motwani und Winograd entwickelt PageRank ausführlicher [4]. Die interessante Beobachtung ist, dass ein Hyperlink Informationen über den Text einer Seite hinaus liefert: Jemand hat sich entschieden, auf eine andere Seite zu verweisen. Vielleicht lässt sich das als Empfehlung interpretieren? Wir modellieren eine Sammlung von Webseiten als gerichteten Graphen $G = (V, E)$. Jede Seite ist ein Knoten, und $(u, v) \in E$ bedeutet, dass Seite u einen Hyperlink auf Seite v enthält. Mehrere Links von derselben Seite auf dasselbe Ziel zählen hier als eine Kante. Ein Crawler kann einen solchen Graphen erfassen, indem er Seiten herunterlädt und ihre Links extrahiert. Der Graph enthält nur die Seiten, die der Crawler zu diesem Zeitpunkt entdecken konnte.

Innehalten und nachdenken – Kurzcheck

Würden Sie eingehende oder ausgehende Links zählen, wenn Sie eine wichtige Webseite finden wollten? Könnte ein Seitenautor eine dieser Zahlen leicht erhöhen?

Eingehende Links zu zählen ist ein vernünftiger erster Versuch. Aber sollte eine Empfehlung einer wichtigen Seite genauso viel zählen wie die einer unbekannten Seite? Und sollte eine Seite, die tausend andere Seiten empfiehlt, jeder davon ebenso viel Unterstützung geben wie eine Seite, die nur eine empfiehlt? PageRank kombiniert zwei Annahmen: Empfehlungen wichtiger Seiten sollen mehr Gewicht haben, und eine Seite soll ihr Gewicht auf ihre ausgehenden Links verteilen.

Hintergrund

Ein mathematisches Modell ist eine Abstraktion, genau wie die Graphmodelle der Königsberger Brücken in Abschnitt 3.2. Ein Hyperlink kann eine Empfehlung sein, aber auch Kritik, Werbung oder ein automatisch erzeugter Link. PageRank misst Wichtigkeit gemäß seinen Annahmen; es misst weder Wahrheit noch die Relevanz einer Seite für jede mögliche Suchanfrage.

Bei dieser Anwendung kamen drei Dinge zusammen: Das Linkmodell war nützlich genug, um die Rangfolge zu verbessern, die Daten ließen sich erfassen, und das entstehende mathematische Problem ließ sich in praktisch vertretbarer Zeit lösen. Ein nützliches Modell, dessen Berechnung Jahrhunderte dauert, hilft einer Suchmaschine nicht. Umgekehrt hilft auch ein schneller Algorithmus für ein Modell, das das Falsche misst, nicht. Der Erfolg des frühen Ansatzes hing sowohl von Mathematik als auch von technischer Umsetzung ab. Durch die Vorabberechnung von PageRank für den erfassten Graphen standen seine Werte für viele spätere Suchanfragen zur Verfügung.

Warum das wichtig ist

Programmierkenntnisse sind notwendig, um ein solches System zu bauen. Aber wie würden wir die Berechnung erfinden, wenn wir die Mathematik nicht kennen? Graphen erlauben uns, die Daten auszudrücken, lineare Algebra ermöglicht die Formulierung der Rangfolge, und mathematische Ergebnisse sagen uns, wann die Berechnung sinnvoll ist. Das Erlernen dieser Konzepte gibt uns Werkzeuge, um Möglichkeiten zu erkennen, die sonst unsichtbar blieben.

8.2 Empfehlungen und der zufällige Surfer

Während der gesamten Berechnung verwenden wir denselben kleinen Webgraphen. Sei $V = \{A, B, C, D\}$ und

$$E = \{(A, B), (A, C), (B, C), (C, A), (D, C)\}.$$

Abbildung 25 zeigt diesen Graphen. Seite C hat drei eingehende Links, A und B jeweils einen, und D keinen.

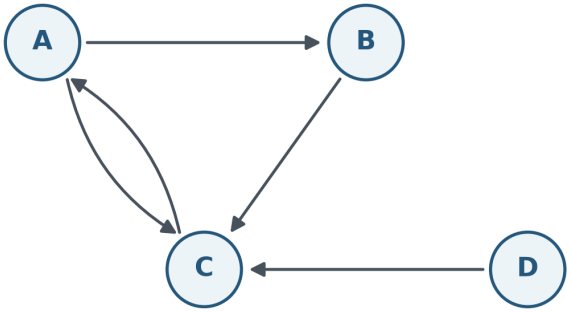


Abbildung 25: Ein kleiner Webgraph. Ein Pfeil zeigt von der Seite, die einen Link enthält, auf dessen Ziel.

Angenommen, Seite v hat einen Wert r_v. Jede Seite u mit einem ausgehenden Link auf v gibt den Anteil $r_u/\text{outdeg}(u)$ weiter. Unser erster Versuch verlangt daher

$$r_v = \sum_{u \in Pre(v)} \frac{r_u}{\text{outdeg}(u)}. \tag{6}$$

Wir verwenden $Pre(v)$ und outdeg(u) wie in Abschnitt 1.2 definiert. Vorerst nehmen wir an, dass jede Seite mindestens einen ausgehenden Link besitzt. Beachten Sie die scheinbare Zirkularität: Um die Wichtigkeit einer Seite zu bestimmen, benötigen wir die Wichtigkeit anderer Seiten, die wiederum von der ersten Seite abhängen kann. Dies ist ein System gleichzeitig zu erfüllender Gleichungen, kein rekursives Verfahren, das Links folgt, bis es eine endgültige Antwort findet.
Dieselbe Gleichung lässt sich anders lesen. Stellen wir uns einen *zufälligen Surfer* vor, der in jedem Schritt einen ausgehenden Link gleichverteilt zufällig auswählt und ihm folgt. Ist r_u die Wahrscheinlichkeit, auf Seite u zu sein, dann ist $r_u/\text{outdeg}(u)$ die Wahrscheinlichkeit, auf u zu sein und als Nächstes ihrem Link nach v zu folgen. Die Summe dieser Beiträge ergibt die Wahrscheinlichkeit, bei v anzukommen. Wir verlangen $r_v \geq 0$ und $\sum_{v \in V} r_v = 1$, denn der Surfer muss irgendwo sein. Gleichung (6) beschreibt eine Verteilung, die sich nach einem Schritt nicht verändert.

Notation

Wir legen die Knotenreihenfolge (A, B, C, D) fest und schreiben Wahrscheinlichkeiten als *Spaltenvektor* r. Wir verwenden eine Matrix P, deren Spalte die *Quelle* und deren Zeile das *Ziel* ist: P_{ij} ist die Wahrscheinlichkeit, von v_j nach v_i zu wechseln. Wenn eine Adjazenzmatrix Zei-

len als Quellen verwendet, nutzt unsere Matrix daher die transponierte Orientierung. Bei einer Zeilenvektorkonvention würde man stattdessen von rechts multiplizieren; beide Konventionen funktionieren, aber wir dürfen sie nicht vermischen.

Definition 8.1. Eine Matrix $P \in \mathbb{R}^{n \times n}$ heißt *spaltenstochastisch* (engl. column-stochastic), wenn $P_{ij} \geq 0$ und $\sum_{i=1}^{n} P_{ij} = 1$ für jede Spalte j gelten. Für einen Webgraphen ohne Seiten mit Ausgangsgrad null definieren wir

$$P_{ij} = \begin{cases} 1/\mathrm{outdeg}(v_j) & \text{falls } (v_j, v_i) \in E, \\ 0 & \text{sonst.} \end{cases}$$

Für unser Beispiel ergibt sich

$$P = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 1/2 & 0 & 0 & 0 \\ 1/2 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \qquad r^{(k+1)} = P r^{(k)}. \tag{7}$$

Der hochgestellte Index (k) zählt Schritte; er ist keine Potenz des Vektors. Prüfen Sie die erste Spalte: Seite A sendet die Hälfte ihrer Wahrscheinlichkeit an B und die andere Hälfte an C. Jede Spalte summiert sich zu eins, daher erhält die Multiplikation mit P die Gesamtwahrscheinlichkeit.

Innehalten und nachdenken – Selbst erarbeiten

Beginnen Sie mit $r^{(0)} = (1/4, 1/4, 1/4, 1/4)^T$ und berechnen Sie $Pr^{(0)}$. Warum verliert Seite D ihre gesamte Wahrscheinlichkeit? Warum erhalten A und B unterschiedliche Anteile, obwohl sie denselben Eingangsgrad besitzen?

8.3 Existenz, Eindeutigkeit und Konvergenz

Nun wissen wir, wie wir einen Schritt beschreiben. Aber existiert eine Verteilung, die unverändert bleibt? Gibt es nur eine? Und finden wir sie durch wiederholte Schritte? Das sind drei unterschiedliche Fragen.

Definition 8.2. Eine *stationäre Verteilung* (engl. stationary distribution) einer spaltenstochastischen Matrix P ist ein Vektor r mit $r_i \geq 0$, $\sum_i r_i = 1$ und $Pr = r$.

Erinnern wir uns: Ein Eigenvektor $x \neq 0$ einer Matrix erfüllt $Px = \lambda x$, wobei λ sein Eigenwert ist. Eine stationäre Verteilung ist daher ein Eigenvektor zum Eigenwert 1, normiert auf die Summe eins. Der Surfer kann sich weiterhin bewegen; unverändert bleibt die Verteilung über die Seiten.

Satz 8.3. *Jede endliche spaltenstochastische Matrix besitzt den Eigenwert* 1 *und mindestens eine stationäre Verteilung. Alle ihre Eigenwerte haben einen Betrag von höchstens* 1.

Die Ergebnisse der linearen Algebra geben wir in diesem Abschnitt ohne Beweis an. Für unsere Anwendung benötigen wir ihre Voraussetzungen und Folgerungen. Stochastizität liefert Existenz, aber für sich allein keine eindeutige Rangfolge. Beispielsweise können zwei voneinander getrennte Gruppen, die jeweils ihre gesamte Wahrscheinlichkeit behalten, abhängig von der Anfangsverteilung unterschiedliche Mengen behalten.

Definition 8.4. Eine nichtnegative Matrix P heißt *irreduzibel* (engl. irreducible), wenn ihr gerichteter Übergangsgraph, mit einer Kante $v_j \to v_i$ genau dann, wenn $P_{ij} > 0$, stark zusammenhängend ist.

Das ist genau die Grapheigenschaft, die wir bereits aus Abschnitt 2 kennen: Jeder Knoten kann jeden anderen erreichen. Gleichwertig existiert für jedes i, j ein $k \geq 0$ mit $(P^k)_{ij} > 0$. Dabei deckt $P^0 = I$ die triviale Wanderung von einem Knoten zu sich selbst ab.

Hintergrund

Übergangsgraph ist lediglich eine spezielle Bezeichnung für einen Graphen, dessen Knoten Zustände sind und dessen Kanten einen Übergang zwischen diesen Zuständen darstellen.

Satz 8.5. *Eine irreduzible endliche spaltenstochastische Matrix besitzt genau eine stationäre Verteilung, und jeder Eintrag dieser Verteilung ist strikt positiv.*

Dies ist eine Folgerung der Perron–Frobenius-Theorie für nichtnegative Matrizen. Doch Vorsicht: Daraus folgt noch nicht, dass wiederholte Multiplikation konvergiert. Betrachten wir zwei Seiten, die nur aufeinander verlinken. Der Graph ist stark zusammenhängend und seine eindeutige stationäre Verteilung ist $(1/2, 1/2)^T$. Beginnen wir jedoch mit $(1, 0)^T$, wechseln die Verteilungen endlos zwischen $(1, 0)^T$ und $(0, 1)^T$.

Die fehlende Bedingung ist *Aperiodizität*. In einem stark zusammenhängenden Übergangsgraphen bedeutet sie, dass der größte gemeinsame Teiler der möglichen positiven Rückkehrlängen zu einem Knoten 1 ist. Im Beispiel mit zwei Seiten sind alle Rückkehrlängen gerade. Eine Schleife in einem stark zusammenhängenden Übergangsgraphen reicht aus, um ihn aperiodisch zu machen, da dann eine Rückkehr in einem Schritt möglich ist.

Satz 8.6. *Ist eine endliche spaltenstochastische Matrix irreduzibel und aperiodisch, dann konvergiert $P^k r^{(0)}$ für jeden anfänglichen Wahrscheinlichkeitsvektor $r^{(0)}$ gegen ihre eindeutige stationäre Verteilung.*

Diese Bedingungen sind hinreichend; wir müssen nicht behaupten, dass jeder reduzible Graph scheitert. Unser Beispiel ist reduzibel, da keine Seite D erreichen kann, dennoch konvergiert die ungedämpfte Iteration gegen $(2/5, 1/5, 2/5, 0)^T$. Das Problem ist, dass ein beliebiger Webgraph nicht die Eigenschaften garantiert, auf die sich ein allgemein einsetzbarer Ranking-Algorithmus verlassen könnte.

8.4 Seiten ohne ausgehende Links, Rangfallen und Teleportation

Das erste Modell hat mehrere Schwierigkeiten, dargestellt in Abbildung 26:

- Eine *Seite ohne ausgehende Links* bietet dem Surfer keinen nächsten Schritt, und die entsprechende Spalte kann nicht durch ihren Ausgangsgrad normiert werden.
- Eine *Rangfalle* ist eine Gruppe ohne nach außen führende Links. Sobald der Surfer sie betritt, kann er sie nicht mehr verlassen. Mehrere solcher abgeschlossenen Gruppen können auch die Eindeutigkeit verhindern.
- Ein periodischer Übergangsgraph kann die Iteration zum Oszillieren bringen, selbst wenn eine eindeutige stationäre Verteilung existiert.

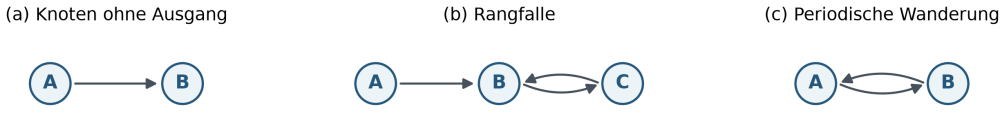


Abbildung 26: Drei Probleme beim ausschließlichen Folgen von Hyperlinks: eine Seite ohne ausgehende Links, eine abgeschlossene Seitengruppe und periodische Bewegung.

Zuerst geben wir einer Seite ohne ausgehende Links ein definiertes Verhalten: Der Surfer wählt eine beliebige der $n = |V|$ Seiten gleichverteilt aus, einschließlich der aktuellen Seite. Wir ersetzen ihre Nullspalte durch $(1/n, \ldots, 1/n)^T$. Von nun an bezeichnet P diese korrigierte spaltenstochastische Matrix. Diese Korrektur allein beseitigt keine Rangfallen an anderen Stellen des Graphen.

Zweitens folgt der Surfer in jedem Schritt mit Wahrscheinlichkeit α den Übergängen in P und wählt mit Wahrscheinlichkeit $1 - \alpha$ eine beliebige Seite gleichverteilt aus. Diese zweite Wahl heißt

Teleportation, und α wird „Dämpfungsfaktor“ genannt. Für unseren zufälligen Surfer modelliert dies das Verhalten, nicht immer einem Link zu folgen, sondern manchmal eine neue Adresse einzugeben und zu einer anderen, von der aktuellen Seite unabhängigen Seite zu springen. Wir könnten dafür auf jeder Seite einen zusätzlichen Link einführen, doch dann hätten Seiten mit mehr Links auch eine geringere Teleportationswahrscheinlichkeit. Wir nehmen $0 < \alpha < 1$ an und verwenden im Beispiel $\alpha = 0.85$, eine übliche Wahl, die auch in der frühen Arbeit verwendet wurde [3]. Beachten Sie, dass α ein Modellparameter und keine mathematische Konstante ist. Möglicherweise können wir dieses Verhalten bei realen Surfern messen und die Wahrscheinlichkeit schätzen. Modellparameter bringen zusätzliche Komplexität mit sich: Sind sie falsch, passt das Modell möglicherweise nicht gut zur realen Welt.

Definition 8.7. Sei $q = (1/n, \ldots, 1/n)^T$ und sei $\mathbf{1}$ der Spaltenvektor aus n Einsen. Die *Google-Matrix* ist

$$M = \alpha P + (1 - \alpha) q \mathbf{1}^T.$$

Der *PageRank-Vektor* r ist die stationäre Verteilung von M:

$$r = Mr = \alpha P r + (1 - \alpha) q, \qquad \sum_i r_i = 1. \tag{8}$$

Jeder Eintrag von M ist mindestens $(1 - \alpha)/n > 0$. Ihr Übergangsgraph ist somit stark zusammenhängend und besitzt Schleifen. Die Sätze 8.5 und 8.6 garantieren nun einen eindeutigen strikt positiven PageRank-Vektor und Konvergenz von jedem anfänglichen Wahrscheinlichkeitsvektor aus. Diese zusätzlichen Übergänge gehören zu unserem Modell; wir haben keine echten Hyperlinks in die Webseiten eingefügt.

Innehalten und nachdenken – Kurzcheck

Was geschieht bei $\alpha = 0$? Welche Garantie verlieren wir bei $\alpha = 1$? Erklären Sie beide Antworten anhand des Surfers, statt nur die Gleichung umzuformen.

Man kann auch eine nicht gleichverteilte Teleportationsverteilung q wählen, um bestimmte Themen oder Startseiten zu bevorzugen. Das ergibt *personalisierten PageRank*. Ist jedes $q_i > 0$, gilt dasselbe Positivitätsargument. Sind einige Einträge null, gilt dieses spezielle Argument nicht mehr; bei der Behauptung strikter Positivität jedes Werts ist daher Vorsicht nötig.

8.5 PageRank mit der Potenzmethode berechnen

Wir könnten Gleichung (8) als lineares Gleichungssystem lösen. Ein Webgraph kann jedoch Millionen von Knoten haben, und wir möchten keine dichte Matrix dieser Größe speichern oder darauf Elimination durchführen. Stattdessen wenden wir die Übergangsregel wiederholt an. Dies heißt *Potenzmethode* (engl. power method), denn nach k Schritten gilt $r^{(k)} = M^k r^{(0)}$. Wir müssen M^k niemals ausdrücklich bilden.

Algorithmus 20 implementiert dieselbe Multiplikation mit ausgehenden Adjazenzlisten. Alle Aktualisierungen verwenden den alten Vektor r und sammeln sich in einem separaten Vektor s. Die Größe z sammelt die Wahrscheinlichkeit auf Seiten ohne ausgehende Links.
Der Abbruchtest misst die gesamte absolute Änderung, auch 1-Norm der Differenz genannt. Kleine Änderung und kleiner Fehler sind nicht dasselbe. Für diese gedämpfte Iteration lässt sich der Fehler des neu berechneten Vektors abschätzen durch

$$\|s - r^*\|_1 \le \frac{\alpha}{1 - \alpha} \|s - r\|_1,$$

wobei r^* der exakte PageRank-Vektor und s der nächste Iterationsvektor nach r ist. Wir beweisen diese Schranke hier nicht; sie folgt daraus, dass ein Schritt die Abstände zwischen Wahrscheinlichkeitsvektoren um einen Faktor von höchstens α verkleinert. Um in exakter Arithmetik einen Fehler von höchstens η zu garantieren, verwendet man $\varepsilon \le \eta(1 - \alpha)/\alpha$. Eine Iterationsgrenze hilft festzustellen, wann die gewünschte Genauigkeit nicht erreicht wurde.

Jede Iteration benötigt $O(|V| + |E|)$ Zeit, und der Graph samt den beiden Vektoren benötigt $O(|V| + |E|)$ Speicher. Für k Iterationen beträgt die Laufzeit $O(k(|V| + |E|))$. Obwohl M wegen der

Algorithmus 20 PageRank durch Potenziteration

Eingabe: Gerichteter Graph $G = (V, E)$, $n = |V| > 0$, $0 < \alpha < 1$, Toleranz $\varepsilon > 0$, Iterationsgrenze $K \geq 1$

Ausgabe: Näherungsweiser PageRank-Vektor oder Meldung, dass die Iterationsgrenze erreicht wurde

$r[v] \leftarrow 1/n$ für jedes $v \in V$
für $k = 1, \ldots, K$ **tue**
    $z \leftarrow \sum_{u \in V:\, \mathrm{outdeg}(u)=0} r[u]$
    $s[v] \leftarrow (1-\alpha)/n + \alpha z/n$ für jedes $v \in V$ ▷ Teleportation
    für $u \in V$ mit $\mathrm{outdeg}(u) > 0$ **tue**
        für $v \in Suc(u)$ **tue**
            $s[v] \leftarrow s[v] + \alpha r[u]/\mathrm{outdeg}(u)$
        Ende für
    Ende für
    $\delta \leftarrow \sum_{v \in V} |s[v] - r[v]|$
    wenn $\delta \leq \varepsilon$ **dann**
        gib zurück s
    Ende wenn
    $r \leftarrow s$
Ende für
gib zurück r mit der Meldung, dass die Iterationsgrenze erreicht wurde

Teleportation dicht ist, durchläuft der Algorithmus nur echte Kanten und addiert einen gleichmäßigen Beitrag. Das ist die entscheidende rechnerische Beobachtung: Die bequeme mathematische Beschreibung zwingt uns nicht, eine dichte Matrix zu speichern. Ein größeres α gibt der Linkstruktur mehr Einfluss, schwächt aber auch die Konvergenzschranke und kann mehr Iterationen erfordern.

8.5.1 Der Zusammenhang mit der transitiven Hülle

Kommt Ihnen die wiederholte Multiplikation bekannt vor? In Abschnitt 4.1 beschrieben boolesche Matrixpotenzen die Erreichbarkeit nach einer festen Anzahl von Schritten. Hier beschreiben Potenzen einer Übergangsmatrix Wahrscheinlichkeiten nach einer festen Anzahl von Schritten. In beiden Fällen kombiniert die Multiplikation Wanderungen über Zwischenknoten. Beispielsweise summiert der Ausdruck

$$(P^2)_{ij} = \sum_{\ell=1}^{n} P_{i\ell} P_{\ell j}$$

die Wahrscheinlichkeiten aller Wanderungen mit zwei Schritten von v_j nach v_i. Fragen wir nur, ob dieser Eintrag positiv ist, erhalten wir die entsprechende boolesche Erreichbarkeitsfrage bei angepasster Zeilen-/Spaltenorientierung zurück. Gewöhnliche Potenzen einer ungewichteten Adjazenzmatrix zählen dagegen Wanderungen.

Der Unterschied liegt darin, was wir behalten. Boolesche Multiplikation kombiniert Möglichkeiten mit UND und ODER; stochastische Multiplikation multipliziert Schrittwahrscheinlichkeiten und addiert Beiträge alternativer Wanderungen. Die transitive Hülle kombiniert erreichbare Paare über verschiedene Längen. PageRank verfolgt eine Wahrscheinlichkeitsverteilung für immer längere Wanderungen, bis sie nahe am Gleichgewicht ist. Es bildet weder eine boolesche Vereinigung noch hält es nach $n-1$ Schritten an: Wanderungen dürfen Knoten beliebig oft erneut besuchen. Außerdem verwendet PageRank Matrix-Vektor-Produkte, während die Konstruktion der Hülle Matrix-Matrix-Produkte berechnet. Durch Teleportation ist jedes Paar in einem Modellschritt erreichbar, doch die langfristigen Wahrscheinlichkeiten können sich weiterhin erheblich unterscheiden.

Innehalten und nachdenken – Selbst erarbeiten

Welche Seiten kann der Surfer in unserem Webgraphen von A aus in genau zwei Schritten entlang von Links erreichen? Berechnen Sie ihre Wahrscheinlichkeiten mit P^2. Erklären Sie dann, welche Informationen eine boolesche Zweischrittmatrix behalten und welche sie verwerfen würde.

8.5.2 Die Berechnung an unserem Webgraphen

Wir wenden nun Algorithmus 20 auf Abbildung 25 an. Es gibt keine Seiten ohne ausgehende Links, daher ist $z = 0$. Für $\alpha = 0.85$ beträgt der gleichmäßige Beitrag $(1 - \alpha)/4 = 0.0375$ pro Seite. Die Aktualisierungsgleichungen lauten

$$\begin{aligned} r_A^{(k+1)} &= 0.85 r_C^{(k)} + 0.0375, \\ r_B^{(k+1)} &= 0.85 r_A^{(k)}/2 + 0.0375, \\ r_C^{(k+1)} &= 0.85(r_A^{(k)}/2 + r_B^{(k)} + r_D^{(k)}) + 0.0375, \\ r_D^{(k+1)} &= 0.0375. \end{aligned}$$

Ausgehend von $r^{(0)} = (0.25, 0.25, 0.25, 0.25)^T$ ergibt die erste Multiplikation

$$r^{(1)} = (0.25,\ 0.14375,\ 0.56875,\ 0.0375)^T.$$

Die Einträge summieren sich weiterhin zu eins. Im nächsten Schritt profitiert A von der erhöhten Wahrscheinlichkeit bei C, während C weniger von B und D erhält. Einzelne Werte müssen sich ihren Grenzwerten daher nicht monoton annähern.

k	$r_A^{(k)}$	$r_B^{(k)}$	$r_C^{(k)}$	$r_D^{(k)}$
0	0.250000	0.250000	0.250000	0.250000
1	0.250000	0.143750	0.568750	0.037500
2	0.520938	0.143750	0.297813	0.037500
3	0.290641	0.258898	0.412961	0.037500
5	0.388517	0.202620	0.371364	0.037500
10	0.375054	0.194937	0.392509	0.037500
20	0.372531	0.195814	0.394155	0.037500
∞	0.372527	0.195824	0.394149	0.037500

Tabelle 1: Potenziteration für das Beispiel mit $\alpha = 0.85$ und gleichverteilter Initialisierung. Die Werte sind gerundet; die letzte Zeile gibt die stationäre Verteilung numerisch an.

Die entstehende Rangfolge lautet $C > A > B > D$, mit Werten von ungefähr 0.394149, 0.372527, 0.195824 beziehungsweise 0.037500. Die Seiten A und B haben denselben Eingangsgrad, aber A erhält den gesamten Beitrag durch Linkverfolgung der hoch bewerteten Seite C, während B nur die Hälfte des Beitrags von A erhält. Seite D besitzt wegen der Teleportation einen positiven Wert. Beachten Sie außerdem, dass das ungedämpfte Modell A und C gleiche Werte zuwies; eine Änderung des Modells kann die Rangfolge verändern.

Innehalten und nachdenken – Programmieraufgabe

Implementieren Sie Algorithmus 20 mit Adjazenzlisten. Reproduzieren Sie Tabelle 1 und entfernen Sie dann die Kante (D, C), um eine Seite ohne ausgehende Links zu erzeugen. Prüfen Sie, dass sich jeder Iterationsvektor weiterhin zu eins summiert. Was geschieht, wenn Sie vergessen, die Wahrscheinlichkeit der Seite ohne ausgehende Links zu verteilen? Vergleichen Sie schließlich die Ergebnisse für $\alpha = 0.5$ und $\alpha = 0.95$.

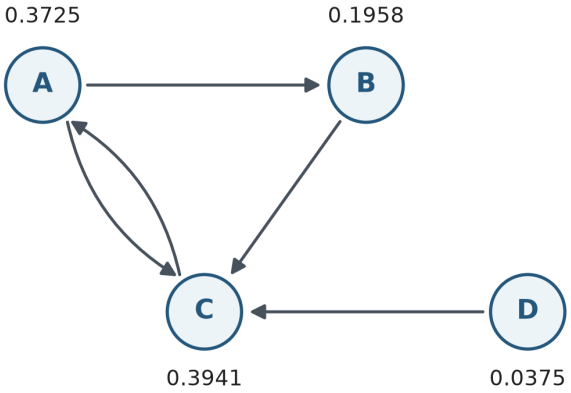


Abbildung 27: Derselbe Webgraph mit seinen PageRank-Werten für $\alpha = 0.85$.

8.6 Weitere Bedeutungen von Zentralität

PageRank weist Wichtigkeit gemäß einem bestimmten Fluss von Empfehlungen zu. Was aber, wenn unsere Knoten Menschen, Router oder Orte statt Webseiten darstellen? Ein nützliches Zentralitätsmaß muss die Frage widerspiegeln, die wir beantworten wollen. Wir vergleichen nun lokale Vernetzung, kurze Distanzen, Vermittlung entlang kürzester Pfade und Verbindungen zu wichtigen Nachbarn. Keines davon ist eine universelle Definition von Wichtigkeit; seine Annahmen und sein Berechnungsaufwand müssen zur Anwendung passen.

8.6.1 Gradzentralität

Die einfachste Wahl ist der bereits bekannte Grad. Für einen einfachen ungerichteten Graphen mit $n > 1$ ist die normierte *Gradzentralität*

$$C_D(v) = \frac{\deg(v)}{n-1}.$$

Der Nenner ist der größtmögliche Grad. Für einen gerichteten Graphen ohne Schleifen können wir entsprechend Eingangsgrad- und Ausgangsgradzentralität mit $\text{indeg}(v)/(n-1)$ und $\text{outdeg}(v)/(n-1)$ definieren. Der Ausschluss von Schleifen ist für diese oberen Schranken wichtig.

Gradzentralität ist lokal: Sie betrachtet nur unmittelbare Nachbarn. In einem Kontaktnetzwerk beschreibt sie die Anzahl direkter Kontakte; in einem Webgraphen zählt der Eingangsgrad eingehende und der Ausgangsgrad ausgehende Links. Für alle Knoten lässt sie sich aus Adjazenzlisten kostengünstig in $O(|V| + |E|)$ Zeit berechnen. Unser PageRank-Beispiel zeigte jedoch bereits, dass gleicher Eingangsgrad unter einem anderen Modell nicht gleiche Wichtigkeit bedeuten muss.

8.6.2 Nähezentralität

Vielleicht ist ein wichtiger Knoten einer, den andere schnell erreichen können. Sei $d(u, v)$ die Distanz eines kürzesten Pfades von u nach v, wie in Abschnitt 5 untersucht. Für einen zusammenhängenden ungewichteten ungerichteten Graphen mit $n > 1$ definieren wir

$$C_C(v) = \frac{n-1}{\sum_{u \in V \setminus \{v\}} d(u, v)}.$$

Diese normierte *Nähezentralität* (engl. closeness centrality) ist der Kehrwert der durchschnittlichen Distanz zu allen anderen Knoten. Ein größerer Wert bedeutet kürzere Distanzen. Die unnormierte Variante lässt den Zähler $n-1$ weg und liefert innerhalb eines festen Graphen dieselbe Rangfolge.

Bei gerichteten Graphen müssen wir eine Richtung wählen: Eingehende Nähezentralität verwendet wie oben $d(u, v)$, ausgehende Nähezentralität dagegen $d(v, u)$. Ein Ort, der für andere leicht

erreichbar ist, muss kein Ort sein, von dem aus andere leicht erreichbar sind. Eingehende Nähezentralität könnte die Erreichbarkeit eines Dienstes modellieren, ausgehende Nähezentralität dagegen, wie schnell ein Sender Empfänger erreichen kann.

Können Knoten einander nicht erreichen, sind einige Distanzen unendlich und die Formel benötigt eine Konvention. Unerreichbare Knoten einfach wegzulassen kann kleine Komponenten irreführend belohnen. Eine Alternative ist die *harmonische Zentralität*,

$$C_H(v) = \frac{1}{n-1} \sum_{u \neq v} \frac{1}{d(u,v)}, \qquad 1/\infty := 0.$$

Unerreichbare Knoten tragen dann null bei. Dies ist ein anderes Maß, nicht bloß eine andere Auswertung derselben Formel. Bei gewichteten Graphen müssen Distanzen sinnvolle Reisekosten beschreiben, wobei die Distanzen zwischen verschiedenen Knoten positiv sein müssen, damit diese Kehrwerte definiert sind.

8.6.3 Betweenness-Zentralität

Ein Knoten kann wenige Nachbarn haben, aber ansonsten getrennte Teile eines Netzwerks verbinden. Abbildung 28 zeigt zwei Dreiecke, die über D verbunden sind. Knoten D hat Grad 2, während C und E Grad 3 haben. Dennoch müssen alle Pfade zwischen den beiden Dreiecken durch D verlaufen.

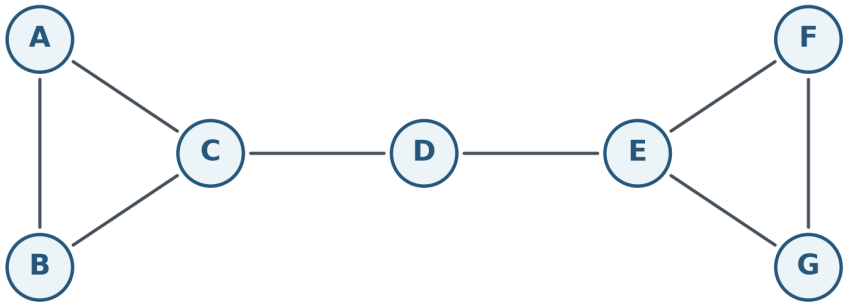


Abbildung 28: Zwei Gruppen, verbunden durch einen Knoten geringen Grades. Die Interpretation von Wichtigkeit hängt vom Zentralitätsmaß ab.

Definition 8.8. Sei σ_{st} die Anzahl kürzester Pfade von s nach t und $\sigma_{st}(v)$ die Anzahl dieser Pfade, die v als inneren Knoten enthalten. Die unnormierte *Betweenness-Zentralität* von v in einem ungerichteten Graphen ist

$$C_B(v) = \sum_{\{s,t\} \in [V \setminus \{v\}]^2} \frac{\sigma_{st}(v)}{\sigma_{st}}.$$

Paare ohne Pfad tragen null bei. Jedes ungeordnete Paar wird einmal gezählt, und Endpunkte zählen nicht als innere Knoten.

Für $n > 2$ normiert die Division durch $\binom{n-1}{2}$ dieses Maß auf $[0, 1]$. In einem gerichteten Graphen summieren wir stattdessen über geordnete Paare verschiedener Endpunkte und normieren durch $(n-1)(n-2)$. Existieren zwei gleich kurze Pfade und verwendet nur einer davon v, trägt ihr Paar $1/2$ bei, nicht 1.

In Abbildung 28 gibt es $3 \cdot 3 = 9$ Paare mit je einem Endpunkt in jedem Dreieck. Jedes besitzt einen eindeutigen kürzesten Pfad durch D, daher gilt $C_B(D) = 9$ und sein normierter Wert ist $9/15 = 0.6$. Zum Vergleich gilt $C_B(C) = C_B(E) = 8$, und die übrigen Knoten haben Betweenness null. Außerdem summieren sich die Distanzen von D zu den anderen Knoten zu 10,

was $C_C(D) = 6/10$ ergibt, während sich die von C zu 11 summieren und $C_C(C) = 6/11$ ergeben. Der Knoten mit kleinerem Grad ist sowohl im Mittel näher als auch auf mehr kürzesten Pfaden gelegen.

Betweenness kann Vermittler zwischen sozialen Gruppen oder mögliche Engpässe in einem Kommunikationsnetzwerk identifizieren. Sie setzt jedoch voraus, dass kürzeste Pfade für den modellierten Prozess relevant sind. Folgt der Verkehr anderen Routen, muss ein hoher Betweenness-Wert die tatsächliche Last nicht vorhersagen.

Innehalten und nachdenken – Selbst erarbeiten

Überprüfen Sie den Wert $C_B(C) = 8$, indem Sie den Graphen nach Entfernen von C in Teile zerlegen. Fügen Sie dann dem ursprünglichen Graphen die Kante $\{B, F\}$ hinzu. Sagen Sie vor jeder Berechnung voraus, welche Knoten Betweenness verlieren werden. Muss auch ihr Grad abnehmen?

Wiederholte BFS berechnet alle ungewichteten Distanzen in $O(|V|(|V|+|E|))$ Zeit. Betweenness benötigt zusätzlich Anzahlen kürzester Pfade und deren Beiträge; ein Vorgänger pro Knoten verliert alternative kürzeste Pfade. Brandes' Algorithmus berechnet Betweenness für zusammenhängende ungewichtete Graphen in $O(|V||E|)$ Zeit [5]. Dieser Aufwand ist bei der Wahl eines Maßes für ein großes Netzwerk relevant.

8.6.4 Eigenvektorzentralität

Kehren wir schließlich zur Idee zurück, dass eine Verbindung zu einem wichtigen Knoten mehr zählen sollte. Für einen zusammenhängenden ungerichteten Graphen mit mindestens zwei Knoten und Adjazenzmatrix A weist die *Eigenvektorzentralität* einen nichtnegativen Bewertungsvektor x zu, der folgende Gleichungen erfüllt:

$$\lambda x_v = \sum_{u \in N(v)} x_u, \qquad Ax = \lambda x.$$

Wählen wir den größten Eigenwert und normieren etwa durch $\sum_v x_v = 1$. Die Perron–Frobenius-Theorie garantiert unter diesen Annahmen einen eindeutigen strikt positiven normierten Vektor. Ein beliebiger Eigenvektor kann negative Einträge haben und wäre für diesen Zweck ungeeignet. Auf bipartiten Graphen kann naive Potenziteration dennoch oszillieren: Existenz und Konvergenz sind getrennte Fragen.

Bei gerichteten Graphen summiert eingehende Eigenvektorzentralität über $Pre(v)$ und verwendet A^T, wenn Zeilen Quellen darstellen. Nicht zusammenhängende oder reduzible Graphen müssen keine eindeutigen strikt positiven Bewertungen besitzen.

Einfache Eigenvektorzentralität gibt den gesamten Wert eines Nachbarn entlang jeder Kante weiter. PageRank verteilt ihn auf die ausgehenden Links und fügt Teleportation hinzu. Mit wichtigen Personen verbunden zu sein und einen großen Anteil zirkulierender Aufmerksamkeit zu erhalten kann daher unterschiedliche Rangfolgen ergeben.

Innehalten und nachdenken – Kurzcheck

Sie sollen eine „wichtige“ Person in einem sozialen Netzwerk finden. Würden Sie jemanden mit vielen direkten Kontakten wählen, jemanden mit geringer Distanz zu allen, jemanden, der verschiedene Gruppen verbindet, oder jemanden, der mit anderen wichtigen Personen verbunden ist? Nennen Sie für jede Antwort eine Anwendung, in der sie sinnvoll wäre.

9 Flussnetzwerke

Wie viel können wir durch ein Netzwerk transportieren? Stellen Sie sich vor, ein Unternehmen schickt Waren von einer Fabrik zu einem Verteilzentrum. Mehrere Routen stehen zur Verfügung, aber jede Straße hat eine begrenzte Kapazität. Alles über einen kürzesten Pfad zu schicken funktioniert möglicherweise nicht: Die kürzeste Route kann vielleicht nur einen kleinen Teil der Lieferung aufnehmen. Nun wollen wir mehrere Routen gleichzeitig nutzen.

Flussprobleme wurden in den 1950er-Jahren zu einem wichtigen Thema des Operations Research. Ford und Fulkerson veröffentlichten 1956 ihre Arbeit zum maximalen Fluss, motiviert durch Fragen zum Transport durch Netzwerke.[4] Heute kann dasselbe Modell Kommunikationsbandbreite, Transport oder die Zuordnung von Aufgaben zu Personen beschreiben. Die letzte Anwendung mag überraschen. Was hat die Zuordnung einer Person zu einer Aufgabe mit dem Transport von etwas zu tun? Das werden wir beim Aufbau des Modells herausfinden.

Hintergrund

Stellen Sie sich einen Fluss als Rate vor, etwa Einheiten pro Stunde, in einem stationären Zustand. Ein Zwischenknoten sammelt nicht dauerhaft Waren an. Alles, was hineinfließt, muss wieder hinausfließen. Die Rohrleitungsanalogie ist für Kapazitäten und Erhaltung nützlich, doch unser Modell beschreibt weder Wasserdruck noch Reisezeiten oder vorübergehende Speicherung.

9.1 Problem des maximalen Flusses

9.1.1 Vom Transportnetzwerk zum Fluss

Gerichtete Graphen und Gewichtsfunktionen kennen wir bereits. Hier beschreibt das Gewicht einer Kante ihre *Kapazität* (engl. capacity), die größte Flussmenge, die die Kante aufnehmen kann.

Definition 9.1. Ein *Flussnetzwerk* ist ein 4-Tupel $N = (G, \omega, s, t)$, wobei $G = (V, E)$ ein endlicher gerichteter Graph ist, $\omega : E \to \mathbb{R}_{\geq 0}$ jeder Kante eine endliche Kapazität zuweist und $s, t \in V$ mit $s \neq t$ die *Quelle* (engl. source) und die *Senke* (engl. sink) sind.

Beachten Sie den Definitionsbereich von ω: Er ist E, denn die Einschränkungen gehören zu den Kanten. Für die Kapazität der Kante (u, v) schreiben wir $\omega(u, v)$ statt $\omega((u, v))$. Die Knoten s und t sind ausgezeichnete Knoten im Graphen, keine zusätzlichen Objekte außerhalb davon.

Bevor wir einen Fluss definieren, vereinfachen wir unsere Graphen an einer Stelle. Ein gerichteter Graph kann sowohl (u, v) als auch (v, u) enthalten. Später führen wir eine Rückwärtskante mit besonderer Bedeutung ein. Wäre bereits eine ursprüngliche Kante in Gegenrichtung vorhanden, müssten wir zwei unterschiedliche Gründe für die Nutzung derselben Richtung unterscheiden. Diese Ablenkung können wir vorerst vermeiden.

Definition 9.2. Ein *orientierter Graph* ist ein gerichteter Graph $G = (V, E)$ ohne Schleifen und ohne Paare entgegengesetzt gerichteter Kanten. Formal gilt

$$(v, v) \notin E \quad \text{für alle } v \in V, \qquad (u, v) \in E \implies (v, u) \notin E.$$

Gleichwertig lässt er sich aus einem einfachen ungerichteten Graphen gewinnen, indem für jede Kante eine Richtung gewählt wird.

Ein orientierter Graph kann trotzdem gerichtete Kreise enthalten! Für die folgenden Definitionen ist unser ursprünglicher Netzwerkgraph orientiert. Für den maximalen Fluss nehmen wir außerdem an, dass keine Kante in s hineinführt und keine Kante t verlässt. Diese Annahmen halten die Notation übersichtlich. Entgegengesetzte ursprüngliche Kanten können behandelt werden, indem eine davon durch eine Route mit zwei Kanten über einen neuen Knoten ersetzt wird, wobei beide Kanten dieselbe Kapazität erhalten.

[4] L. R. Ford, Jr. und D. R. Fulkerson, *Maximal Flow Through a Network*, Canadian Journal of Mathematics 8 (1956), S. 399–404. https://doi.org/10.4153/CJM-1956-045-5

Innehalten und nachdenken – Kurzcheck

Ist der gerichtete Kreis $a \to b \to c \to a$ ein orientierter Graph? Was ändert sich, wenn wir die Kante $b \to a$ hinzufügen?

Das Netzwerk beschreibt, was möglich ist. Ein Fluss beschreibt eine konkrete Wahl, wie viel über jede Kante geschickt wird. Kapazitäten sind somit Eingabedaten, Flusswerte dagegen Entscheidungen.

Definition 9.3. Ein *zulässiger Fluss* (engl. feasible flow) in N ist eine Funktion $f : E \to \mathbb{R}_{\geq 0}$, die folgende zwei Bedingungen erfüllt:

$$0 \leq f(u,v) \leq \omega(u,v) \qquad \text{für jedes } (u,v) \in E, \tag{9}$$

$$\sum_{u \in Pre(v)} f(u,v) = \sum_{w \in Suc(v)} f(v,w) \qquad \text{für jedes } v \in V \setminus \{s,t\}. \tag{10}$$

Die erste Bedingung ist die *Kapazitätsbedingung*. Die zweite ist die *Flusserhaltung* (engl. flow conservation): An jedem Zwischenknoten ist der eingehende Fluss gleich dem ausgehenden Fluss.

Erinnern wir uns an Abschnitt 1.2: $Pre(v)$ und $Suc(v)$ enthalten die Vorgänger beziehungsweise Nachfolger von v. Die Summen berücksichtigen daher nur vorhandene Kanten. Eine leere Summe hat den Wert null.

Definition 9.4. Der *Wert* eines Flusses ist die Nettomenge, die die Quelle verlässt:

$$|f| = \sum_{v \in Suc(s)} f(s,v) - \sum_{u \in Pre(s)} f(u,s).$$

Unter unserer Annahme, dass keine Kante in s hineinführt, ist dies einfach die erste Summe. Das *Problem des maximalen Flusses* fragt nach einem zulässigen Fluss mit größtmöglichem Wert.

Die Notation $|f|$ bedeutet nicht, dass wir Kanten zählen oder alle Kantenflüsse addieren. Eine Einheit durch drei Kanten zu schicken liefert weiterhin nur eine Einheit. Aus der Erhaltung folgt, dass der Wert auch der Nettomenge entspricht, die in t eintritt: Summieren wir die Bilanzen über alle Knoten, tritt jede Kante einmal als ausgehend und einmal als eingehend auf und hebt sich auf.

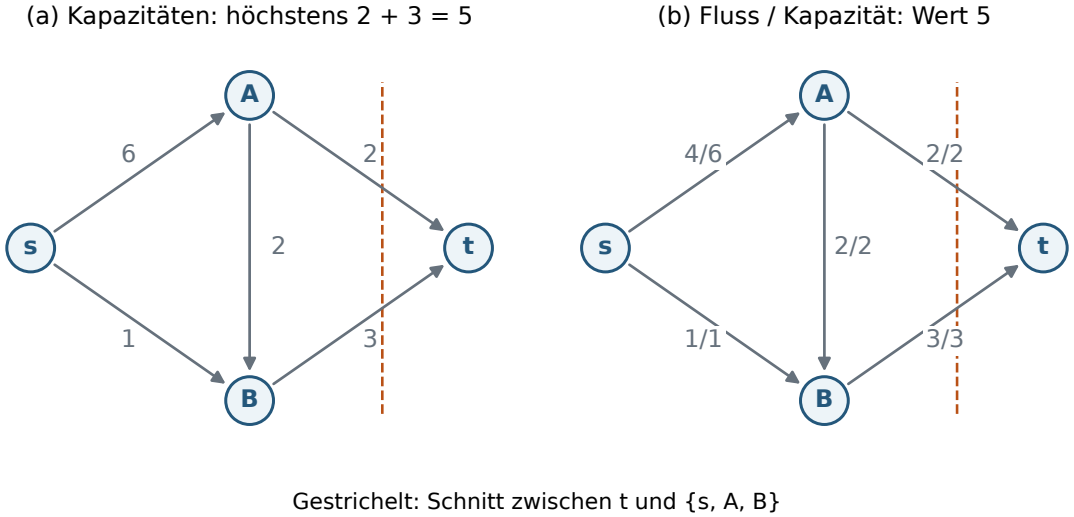


Abbildung 29: Das Netzwerk aus dem einführenden Flussbeispiel der Folien. Die Kapazitäten aus s summieren sich zu 7, aber die Senke kann höchstens 5 aufnehmen. Die rechte Zeichnung zeigt einen zulässigen Fluss mit Wert 5; die Beschriftungen bedeuten dort Fluss/Kapazität.

Betrachten wir Abbildung 29. Wir können vier Einheiten von s nach A schicken, von denen zwei direkt nach t und zwei weiter über B gehen. Eine weitere Einheit geht direkt von s nach B, sodass drei Einheiten B in Richtung t verlassen. Prüfen Sie die Erhaltung an beiden Zwischenknoten. Wir liefern fünf Einheiten, obwohl die Summe der Flusswerte auf allen fünf Kanten zwölf beträgt.

9.1.2 Schnitte liefern eine obere Schranke

Woher wissen wir, dass fünf Einheiten das Bestmögliche sind? Jede Einheit muss letztlich in t eintreten. Die beiden in t eingehenden Kanten haben die Gesamtkapazität $2 + 3 = 5$. Keine Routenwahl kann diese Grenze überwinden. Wir können diese Beobachtung verallgemeinern, indem wir irgendwo zwischen s und t eine gedachte Grenze ziehen.

Definition 9.5. Ein *s-t-Schnitt* ist eine Zerlegung $(S, V \setminus S)$ der Knoten mit $s \in S$ und $t \notin S$. Seine *Kapazität* ist

$$\omega(S, V \setminus S) = \sum_{\substack{(u,v)\in E \\ u\in S,\ v\notin S}} \omega(u, v).$$

Nur Kanten, die von der Quellseite zur Senkenseite gerichtet sind, tragen zu dieser Kapazität bei.

Ein Schnitt verlangt nicht, dass eine seiner Seiten zusammenhängend ist. Wir zerlegen die Knoten; die zwischen den Teilen verlaufenden Kanten ergeben sich dann aus dieser Wahl. Summieren wir die Erhaltung über die Knoten in S außer der Quelle, erhalten wir

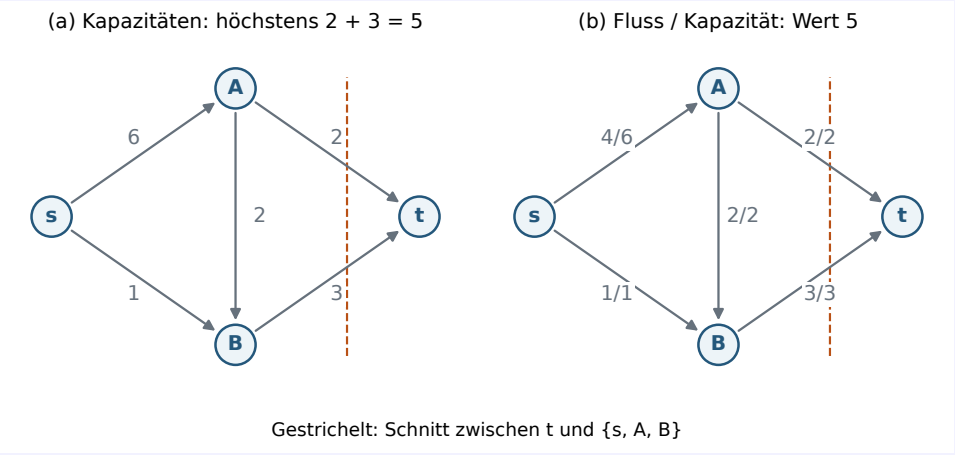
$$|f| = \sum_{\substack{(u,v)\in E \\ u\in S,\ v\notin S}} f(u, v) - \sum_{\substack{(u,v)\in E \\ u\notin S,\ v\in S}} f(u, v) \ \le\ \omega(S, V \setminus S). \tag{11}$$

Alle Flüsse innerhalb von S heben sich auf. Fluss, der nach S zurückkehrt, muss abgezogen werden, da er nicht zur Nettolieferung beiträgt. Somit ist *jede Schnittkapazität eine obere Schranke für jeden zulässigen Flusswert.*

Wählen wir in Abbildung 29 die Menge $S = \{s, A, B\}$. Ihre Kapazität ist fünf, und wir haben einen Fluss mit Wert fünf gefunden. Zusammen zertifizieren diese beiden Objekte Optimalität. Ein Fluss allein liefert eine untere Schranke für das Erreichbare; ein Schnitt liefert eine obere Schranke. Stimmen sie überein, lässt sich nichts mehr verbessern.

Innehalten und nachdenken – Selbst erarbeiten

Berechnen Sie für dasselbe Netzwerk die Kapazitäten der Schnitte mit den Quellseiten $\{s\}$, $\{s, A\}$ und $\{s, B\}$. Welcher davon zertifiziert ebenfalls, dass der dargestellte Fluss maximal ist?



9.1.3 Der Residualgraph: Was können wir noch ändern?

Wir könnten einen Fluss konstruieren, indem wir einen s-t-Pfad suchen, so viel wie möglich darüber schicken und dies wiederholen. Was aber, wenn ein Pfad eine Kante verwendet, die wir später für eine andere Route benötigen? Ein nützlicher Algorithmus muss frühere Entscheidungen revidieren dürfen.
Nehmen wir eine ursprüngliche Kante (u, v) mit Kapazität fünf, die aktuell zwei Einheiten trägt. Es gibt genau zwei mögliche Änderungen: ihren Fluss um höchstens drei erhöhen oder um höchstens zwei verringern. Die erste Änderung stellen wir durch einen Pfeil $u \to v$ dar, die zweite durch einen Pfeil $v \to u$.



Abbildung 30: Eine ursprüngliche Kante ergibt zwei mögliche Anpassungen. Eine Vorwärts-Residualkante erlaubt zusätzlichen Fluss; eine Rückwärts-Residualkante erlaubt, Fluss zurückzunehmen. Residualbeschriftungen geben die maximal erlaubte Anpassung an.

Definition 9.6. Für einen zulässigen Fluss f in einem orientierten Netzwerk ist sein *Residualgraph*, auch Restnetz (engl. residual graph), $G_f = (V, E_f)$, wobei

$$E_f = \{(u,v) \mid (u,v) \in E,\ f(u,v) < \omega(u,v)\} \\ \cup \{(v,u) \mid (u,v) \in E,\ f(u,v) > 0\}.$$

Seine *Residualkapazität* $r_f : E_f \to \mathbb{R}_{>0}$ ist definiert durch

$$r_f(u,v) = \begin{cases} \omega(u,v) - f(u,v), & \text{falls } (u,v) \in E, \\ f(v,u), & \text{falls } (v,u) \in E. \end{cases}$$

Wir nennen diese Fälle *vorwärts* beziehungsweise *rückwärts*. Kanten mit Residualkapazität null werden weggelassen.

Die beiden Fälle sind eindeutig, weil der ursprüngliche Graph keine entgegengesetzten Kanten hat. Der Residualgraph kann jedoch beide Richtungen enthalten und ist daher nicht notwendigerweise orientiert. Wir verwenden einen anderen Graphen für eine andere Frage: G beschreibt Transportmöglichkeiten, während G_f zulässige Anpassungen am aktuellen Fluss beschreibt.

Notation

Wir speichern f nur auf ursprünglichen Kanten, und jeder gespeicherte Wert bleibt nichtnegativ. Eine Rückwärts-Residualkante ist keine neue Transportverbindung und trägt keinen negativen physikalischen Fluss. Das Durchlaufen von $v \to u$ im Residualgraphen bedeutet, dass wir vom gespeicherten Wert $f(u,v)$ etwas *abziehen*.

Innehalten und nachdenken – Kurzcheck

Zeichnen Sie für eine Kante mit Kapazität fünf die Residualkanten bei Fluss null, zwei und fünf. In welchem Fall existieren beide Residualrichtungen?

Angenommen, wir finden nun einen gerichteten s-t-Pfad P in G_f. Wir können alle seine Anpassungen kombinieren, sofern keine ihre Residualkapazität überschreitet.

Definition 9.7. Ein *augmentierender Pfad* ist ein gerichteter s-t-Pfad im Residualgraphen. Wie zuvor wiederholt ein Pfad keine Knoten. Seine *Engpasskapazität* ist

$$\Delta = \min_{(u,v) \text{ auf } P} r_f(u,v).$$

Um entlang von P zu *augmentieren*, aktualisieren wir für jede Residualkante (u, v) auf dem Pfad die entsprechende ursprüngliche Kante:

$$\begin{cases} f(u,v) \leftarrow f(u,v) + \Delta, & \text{falls } (u,v) \in E, \\ f(v,u) \leftarrow f(v,u) - \Delta, & \text{falls } (v,u) \in E. \end{cases}$$

Alle anderen ursprünglichen Kantenflüsse bleiben unverändert.

Warum bleibt dabei die Erhaltung gewahrt? Ein Residualschritt $u \to v$ erhöht den Nettoausfluss bei u um Δ und verringert ihn bei v um Δ, unabhängig davon, ob wir Vorwärtsfluss hinzufügen oder Rückwärtsfluss zurücknehmen. An einem inneren Pfadknoten heben sich eingehende und ausgehende Anpassungen auf. Nur s gewinnt Nettoausfluss und t Nettozufluss. Der neue Fluss ist daher zulässig, und sein Wert steigt um Δ. Das Minimum in der Definition von Δ garantiert die Kapazitätsgrenzen einschließlich der Nichtnegativität beim Subtrahieren.

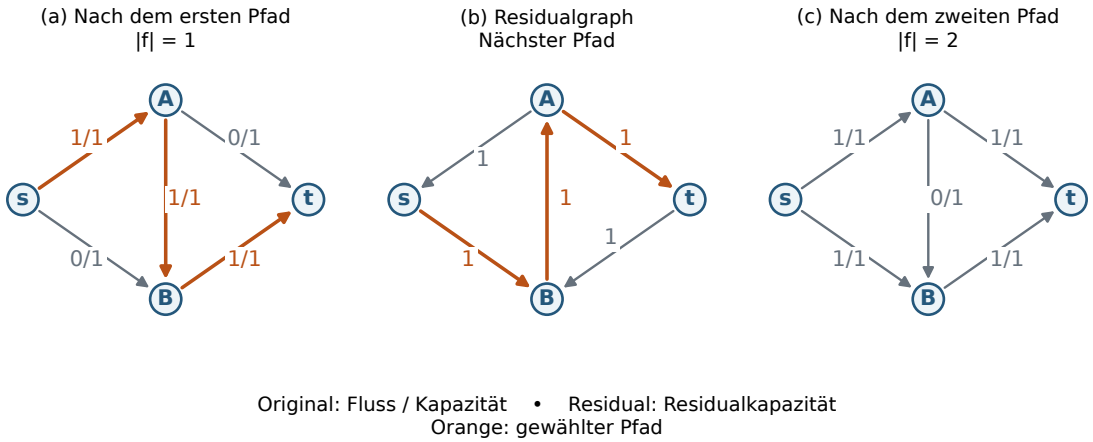


Abbildung 31: Alle ursprünglichen Kapazitäten sind eins. Nach der Wahl von s, A, B, t ist keine reine Vorwärtsaugmentation möglich. Der Residualpfad s, B, A, t nimmt $A \to B$ zurück und erzeugt einen Fluss mit Wert zwei.

Abbildung 31 zeigt ein kleines Beispiel, in dem das Zurücknehmen wesentlich ist. Zunächst schicken wir eine Einheit entlang von (s, A, B, t). Die Kanten $s \to A$ und $B \to t$ sind nun voll. Ein Algorithmus, der nur Vorwärtskanten betrachtet, bleibt beim Wert eins stecken. Der Residualpfad (s, B, A, t) ist jedoch verfügbar. Sein mittlerer Schritt nimmt die Einheit auf $A \to B$ zurück, während seine übrigen Schritte je eine Einheit auf $s \to B$ und $A \to t$ hinzufügen. Wir erhalten je eine Einheit auf den Routen (s, A, t) und (s, B, t).

Nichts muss physisch rückwärts transportiert werden. Wir berechnen einen Plan und ersetzen einen früheren Plan durch einen besseren. Diese Interpretation hilft uns auch, den Residualgraphen zu verstehen, wenn wir Flüsse zur Darstellung von Zuordnungen verwenden.

9.1.4 Ford-Fulkerson-Algorithmus

Nun haben wir alle Bestandteile. Wir beginnen mit dem Nullfluss, suchen einen augmentierenden Pfad und wenden seine Anpassungen an. Das wiederholen wir, bis kein augmentierender Pfad mehr existiert. Algorithmus 21 beschreibt die Ford-Fulkerson-Methode. Er lässt absichtlich offen, *welcher* Pfad gewählt wird.

Wir haben argumentiert, dass jede Augmentation zulässig ist und den Wert erhöht. Eine lokal nicht verbesserbare Lösung ist aber nicht automatisch global optimal. Warum reicht hier das Fehlen eines augmentierenden Pfades aus?

Satz 9.8 (Max-Flow-Min-Cut)**.** *Der maximale Wert eines zulässigen Flusses ist gleich der minimalen Kapazität eines s-t-Schnitts. Insbesondere ist ein zulässiger Fluss genau dann maximal, wenn sein Residualgraph keinen s-t-Pfad enthält.*

Algorithmus 21 Ford-Fulkerson-Methode

Eingabe: Ein orientiertes Netzwerk $N = (G, \omega, s, t)$ mit ganzzahligen Kapazitäten
Ausgabe: Ein maximaler Fluss f

```
1:  f(e) ← 0 für jedes e ∈ E
2:  Konstruiere G_f und r_f
3:  solange ein s-t-Pfad P in G_f existiert tue
4:      Δ ← min_{(u,v) auf P} r_f(u, v)
5:      für jede Residualkante (u, v) auf P tue
6:          wenn (u, v) ∈ E dann
7:              f(u, v) ← f(u, v) + Δ
8:          sonst
9:              f(v, u) ← f(v, u) − Δ
10:         Ende wenn
11:     Ende für
12:     Aktualisiere G_f und r_f mit dem neuen Fluss
13: Ende solange
14: gib zurück f
```

Beweis. Existiert ein augmentierender Pfad, können wir den Wert erhöhen; der aktuelle Fluss ist also nicht maximal. Andernfalls sei S die Menge aller von s aus in G_f erreichbaren Knoten. Da t nicht erreichbar ist, ist $(S, V \setminus S)$ ein s-t-Schnitt.

Jede ursprüngliche Kante von S nach $V \setminus S$ muss gesättigt sein. Hätte sie freie Kapazität, würde ihre Vorwärts-Residualkante ihren Endknoten erreichbar machen. Jede ursprüngliche Kante von $V \setminus S$ nach S muss Fluss null tragen. Trüge sie positiven Fluss, würde ihre Rückwärts-Residualkante wiederum von S zu einem angeblich unerreichbaren Knoten führen.

Setzen wir diese Tatsachen in Gleichung (11) ein: Die ausgehenden Terme entsprechen ihren Kapazitäten, die eingehenden sind null. Daher gilt $|f| = \omega(S, V \setminus S)$. Kein zulässiger Fluss kann irgendeine Schnittkapazität überschreiten, also ist dieser Fluss maximal und dieser Schnitt minimal. Die Anwendung des Arguments auf einen maximalen Fluss beweist die Gleichheit. □

Der Beweis liefert auch ein praktisches Ergebnis: Eine abschließende Graphsuche erzeugt ein Optimalitätszertifikat. Die Quellseite besteht aus den erreichbaren Knoten, auch wenn sie ganz anders als eine senkrechte Linie durch die Zeichnung aussieht. Für ganzzahlige Kapazitäten garantiert das folgende Argument außerdem, dass der Algorithmus einen solchen Fluss erreicht.[5]

Satz 9.9 (Ganzzahligkeit)**.** *Ist jede Kapazität ganzzahlig, terminiert Ford-Fulkerson und erzeugt einen ganzzahligen maximalen Fluss.*

Beweis. Anfangs sind alle Flüsse null. Sind alle aktuellen Flüsse ganzzahlig, dann sind auch alle Residualkapazitäten ganzzahlig. Ihr positives Minimum Δ ist eine ganze Zahl, daher erhält die Augmentation die Ganzzahligkeit und erhöht den Wert um mindestens eins. Der Wert ist durch die endliche Gesamtkapazität der aus s ausgehenden Kanten nach oben beschränkt. Daher sind nur endlich viele Augmentationen möglich, und der abschließende Fluss ist nach Satz 9.8 maximal. □

Dies ist zugleich eine Existenzaussage: Ganzzahlige Kapazitäten garantieren, dass *ein* ganzzahliges Optimum existiert. Sie besagen nicht, dass jeder zulässige Fluss oder jedes Optimum ganzzahlig sein muss.

Ist F der ganzzahlige maximale Flusswert, gibt es höchstens F Augmentationen. Eine Suche samt Residualaktualisierung mit Adjazenzlisten benötigt $O(|V|+|E|)$ Zeit, insgesamt also $O((|V|+|E|)(F+1))$ einschließlich der abschließenden Suche. Nach Entfernen isolierter Knoten lautet die übliche Kurzform $O(|E|F)$ für $F \geq 1$. Die Abhängigkeit von einem Zahlenwert ist wichtig: Eine große Kapazität binär darzustellen benötigt viel weniger als entsprechend viele Bits. Dies ist eine *pseudopolynomielle* Schranke, keine polynomielle Schranke in der Eingabelänge.

[5] Für reelle Kapazitäten folgt die Existenz eines Maximums aus einem Satz der Analysis: Eine stetige Funktion nimmt auf einer nichtleeren kompakten Menge ein Maximum an. Die zulässigen Flüsse bilden eine abgeschlossene und beschränkte Teilmenge des endlichdimensionalen Quaders $\prod_{e \in E}[0, \omega(e)]$.

9.1.5 Warum die Pfadwahl wichtig ist

Betrachten wir das Netzwerk mit Kapazitäten

$$\omega(s, A) = \omega(A, t) = \omega(s, B) = \omega(B, t) = 100, \qquad \omega(A, B) = 1.$$

Der maximale Fluss ist offensichtlich 200: Je 100 Einheiten werden entlang von (s, A, t) und (s, B, t) geschickt. Angenommen jedoch, unsere Pfadsuche wählt zuerst (s, A, B, t) und dann (s, B, A, t) im Residualgraphen.

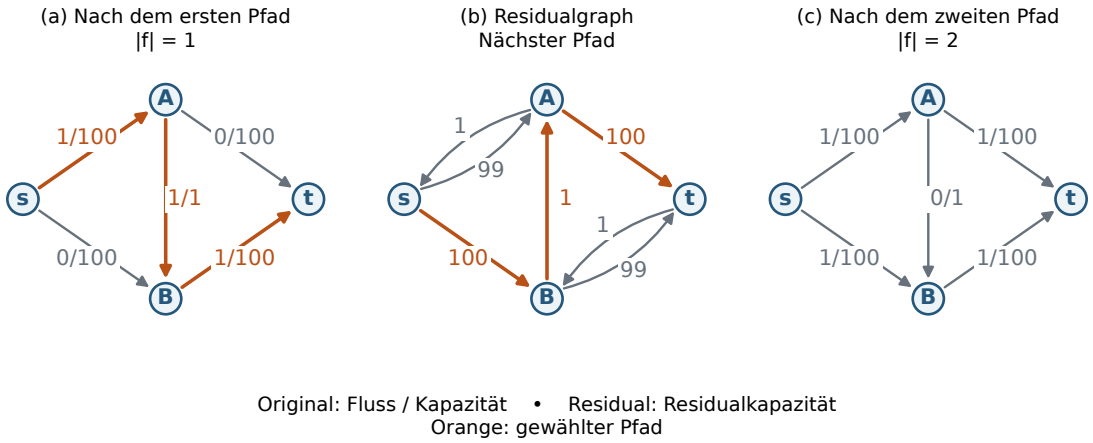


Abbildung 32: Eine zulässige, aber ungünstige Pfadwahl. Die mittlere Kante abwechselnd zu verwenden und zurückzunehmen erhöht den Fluss pro Augmentation nur um eine Einheit. Große äußere Kapazitäten helfen bei dieser Pfadwahl nicht.

Schritt	Residualpfad	$\lvert f\rvert$	$f(s, A)$	$f(A, t)$	$f(s, B)$	$f(B, t)$	$f(A, B)$
0	–	0	0	0	0	0	0
1	(s, A, B, t)	1	1	0	0	1	1
2	(s, B, A, t)	2	1	1	1	1	0
3	(s, A, B, t)	3	2	1	1	2	1
4	(s, B, A, t)	4	2	2	2	2	0

Nach $2k$ Augmentationen tragen alle vier äußeren Kanten k Einheiten und die mittlere Kante null. Nach 200 Augmentationen erreichen wir schließlich das Maximum. Ersetzen wir 100 durch eine größere ganze Zahl, wird das Beispiel beliebig langsam, obwohl seine Struktur mit fünf Kanten unverändert bleibt.

Hintergrund

Ford-Fulkerson liefert in diesem Beispiel keine falsche Antwort; es macht nur geringe Fortschritte. Auch rationale Kapazitäten garantieren Terminierung, da wir sie zu ganzen Zahlen skalieren können. Bei irrationalen Kapazitäten können einige Netzwerke und Pfadwahlen unendlich viele Augmentationen verursachen, und der Grenzwert muss nicht maximal sein. Das ist ein anderer Fehlerfall als das Verhalten im obigen ganzzahligen Beispiel.

9.1.6 Edmonds-Karp-Algorithmus

Wie könnten wir dieses Verhalten verhindern, ohne eine völlig andere Methode zu erfinden? Edmonds-Karp verwendet Ford-Fulkerson mit einer präzisen Regel: Wähle stets einen augmentierenden Pfad mit den *wenigsten Kanten*. Eine Breitensuche (BFS) im aktuellen Residualgraphen findet einen solchen Pfad. Residualkapazitäten bestimmen, ob eine Kante verfügbar ist und um wie viel wir augmentieren können; sie sind keine Distanzen für die BFS.
Im selben 100–1-Netzwerk wählt BFS (s, A, t) oder (s, B, t): Beide haben zwei Kanten, während die Pfade über die mittlere Kante drei haben. Die erste Augmentation schickt 100 Einheiten. Die zweite

Algorithmus 22 Edmonds-Karp-Algorithmus

Eingabe: Ein orientiertes Netzwerk $N = (G, \omega, s, t)$
Ausgabe: Ein maximaler Fluss f

```
1: f(e) ← 0 für jedes e ∈ E
2: Konstruiere G_f und r_f
3: solange BFS von s aus t in G_f erreicht tue
4:     Rekonstruiere einen kürzesten s-t-Pfad P aus den BFS-Vorgängern
5:     Δ ← min_{(u,v) auf P} r_f(u, v)
6:     Augmentiere f entlang von P wie in Algorithmus 21
7:     Aktualisiere G_f und r_f
8: Ende solange
9: gib zurück f
```

schickt 100 über den anderen Zweikantenpfad. Zwei Augmentationen lösen genau die Instanz, für die die abwechselnden Entscheidungen 200 benötigten.

Satz 9.10. *Edmonds-Karp führt $O(|V||E|)$ Augmentationen aus und hat nach Entfernen isolierter Knoten die Laufzeit $O(|V||E|^2)$. Die Schranke hängt nicht von den Kapazitätswerten ab.*

Beweis. Wir skizzieren das Argument. Sei $d_f(v)$ die BFS-Distanz von s nach v, mit unendlich für unerreichbare Knoten. Nach einer Augmentation entlang eines kürzesten Pfades nehmen diese Distanzen niemals ab. Vorhandene Residualkanten erfüllen bereits $d_f(v) \leq d_f(u) + 1$ für jeden erreichbaren Startknoten u. Eine neu auftretende Rückwärtskante zeigt entlang des gewählten Pfades rückwärts, von einem Knoten auf Ebene $i + 1$ zu einem auf Ebene i, und erfüllt diese Ungleichung ebenfalls. Daher hat jeder neue Residualpfad zu einem Knoten mindestens dessen bisherige Distanz.

Jede Augmentation sättigt mindestens eine Residualkante (u, v) auf ihrem kürzesten Pfad, wobei $d_f(v) = d_f(u) + 1$ gilt. Bevor dieselbe Residualrichtung erneut gesättigt werden kann, muss eine spätere Augmentation (v, u) verwenden, um ihre Kapazität wiederherzustellen. Zu diesem späteren Zeitpunkt erfüllen die Ebenen der kürzesten Pfade $d(v) + 1 = d(u)$. Da die Distanzen nicht abgenommen haben, muss u dann mindestens zwei Ebenen weiter von s entfernt sein als bei der vorherigen Sättigung von (u, v).

Endliche BFS-Distanzen betragen höchstens $|V| - 1$. Jede Residualrichtung kann daher nur $O(|V|)$-mal gesättigt werden, und es gibt höchstens $2|E|$ Richtungen. Ordnen wir jede Augmentation einer gesättigten Kante zu, ergeben sich $O(|V||E|)$ Augmentationen. Jede BFS samt Aktualisierung benötigt $O(|V| + |E|)$ Zeit beziehungsweise $O(|E|)$ nach Entfernen isolierter Knoten, woraus die behauptete Schranke folgt. □

Selbst bei reellen Kapazitäten ist die Anzahl der Augmentationen unter der Annahme exakter Arithmetik so beschränkt. In Software kann Gleitkommarundung winzige scheinbare Residualkapazitäten erzeugen; dies ist ein numerisches Problem, das der mathematische Beweis nicht behandelt.

Hinweis zur Implementierung

Eine nützliche Darstellung mit Adjazenzlisten speichert für jede ursprüngliche Kante zwei verknüpfte Einträge: einen Vorwärtseintrag mit anfänglicher Residualkapazität $\omega(u, v)$ und einen Rückwärtseintrag mit anfänglicher Kapazität null. Jeder Eintrag speichert den Index seines Partners. Eine Augmentation subtrahiert Δ vom gewählten Eintrag und addiert Δ zu seinem Partner. BFS folgt nur positiven Residualkapazitäten und speichert Kanteneinträge als Vorgänger. Mit dieser Darstellung lassen sich auch entgegengesetzte ursprüngliche Kanten als getrennte Eintragspaare verwalten.

Innehalten und nachdenken – Programmieraufgabe

Implementieren Sie sowohl Ford-Fulkerson mit beliebiger Pfadwahl als auch Edmonds-Karp. Reproduzieren Sie die abwechselnden Entscheidungen am 100–1-Netzwerk und erhöhen Sie dann die vier äußeren Kapazitäten. Zählen Sie Augmentationen und vergleichen Sie die Laufzeiten. Prüfen Sie am Ende Kapazitätsbedingungen und Erhaltung, und berechnen Sie mit einer abschließenden Residualsuche einen Schnitt, dessen Kapazität dem zurückgegebenen Flusswert entspricht.

9.2 Bipartites Matching

Angenommen, wir möchten Studierende Projekten zuordnen. Jede Person kann an höchstens einem Projekt arbeiten, und jedes Projekt bietet Platz für eine Person. Manche Zuordnungen sind möglich, andere nicht: Vielleicht erfordert ein Projekt eine bestimmte Fähigkeit. Wie viele Studierende können wir zuordnen? Zunächst sieht das nicht wie eine Frage nach dem Transport durch ein Netzwerk aus. Sehen wir uns an, wie es zu einer solchen wird.

Definition 9.11. Ein ungerichteter Graph $G = (V, E)$ heißt *bipartit*, wenn V in zwei disjunkte Mengen A und B zerlegt werden kann, sodass jede Kante je einen Endpunkt in jeder Menge besitzt. Ein *Matching* ist eine Menge $M \subseteq E$, in der keine zwei Kanten einen Endpunkt gemeinsam haben. Ein Matching heißt *Maximum-Matching*, wenn seine Größe größtmöglich ist. Es ist *perfekt*, wenn jeder Knoten zu einer seiner Kanten gehört.

In unserem Beispiel enthält A die Studierenden, B die Projekte, und $\{a, b\} \in E$ bedeutet, dass die Zuordnung von Person a zu Projekt b möglich ist. Ein Matching beschreibt eine Menge von Zuordnungen, die nicht miteinander im Konflikt stehen.

Hintergrund

Verwechseln Sie *Maximum-Matching* nicht mit *maximalem Matching*. Ein maximales Matching lässt sich nicht durch einfaches Hinzufügen einer weiteren Kante erweitern. Ein Maximum-Matching besitzt die größtmögliche Anzahl von Kanten, auch wenn wir frühere Entscheidungen ersetzen müssen, um es zu erhalten.

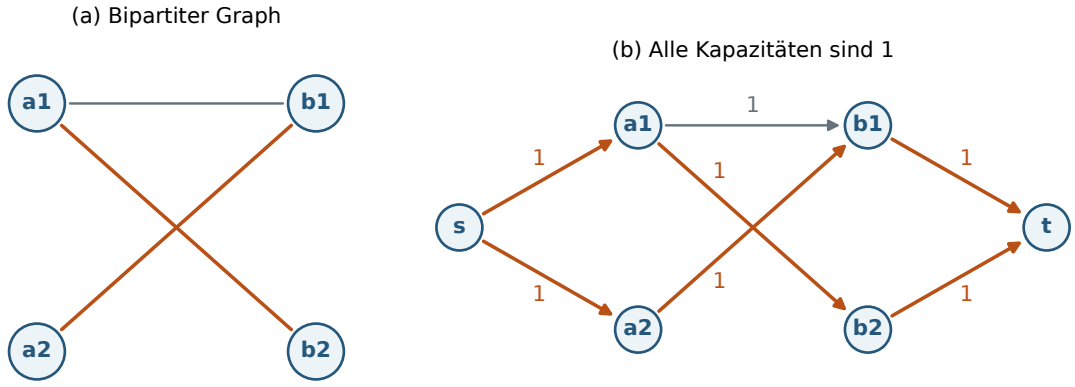
Betrachten wir $A = \{a_1, a_2\}$, $B = \{b_1, b_2\}$ und die drei Kanten $\{a_1, b_1\}$, $\{a_1, b_2\}$ und $\{a_2, b_1\}$. Wählen wir zuerst $\{a_1, b_1\}$, kann keine der verbleibenden Kanten hinzugefügt werden. Wir haben ein maximales Matching der Größe eins. Aber $\{\{a_1, b_2\}, \{a_2, b_1\}\}$ ist ein Matching der Größe zwei! Um die bessere Lösung zu erhalten, müssen wir die erste Zuordnung rückgängig machen. Kommt Ihnen das bekannt vor?

9.2.1 Vom Matching zum maximalen Fluss

Wir konstruieren ein Netzwerk, indem wir zwei neue Knoten s und t hinzufügen. Wir richten jede ursprüngliche Kante von A nach B, fügen eine Kante von s zu jedem Knoten in A und eine Kante von jedem Knoten in B nach t hinzu. *Jede* Kante erhält Kapazität eins. Formal besitzt der neue gerichtete Graph

$$V' = A \cup B \cup \{s, t\}, \qquad E' = \{(s, a) \mid a \in A\} \cup \{(a, b) \mid \{a, b\} \in E\} \cup \{(b, t) \mid b \in B\}.$$

Eine ganzzahlige Flusseinheit nimmt einen Pfad $s \to a \to b \to t$ und stellt die Zuordnung von a zu b dar. Warum können wir eine Person nicht zweimal zuordnen? Die Kante (s, a) hat Kapazität eins. Ebenso verhindert die Kante (b, t), dass ein Projekt zwei Studierende erhält.



Orange: Matching der Größe 2 (links), entsprechender Fluss mit Wert 2 (rechts)

Abbildung 33: Ein bipartiter Graph und sein Flussnetzwerk. Orange Kanten zeigen ein Matching der Größe zwei und den entsprechenden Fluss. Jede Netzwerkkante hat Kapazität eins; die Kreuzung zweier gezeichneter Kanten ist kein Knoten.

Satz 9.12. *Die Größe eines Maximum-Matchings in einem bipartiten Graphen ist gleich dem Wert eines maximalen Flusses im oben konstruierten Netzwerk.*

Beweis. Gegeben sei ein Matching der Größe k. Für jedes zugeordnete Paar schicken wir eine Einheit entlang von $s \to a \to b \to t$. Da keine Endpunkte gemeinsam sind, wird keine Kapazität überschritten, und die Erhaltung gilt an jedem Studierenden- und Projektknoten. Dies ist ein zulässiger Fluss mit Wert k.

Umgekehrt wählen wir einen ganzzahligen maximalen Fluss, der existiert, weil alle Kapazitäten ganzzahlig sind. Wir wählen genau diejenigen ursprünglichen Kanten, deren entsprechende gerichtete Kante eine Einheit trägt. Erhaltung und die Kanten mit Kapazität eins an Quelle und Senke stellen sicher, dass jeder Knoten höchstens einmal ausgewählt wird. Die gewählten Kanten bilden daher ein Matching, dessen Anzahl dem Flusswert entspricht. Die beiden Konstruktionen beweisen die Behauptung. □

In unserem kleinen Beispiel schickt die anfängliche Zuordnung von a_1 zu b_1 Fluss entlang von $s \to a_1 \to b_1 \to t$. Der Residualgraph enthält dann den Pfad

$$s \to a_2 \to b_1 \to a_1 \to b_2 \to t.$$

Die Rückwärts-Residualkante $b_1 \to a_1$ hebt die alte Zuordnung auf. Eine Augmentation entlang dieses Pfades ordnet a_2 dem Projekt b_1 und a_1 dem Projekt b_2 zu. Der Residualgraph drückt hier somit eine nützliche Operation aus: *jemanden neu zuordnen, um Platz für eine weitere Zuordnung zu schaffen.*

Innehalten und nachdenken – Selbst erarbeiten

Angenommen, ein Projekt kann drei Studierende aufnehmen, aber jede Person darf weiterhin nur einem Projekt beitreten. Welche Kapazitäten würden Sie ändern? Warum reicht es nicht, nur die Kanten zwischen Studierenden und Projekten zu ändern?

9.2.2 Problem der stabilen Heirat

Bisher war jede mögliche Zuordnung gleich gut. Was ändert sich, wenn Studierende manche Projekte anderen vorziehen und Projekte ebenfalls Präferenzen unter den Studierenden haben? Alle zuzuordnen führt nicht notwendigerweise zu einem zufriedenstellenden Ergebnis. Zwei Beteiligte könnten einander ihren zugewiesenen Partnern vorziehen und einen Grund haben, ihre Zuordnungen zu verlassen.

Das klassische *Problem der stabilen Heirat* betrachtet zwei disjunkte Mengen A und B mit $|A| = |B| = n$. Jede beteiligte Person gibt eine strikte Rangfolge *aller* Personen der anderen Seite an. Es gibt also keine Gleichstände, jeder mögliche Partner ist akzeptabel, und wir suchen ein perfektes Matching. Diese Annahmen sind für die folgenden Garantien wichtig.

Notation

Wir schreiben $b \succ_a b'$, wenn Person a die Person b gegenüber b' bevorzugt. Ein perfektes Matching beschreiben wir durch eine Bijektion $\mu : A \to B$. Somit ist $\mu(a)$ der Partner von a und $\mu^{-1}(b)$ der Partner von b.

Definition 9.13. Ein Paar $(a, b) \in A \times B$ ist ein *blockierendes Paar* für ein Matching μ, wenn

$$b \succ_a \mu(a) \quad \text{und} \quad a \succ_b \mu^{-1}(b).$$

Ein Matching heißt *stabil*, wenn es kein blockierendes Paar besitzt.

Beide Beteiligten müssen den Wechsel bevorzugen. Der Wunsch einer einzelnen Person nach einem besseren Partner reicht nicht aus, um das Matching instabil zu machen. Angenommen, Alice und Carol bevorzugen beide Bob gegenüber Dave, während Bob und Dave beide Alice gegenüber Carol bevorzugen. Alice mit Dave und Carol mit Bob zu verbinden ist instabil: Alice und Bob bilden ein blockierendes Paar. Alice mit Bob und Carol mit Dave zu verbinden ist stabil, obwohl Carol und Dave beide ihre zweite Wahl erhalten.

Innehalten und nachdenken – Kurzcheck

Warum löst ein Algorithmus für Maximum-Matching allein das Problem der stabilen Heirat nicht? Welche Information fehlt im Netzwerk mit Kapazität eins?

Der Gale–Shapley-Algorithmus. David Gale und Lloyd Shapley führten das Verfahren 1962 ein.[6] Die Idee besteht darin, vorläufige Zuordnungen zuzulassen. Eine Seite macht Anträge, die andere behält ihr bisher bestes Angebot. Ein vorläufiger Partner kann ersetzt werden, wenn ein besserer Antrag eintrifft. Eine abgewiesene Person setzt ihre Präferenzliste fort. Sei A die antragstellende Seite.

Algorithmus 23 Gale–Shapley mit antragstellender Seite A

Eingabe: Disjunkte Mengen A, B gleicher Größe; vollständige, strikte Präferenzlisten
Ausgabe: Ein stabiles perfektes Matching M

```
M ← ∅; markiere alle Beteiligten als frei
Setze für jedes a ∈ A einen Zeiger an den Anfang seiner Präferenzliste
solange ein a ∈ A frei ist tue
    b ← nächste Person auf der Liste von a; rücke den Zeiger von a weiter
    wenn b frei ist dann
        Füge (a, b) zu M hinzu; markiere a und b als gebunden
    sonst wenn b die Person a ihrem aktuellen Partner a' vorzieht dann
        Ersetze (a', b) in M durch (a, b)
        Markiere a' als frei und a als gebunden
    sonst
        b weist a zurück; a bleibt frei
    Ende wenn
Ende solange
gib zurück M
```

Beachten Sie die Asymmetrie. Antragstellende gehen ihre Listen abwärts durch, während Empfangende ihre vorläufigen Partner nur verbessern können. Ein Antrag wird nie wiederholt. Es gibt

[6] D. Gale und L. S. Shapley, *College Admissions and the Stability of Marriage*, American Mathematical Monthly 69(1), S. 9–15. https://doi.org/10.1080/00029890.1962.11989827

n^2 mögliche Anträge, daher terminiert der Algorithmus nach höchstens n^2 Iterationen. Eine freie antragstellende Person kann ihre Liste nicht ausgeschöpft haben: Dann hätten alle n Empfangenden einen Antrag erhalten, und jede würde weiterhin einen halten. Damit wären alle n Antragstellenden gebunden, im Widerspruch dazu, dass eine Person frei ist. Die Terminierung liefert daher ein perfektes Matching.

Hinweis zur Implementierung

Für eine Laufzeit von $\mathcal{O}(n^2)$ speichern wir einen Zeiger in jeder Liste der Antragstellenden, verwalten eine Warteschlange freier Antragstellender und bereiten für jede empfangende Person ein Rangarray vor. Eine empfangende Person kann dann zwei Antragstellende in konstanter Zeit vergleichen. Die Suche in einer Präferenzliste bei jedem Vergleich würde einen unnötigen zusätzlichen Faktor n verursachen.

Satz 9.14. *Gale–Shapley erzeugt ein stabiles Matching.*

Beweis. Angenommen, (a, b) wäre im endgültigen Matching ein blockierendes Paar. Da a die Person b ihrem endgültigen Partner vorzieht, muss a ihr früher einen Antrag gemacht haben. Person b hat a entweder sofort zurückgewiesen oder später ersetzt. In beiden Fällen hielt b danach einen Partner, den sie a vorzog. Da sich der Partner einer empfangenden Person nur verbessert, kann b die Person a nicht ihrem endgültigen Partner vorziehen. Das widerspricht der Annahme, dass (a, b) ein blockierendes Paar ist. □

Wer profitiert vom Antragstellen? Betrachten wir die Präferenzen in Tabelle 2. Jede Zeile listet die andere Seite von der am meisten bis zur am wenigsten bevorzugten Person auf.

	Erste	Zweite	Dritte		Erste	Zweite	Dritte
a_1	b_1	b_3	b_2	b_1	a_3	a_2	a_1
a_2	b_2	b_1	b_3	b_2	a_1	a_3	a_2
a_3	b_2	b_3	b_1	b_3	a_2	a_1	a_3

Tabelle 2: Vollständige, strikte Präferenzen für je drei Beteiligte auf jeder Seite.

Wählen wir in jedem Schritt die freie antragstellende Person mit dem kleinsten Index. Wenn A Anträge stellt, ergibt sich der folgende Ablauf. Gehaltene Paare sind bis zum Ende des Algorithmus vorläufig.

Schritt	Antrag	Entscheidung der empfangenden Person
1	$a_1 \to b_1$	Behalte a_1.
2	$a_2 \to b_2$	Behalte a_2.
3	$a_3 \to b_2$	Ersetze a_2 durch a_3; a_2 wird frei.
4	$a_2 \to b_1$	Ersetze a_1 durch a_2; a_1 wird frei.
5	$a_1 \to b_3$	Behalte a_1.

Das entstehende Matching lautet

$$M_A = \{(a_1, b_3), (a_2, b_1), (a_3, b_2)\}.$$

Tauschen wir nun die Rollen. Person b_1 macht a_3 einen Antrag, b_2 macht a_1 einen Antrag, und b_3 macht a_2 einen Antrag. Dies sind drei verschiedene Empfangende, daher werden alle Anträge gehalten und der Algorithmus endet sofort:

$$M_B = \{(a_1, b_2), (a_2, b_3), (a_3, b_1)\}.$$

Beide Matchings sind stabil. Die Beteiligten beurteilen sie jedoch nicht gleich. Zum Vergleich vergeben wir drei Punkte für die erste Wahl, zwei für die zweite und einen für die dritte.

Hier ist jede beteiligte Person besser gestellt, wenn ihre eigene Seite Anträge stellt. Der Unterschied ist eine Eigenschaft des Algorithmus und kein Zufall bei der Wahl der nächsten freien antragstellenden Person.

Antragseite	a_1	a_2	a_3	b_1	b_2	b_3	Summe A	Summe B	Gesamt
A	2	2	3	2	2	2	7	6	13
B	1	1	1	3	3	3	3	9	12

Tabelle 3: Präferenzpunkte der beiden Gale–Shapley-Ergebnisse.

Satz 9.15 (Optimalität für Antragstellende)**.** *Bei vollständigen, strikten Präferenzen gibt Gale–Shapley jeder antragstellenden Person ihren am meisten bevorzugten Partner unter allen stabilen Matchings. Jede empfangende Person erhält ihren am wenigsten bevorzugten Partner unter allen stabilen Matchings.*

Die Einschränkung *unter allen stabilen Matchings* ist wesentlich. Optimalität für Antragstellende verspricht nicht allen ihre erste Wahl. In M_A erhalten etwa a_1 und a_2 ihre zweite Wahl, aber keine von beiden kann in irgendeinem stabilen Matching ihre erste Wahl erhalten.

Beweis. Die entscheidende Beobachtung ist, dass ein zurückgewiesenes Paar in keinem stabilen Matching vorkommen kann. Nehmen wir das Gegenteil an und betrachten wir die erste Zurückweisung eines Paars (a, b), das zu einem stabilen Matching M gehört. Bei dieser Zurückweisung hält b eine antragstellende Person a', die sie a vorzieht. Bevor a' der Person b einen Antrag machte, wurde a' von allen Personen zurückgewiesen, die sie b vorzieht. Aufgrund unserer Wahl der ersten Zurückweisung kann keines dieser früher zurückgewiesenen Paare zu einem stabilen Matching gehören. Daher zieht a' die Person b ihrem Partner in M vor. Da b die Person a' ihrem Partner a in M vorzieht, blockiert (a', b) das Matching M – ein Widerspruch.

Eine antragstellende Person wurde von allen zurückgewiesen, die sie ihrem endgültigen Partner vorzieht. Keines dieser zurückgewiesenen Paare kann in einem stabilen Matching vorkommen; damit ist die Optimalität für Antragstellende bewiesen. Angenommen schließlich, eine empfangende Person b hätte in einem anderen stabilen Matching M einen schlechteren Partner. Ihr Gale–Shapley-Partner a zieht aufgrund der Optimalität für Antragstellende und der strikten Präferenzen ebenfalls b seinem Partner in M vor. Das Paar (a, b) würde M blockieren. Empfangende erhalten somit ihre schlechtesten stabilen Partner. □

Hintergrund

Stabilität bedeutet nicht Fairness. Sie schließt für beide Seiten attraktive Abweichungen aus, gleicht aber weder die Interessen der beiden Seiten aus noch maximiert sie die gesamten Präferenzpunkte. Die Punkte in Tabelle 3 sind eine zusätzliche Modellentscheidung: Eine Rangfolge allein sagt nicht, dass der Unterschied zwischen erster und zweiter Wahl genau einen Punkt wert ist. Die Tabelle macht die Asymmetrie unter einer einfachen, ausdrücklich gewählten Bewertungsregel sichtbar.

Innehalten und nachdenken – Selbst erarbeiten

In diesem Beispiel gibt es nur $3! = 6$ perfekte Matchings. Listen Sie diese auf und finden Sie für jedes Matching außer M_A und M_B ein blockierendes Paar. Welches stabile Matching maximiert die Gesamtpunktzahl? Würden Sie dieses Ergebnis notwendigerweise als fair bezeichnen?

Benötigt eine Anwendung ein anderes Ziel, müssen wir es ausdrücklich formulieren. Die Summe von Zuordnungsbewertungen zu maximieren ist ein gewichtetes Matching-Problem, das sich mit kostenminimalem Fluss modellieren lässt. *Zusätzlich* Stabilität zu verlangen benötigt weitere Bedingungen; das gewöhnliche Matching-Netzwerk erzwingt nicht das Fehlen blockierender Paare. Diese Unterscheidung zwischen einem effizienten spezialisierten Algorithmus und einem flexibleren Modell begleitet uns im nächsten Teil dieses Kapitels.

10 Kostenminimaler Fluss

Bisher haben wir gefragt, wie viel wir transportieren können. Angenommen, wir wissen bereits, wie viel ankommen muss, aber verschiedene Routen haben unterschiedliche Kosten. Eine schnelle Verbindung kann teuer sein, eine günstige wenig Kapazität haben, und mehrere Lieferungen können um dieselbe Verbindung konkurrieren. Welchen zulässigen Fluss sollten wir wählen?

Den Großteil unseres Formalismus können wir beibehalten. Das Netzwerk $N = (G, \omega, s, t)$ beschreibt weiterhin die Verbindungen und ihre Kapazitäten. Nun ergänzen wir eine Kostenfunktion $c : E \to \mathbb{R}$ und eine geforderte Menge $d \geq 0$. Die Zahl $c(u, v)$ bezeichnet die Kosten für den Transport *einer Einheit* entlang von (u, v). Sie ist nicht der Preis für das Öffnen dieser Verbindung und ersetzt auch nicht deren Kapazität.

Definition 10.1. Für einen Fluss f sind seine *Gesamtkosten*

$$C(f) = \sum_{(u,v)\in E} c(u, v)f(u, v).$$

Das *Problem des kostenminimalen Flusses* sucht einen zulässigen Fluss mit Wert d, der $C(f)$ minimiert, oder meldet, dass kein solcher Fluss existiert.

Der geforderte Wert ist wichtig. Sind alle Kosten positiv und fragen wir lediglich nach einem günstigsten zulässigen Fluss, ist nichts zu schicken eine ausgezeichnete Lösung! Beim *Problem des kostenminimalen maximalen Flusses* bestimmen wir zuerst den maximalen Wert d und minimieren anschließend die Kosten unter den Flüssen dieses Werts. Dies sind zwei Ziele in einer festgelegten Reihenfolge: Ein günstiger Fluss mit kleinerem Wert genügt nicht.

10.1 Angebote und Nachfragen

Häufig gibt es mehr als einen Lieferanten und mehr als einen Kunden. Statt jedem Knoten einen besonderen Namen zu geben, können wir ihm eine Zahl zuweisen. Eine positive Zahl bedeutet, dass der Knoten Waren liefert, eine negative, dass er Waren verbraucht, und null, dass er Waren nur weiterleitet.

Definition 10.2. Eine *Bilanzfunktion* $b : V \to \mathbb{R}$ gibt den erforderlichen Nettoausfluss an jedem Knoten an. Ein zulässiger Fluss erfüllt

$$0 \leq f(u, v) \leq \omega(u, v) \qquad \text{für alle } (u, v) \in E,$$

$$\sum_{w:(v,w)\in E} f(v, w) - \sum_{u:(u,v)\in E} f(u, v) = b(v) \qquad \text{für alle } v \in V.$$

Das Problem des kostenminimalen Flusses mit diesen Bilanzen minimiert $C(f)$ über alle solchen Flüsse.

Lesen Sie die zweite Gleichung als „was hinausgeht minus was ankommt“. Eine Fabrik mit $b(v) = 6$ sendet also sechs Einheiten mehr aus, als sie erhält. Ein Kunde mit $b(v) = -5$ erhält fünf Einheiten mehr, als er aussendet. Unser Problem mit einer einzigen Quelle ist der Spezialfall $b(s) = d$, $b(t) = -d$ und $b(v) = 0$ an allen anderen Knoten. Gilt $b(v) = 0$ an *jedem* Knoten, nennen wir den Fluss eine *Zirkulation*.

Innehalten und nachdenken – Kurzcheck

Summieren Sie die Bilanzgleichungen über alle Knoten. Warum muss $\sum_{v\in V} b(v) = 0$ gelten? Garantiert diese Bedingung allein, dass ein zulässiger Fluss existiert?

Jede Kante trägt einmal positiv und einmal negativ bei, daher muss das Gesamtangebot gleich der Gesamtnachfrage sein. Kapazitäten oder fehlende Verbindungen können eine Lieferung dennoch verhindern. Prüfen wir zuerst, dass die Gesamtbilanz null ist; andernfalls melden wir Unzulässigkeit. Um einen zulässigen Anfangsfluss zu finden, führen wir eine neue Quelle s_0 und Senke t_0 ein. Für

jedes $b(v) > 0$ fügen wir (s_0, v) mit Kapazität $b(v)$ hinzu. Für jedes $b(v) < 0$ fügen wir (v, t_0) mit Kapazität $-b(v)$ hinzu. Die ursprünglichen Kanten bleiben erhalten, ihre Kosten ignorieren wir in diesem Schritt. Wir berechnen einen maximalen Fluss von s_0 nach t_0. Ein zulässiger Fluss existiert genau dann, wenn sein Wert $\sum_{v:b(v)>0} b(v)$ ist. Dann sind alle Kanten aus s_0 und in t_0 gesättigt. Nach Entfernen der neuen Knoten bleibt ein Fluss mit genau den geforderten Bilanzen.

10.2 Residualkosten: Eine Entscheidung rückgängig zu machen hat einen Preis

Die Residualkapazitäten haben dieselbe Bedeutung wie zuvor: Wie viel können wir noch hinzufügen, und wie viel können wir zurücknehmen? Auch die Kosten müssen diese beiden Operationen beschreiben. Eine Einheit zu einer ursprünglichen Kante hinzuzufügen kostet $c(u, v)$. Eine Einheit zu entfernen spart genau diesen Betrag. Eine Rückwärts-Residualkante hat daher die *negativen* ursprünglichen Kosten.

Für jede ursprüngliche Kante (u, v) enthält der Residualgraph

Residualkante	Bedingung	Kapazität	Kosten
(u, v)	$f(u, v) < \omega(u, v)$	$\omega(u, v) - f(u, v)$	$c(u, v)$
(v, u)	$f(u, v) > 0$	$f(u, v)$	$-c(u, v)$.

Wir schreiben c_f für diese Residualkosten und r_f für die Residualkapazitäten. Wie zuvor macht die Annahme eines orientierten ursprünglichen Graphen jede Residualkante eindeutig.

Innehalten und nachdenken – Kurzcheck

Eine Kante hat Kapazität 8, Kosten 5 und trägt aktuell 3 Einheiten. Welche Residualkanten existieren, mit welchen Kapazitäten und Kosten? Was ändert sich, wenn wir zwei Einheiten zurücknehmen?

Ein augmentierender s-t-Pfad ändert die gelieferte Menge. Hier ist diese Menge bereits festgelegt. Stattdessen können wir den Fluss entlang eines *gerichteten Kreises* im Residualgraphen ändern. An jedem Knoten des Kreises tritt eine Residualkante ein und eine aus. Die Änderungen heben sich in der Bilanzgleichung auf, auch wenn eine Residualkante das Zurücknehmen einer ursprünglichen Kante darstellt.

Ist Q ein solcher Kreis, sei

$$c_f(Q) = \sum_{(u,v)\in Q} c_f(u, v), \qquad \Delta = \min_{(u,v)\in Q} r_f(u, v).$$

Eine Augmentation um Δ entlang von Q ändert die Kosten um $\Delta c_f(Q)$. Ein Kreis mit negativen Gesamtkosten verbessert somit den Fluss, während jedes Angebot und jede Nachfrage erhalten bleibt.

Betrachten wir Abbildung 34. Beide Routen haben Kapazität 3, und wir müssen $d = 3$ liefern. Anfangs schicken wir alles entlang von $s \to A \to t$ zu Gesamtkosten von $3 \cdot 4 = 12$. Der Residualkreis

$$s \to B \to t \to A \to s$$

hat Kosten $1 + 0 + 0 - 4 = -3$ und einen residualen Engpass von 3. Wir fügen drei Einheiten auf der günstigen Route hinzu und nehmen drei auf der teuren zurück. Die gelieferte Menge bleibt 3, aber ihre Kosten sinken um $3 \cdot 3 = 9$, von 12 auf 3.

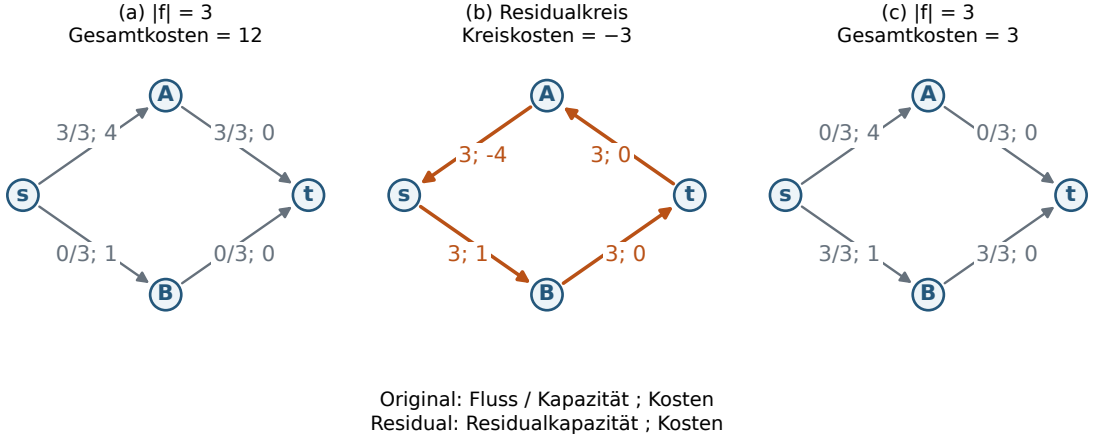


Abbildung 34: Verbesserung eines Flusses mit festem Wert durch einen negativen Residualkreis. Die Rückwärts-Residualkanten nehmen die teure Lieferung zurück.

Eine negative Residualkante allein reicht nicht zur Verbesserung des Flusses. Wir müssen Änderungen so kombinieren, dass die Bilanzgleichungen weiterhin gelten. Ebenso stellt der negative Kreis keine Waren dar, die physisch rückwärts durch das Netzwerk reisen: Er beschreibt eine Überarbeitung unseres Lieferplans.

10.3 Kleins Kreisaufhebungsalgorithmus

Nun haben wir fast denselben Rahmen wie bei Ford-Fulkerson. Wir beginnen mit einem zulässigen Fluss, finden eine Verbesserung im Residualgraphen und augmentieren um ihren Engpass. Beim maximalen Fluss war die Verbesserung ein Pfad, der den Flusswert erhöhte. Bei minimalen Kosten ist es ein Kreis, der die Kosten verringert.

Algorithmus 24 Kleins Kreisaufhebungsalgorithmus

Eingabe: Orientierter Graph G, endliche ganzzahlige Kapazitäten ω, ganzzahlige Kosten c und Bilanzen b
Ausgabe: Ein kostenminimaler zulässiger Fluss oder eine Meldung der Unzulässigkeit

Prüfe $\sum_v b(v) = 0$; falls falsch, gib unzulässig zurück
Finde einen ganzzahligen zulässigen Fluss f mittels der Hilfskonstruktion für maximalen Fluss
wenn kein zulässiger Fluss existiert **dann**
    gib zurück unzulässig
Ende wenn
Konstruiere Residualgraph, Kapazitäten r_f und Kosten c_f
solange ein gerichteter Kreis Q mit $c_f(Q) < 0$ im Residualgraphen existiert **tue**
    $\Delta \leftarrow \min_{(u,v) \in Q} r_f(u, v)$
    Erhöhe f um Δ auf ursprünglichen Kanten, die Q vorwärts verwendet
    Verringere f um Δ auf ursprünglichen Kanten, die Q rückwärts verwendet
    Aktualisiere Residualgraph, Kapazitäten und Kosten
Ende solange
gib zurück f

Diese Methode wird Morton Kleins Arbeit von 1967 zugeordnet: *A Primal Method for Minimal Cost Flows with Applications to the Assignment and Transportation Problems*, Management Science 14(3), 205–220.[7]

Satz 10.3 (Optimalitätskriterium über negative Kreise)**.** *Ein zulässiger Fluss hat für seine vorgegebenen Bilanzen genau dann minimale Kosten, wenn sein Residualgraph keinen gerichteten Kreis mit negativen Gesamtkosten besitzt.*

Beweis. Existiert ein negativer Residualkreis, ergibt eine Augmentation entlang dieses Kreises einen zulässigen Fluss mit kleineren Kosten. Der aktuelle Fluss kann daher nicht optimal sein.

Für die Umkehrung nehmen wir an, dass ein günstigerer zulässiger Fluss g existiert. Wir vergleichen g und f auf jeder ursprünglichen Kante. Wo g größer ist, legen wir die Differenz auf die Vorwärts-Residualkante; wo es kleiner ist, auf die Rückwärts-Residualkante. Der entstehende Residualfluss hält die Residualkapazitäten ein. Da f und g dieselben Bilanzen haben, hat ihre Differenz an jedem Knoten Bilanz null: Sie ist eine Zirkulation.

Eine nichtnegative Zirkulation, die nicht null ist, lässt sich in gerichtete Kreise zerlegen. Folgen wir dazu Kanten mit positivem Fluss. Die Erhaltung verhindert, dass die Wanderung stecken bleibt, und die Endlichkeit erzwingt einen wiederholten Knoten. Dies ergibt einen Kreis. Wir subtrahieren seinen kleinsten Kantenfluss, wodurch mindestens eine positive Kante entfällt, und wiederholen den Vorgang. Die Kostendifferenz $C(g) - C(f)$ ist somit eine Summe von Kreiskosten, multipliziert mit positiven Mengen. Da diese Differenz negativ ist, hat mindestens einer der Kreise negative Kosten. Dies widerspricht dem angenommenen Fehlen negativer Residualkreise. □

Wie finden wir diese Kreise? Das wichtigste Werkzeug haben wir bereits: Bellman-Moore-Ford. Residualkosten können negativ sein, daher ist gewöhnlicher Dijkstra hier ungeeignet. Wir müssen einen Kreis *irgendwo* im Residualgraphen finden, auch in Komponenten, die von der ursprünglichen Quelle aus unerreichbar sind. Wir initialisieren jede Distanz mit null, als hätte eine zusätzliche Quelle eine Kante mit Kosten null zu jedem Knoten. Wir relaxieren alle Residualkanten in $|V|$ Runden. Ändert sich in der letzten Runde eine Distanz, existiert ein negativer Kreis. Wir speichern Vorgänger; von einem in dieser Runde aktualisierten Knoten aus folgen wir den Vorgängern $|V|$-mal, um in einen Kreis zu gelangen, und gehen dann weiter, bis sich ein Knoten wiederholt. Die Vorgängerwanderung läuft rückwärts, daher kehren wir sie um, um den gerichteten Kreis zu erhalten. Dies kostet $O(|V||E|)$ pro Suche, da es höchstens $2|E|$ Residualkanten gibt.

Hintergrund

Bei ganzzahligen Kapazitäten, Bilanzen und Kosten wählen wir einen ganzzahligen zulässigen Anfangsfluss. Jede Augmentation bleibt ganzzahlig und verringert die Gesamtkosten um mindestens eins. Mit $B = \sum_{e \in E} \omega(e)|c(e)|$ liegen alle zulässigen Kosten zwischen $-B$ und B, daher gibt es höchstens $2B$ Kreisaufhebungen. Dies ist eine *pseudopolynomielle* Schranke: B hängt von Zahlenwerten ab, nicht nur von der Anzahl der zu ihrer Codierung benötigten Bits. Rationale Daten lassen sich zu ganzen Zahlen skalieren, doch dabei können große Zahlen entstehen. Dieses Argument liefert keine Terminierungsgarantie für beliebige reelle Daten. Anspruchsvollere Algorithmen für kostenminimalen Fluss bieten stärkere Schranken; die grundlegende Kreisaufhebungsmethode ist hier nützlich, weil ihr Verbesserungsschritt leicht verständlich ist.

Innehalten und nachdenken – Selbst erarbeiten

Verringern Sie im Zweiroutenbeispiel die Kapazität von (s, B) auf 2, lassen Sie die anderen Kapazitäten bei 3 und die Nachfrage bei 3. Beginnen Sie erneut mit der teuren Lieferung. Finden Sie den negativen Residualkreis, führen Sie eine Kreisaufhebung durch und erklären Sie, warum die neuen Kosten 6 betragen. Warum können wir nicht mehr alle drei Einheiten über die günstigere Route schicken?

[7] `https://doi.org/10.1287/mnsc.14.3.205`

10.4 Ein allgemeineres Modell

Kostenminimaler Fluss kann Entscheidungen ausdrücken, die maximaler Fluss allein nicht unterscheiden kann. Nachdem wir etwa ein größtmögliches bipartites Matching gefunden haben, möchten wir vielleicht ein Matching dieser Größe mit kleinsten gesamten Zuordnungskosten. Wir fügen jeder Person-Aufgabe-Kante Kosten hinzu und geben die zuvor berechnete Matching-Größe als Flusswert vor. Dadurch wird die Summe dieser Kosten optimiert. Codieren die Kosten Präferenzränge, muss das entstehende Matching nicht stabil sein: Blockierende Paare sind eine andere Anforderung.

Auch kürzeste Pfade passen in dieses Modell. Für einen Graphen mit nichtnegativen Kantenlängen und einem s-t-Pfad geben wir jeder Kante Kapazität eins, verwenden ihre Länge als Stückkosten und verlangen eine Einheit von s nach t. Ein ganzzahliges Optimum enthält einen s-t-Pfad; zusätzliche Kreise können ohne Kostenerhöhung entfernt werden. Der verbleibende Pfad ist ein kürzester. Mehrere der behandelten Probleme lassen sich somit mit demselben Flussmodell ausdrücken.

Wir können auch ein Problem des maximalen Flusses mit einem Algorithmus für kostenminimalen Fluss lösen, ohne den maximalen Wert im Voraus zu kennen. Wir geben jeder ursprünglichen Kante Kosten null. Wir fügen eine Rückkehrkante (t, s) mit Kapazität

$$U = \sum_{v:(s,v)\in E} \omega(s, v)$$

und Kosten -1 hinzu. Wir setzen jede Bilanz auf null und berechnen eine kostenminimale Zirkulation. Der Nullfluss ist bereits zulässig. Jede Einheit auf der Rückkehrkante muss durch eine Nettoeinheit von s nach t durch das ursprüngliche Netzwerk ausgeglichen werden. Die Gesamtkosten sind genau der negative Fluss auf der Rückkehrkante. Ihre Minimierung maximiert daher den ursprünglichen Flusswert. Nach Entfernen der Rückkehrkante erhalten wir den gewünschten maximalen Fluss.

Würde eine direkte Kante (s, t) nach dem Hinzufügen von (t, s) unsere Konvention für orientierte Graphen verletzen, verwenden wir stattdessen einen neuen Knoten r und den Rückkehrpfad $t \to r \to s$. Beide Kanten erhalten Kapazität U, eine Kosten -1 und die andere Kosten null. Die Erhaltung bei r erzwingt dieselbe Menge auf beiden. Dieselbe Unterteilung funktioniert auch bei einer allgemeineren Eingabe, die bereits eine Rückkehrkante enthält.

Innehalten und nachdenken – Kurzcheck

Warum würde die Zuweisung von Kosten null zur Rückkehrkante das Ziel des maximalen Flusses nicht codieren? Was dürfte ein Algorithmus für kostenminimalen Fluss zurückgeben?

Dies ist eine *Reduktion*: Wir übersetzen eine Instanz und anschließend ihre Lösung zurück. Sie erklärt präzise, warum kostenminimaler Fluss ein allgemeineres Modell ist. Ein spezialisierter Algorithmus für maximalen Fluss benötigt oft weniger Hilfsmittel und kann für diesen Spezialfall schneller sein. „Allgemeiner“ ist jedoch für sich genommen kein Satz über die Laufzeit. Die gewählten Algorithmen, Darstellungen und Instanzen bleiben entscheidend.

10.5 Ein Einblick in die lineare Programmierung

Betrachten wir erneut die Gleichungen des kostenminimalen Flusses. Für jede Kante gibt es eine Unbekannte $f(u, v)$. Der Graph sagt uns, welche Unbekannten in welcher Bilanzgleichung auftreten, doch die Gleichungen selbst verwenden nur Summen und Multiplikation mit Konstanten:

$$\begin{aligned} \text{minimiere} \quad & \sum_{(u,v)\in E} c(u, v) f(u, v) \\ \text{unter den Bedingungen} \quad & \sum_{w:(v,w)\in E} f(v, w) - \sum_{u:(u,v)\in E} f(u, v) = b(v) && \text{für alle } v \in V, \\ & 0 \le f(u, v) \le \omega(u, v) && \text{für alle } (u, v) \in E. \end{aligned}$$

Die Variablen beschreiben unsere Entscheidungen. Die Nebenbedingungen beschreiben, welche Entscheidungen erlaubt sind. Die Zielfunktion vergleicht die erlaubten Entscheidungen.

Definition 10.4. Ein *lineares Programm* (LP) minimiert oder maximiert einen linearen Ausdruck in reellwertigen Entscheidungsvariablen unter linearen Gleichungen und Ungleichungen.

„Linear“ bedeutet hier, dass jede Variable nur mit einer Konstanten multipliziert wird und diese Terme addiert werden. Ein Term wie $4f(s, A)$ ist linear; ein Produkt zweier Entscheidungsvariablen oder eine quadrierte Entscheidungsvariable ist es nicht. Lineare Programmierung erlaubt viele Nebenbedingungen über Graphbilanzen hinaus. Beispielsweise muss eine gemeinsame Ressourcenbeschränkung $2x_1 + 5x_2 \leq 40$ überhaupt keiner Netzwerkkante entsprechen. Kostenminimaler Fluss ist somit ein Spezialfall der linearen Programmierung, die auch Probleme außerhalb der Graphentheorie modelliert.

Wir belassen es bei diesem Modellierungszusammenhang; das Lösen allgemeiner linearer Programme wäre ein eigenes Thema. Es gibt Polynomialzeitalgorithmen für rationale LPs. Allgemeine Solver bieten größere Modellierungsfreiheit, während spezialisierte Flussalgorithmen die Netzwerkstruktur ausnutzen können. Die Forderung ganzzahliger Variablen führt zur *ganzzahligen linearen Programmierung*, die im Allgemeinen NP-schwer ist. Dies ist eine bedeutsame Änderung: Bruchteilige Produktionsmengen können akzeptabel sein, die Zuweisung einer halben Person zu jeweils einer von zwei unteilbaren Aufgaben möglicherweise nicht.

Hintergrund

Netzwerkfluss besitzt eine besondere Ganzzahligkeitseigenschaft. Bei ganzzahligen Kapazitäten und Bilanzen hat ein zulässiges Problem des kostenminimalen Flusses ein ganzzahliges Optimum, obwohl seine Variablen reell sein dürfen. Wir benötigen also keine Ganzzahligkeitsbedingungen, um einen ganzzahligen optimalen Fluss zu erhalten. Diese Eigenschaft besagt nicht, dass jeder zulässige oder optimale Fluss ganzzahlig sein muss, und sie muss bei zusätzlichen beliebigen linearen Nebenbedingungen nicht erhalten bleiben.

10.5.1 Dasselbe Beispiel im Excel-Solver

Nun können wir unser kleines Beispiel einem allgemeinen Solver übergeben. Die begleitende Arbeitsmappe `5_network_flows_solver.xlsx` enthält ein Blatt `MinCostFlow` mit diesem Modell und ein Blatt `MaxFlow` für das 100–1-Beispiel. Beide enthalten eine geprüfte optimale Lösung und Anweisungen zur manuellen Einrichtung des Solvers. Um das Modell für minimale Kosten selbst aufzubauen, erstellen Sie die folgende Tabelle in den Zellen `A1:E5`. Die Einträge in Spalte E sind die Entscheidungsvariablen; tragen Sie dort zunächst null ein.

Zeile	A: Von	B: Nach	C: Kapazität	D: Stückkosten	E: Fluss
2	s	A	3	4	0
3	A	t	3	0	0
4	s	B	3	1	0
5	B	t	3	0	0

Tragen Sie für die Gesamtkosten `=SUMPRODUCT(D2:D5,E2:E5)` in `E7` ein. Tragen Sie anschließend diese Bilanztabelle in die Spalten G–I ein. Spalte H berechnet den Nettoausfluss, Spalte I gibt die geforderte Bilanz an.

Zeile	G: Knoten	H: Nettoausfluss	I: Gefordert
2	s	`=E2+E4`	3
3	A	`=E3-E2`	0
4	B	`=E5-E4`	0
5	t	`=-E3-E5`	−3

Minimieren Sie im Solver `E7` durch Ändern von `E2:E5`. Fügen Sie die Nebenbedingungen `E2:E5 >= 0`, `E2:E5 <= C2:C5` und `H2:H5 = I2:I5` hinzu. Wählen Sie die Lösungsmethode *Simplex LP*. Ist der Solver nicht verfügbar, aktivieren Sie zuerst das Solver-Add-in. Die obige Formel verwendet englische Funktionsnamen; eine lokalisierte Installation kann einen übersetzten Namen wie `SUMMENPRODUKT` und Semikolons statt Kommas verwenden.

Das erwartete Ergebnis in `E2:E5` ist $(0, 0, 3, 3)$ mit Gesamtkosten 3. Dieses Modell enthält keine Ganzzahligkeitsbedingung. Prüfen Sie neben der Zielfunktion auch die Bilanzzellen: Eine günstige Lieferung, die die geforderte Menge nicht liefert, ist keine Lösung. Die anfänglichen Nulleinträge verletzen absichtlich die Nachfragebedingungen; der Solver findet aus dem Modell ein zulässiges Optimum.

Innehalten und nachdenken – Selbst erarbeiten

Ändern Sie Zelle `C4` auf 2 und lösen Sie erneut. Vergleichen Sie das Ergebnis mit Ihrer Berechnung über negative Kreise. Erhöhen Sie dann die geforderte Lieferung auf 7, indem Sie die Quell- und Senkenbilanzen auf 7 und -7 setzen. Erklären Sie die Unzulässigkeitsmeldung des Solvers allein anhand der Kapazitäten der aus s ausgehenden Kanten.

11 Graphfärbung

Wie viele Farben benötigen Sie, um eine Landkarte zu färben? Natürlich könnten Sie jeder Region eine eigene Farbe geben. Das erscheint aber verschwenderisch. Weit voneinander entfernte Regionen können dieselbe Farbe haben, ohne Verwirrung zu stiften. Nur Regionen mit einer gemeinsamen Grenze sollen unterschiedliche Farben erhalten. Wie viele Farben sind also wirklich nötig?

1852 fragte Francis Guthrie, ob vier Farben für eine Landkarte immer ausreichen würden. Die Frage ist leicht zu erklären, und Sie können es selbst mit einem Stift und einigen Farben ausprobieren. Ein Beweis muss jedoch *jede* mögliche Landkarte berücksichtigen. Das Problem blieb mehr als ein Jahrhundert offen. 1976 bewiesen Kenneth Appel und Wolfgang Haken den Vierfarbensatz mithilfe umfangreicher Computerberechnungen.[8] Wir kommen auf den Satz zurück, nachdem wir das Problem als Graphen ausgedrückt haben.

Die Idee hat Anwendungen weit über Landkarten hinaus. Stellen Sie sich vor, dass an zwei Prüfungen dieselben Studierenden teilnehmen. Sie müssen zu unterschiedlichen Zeiten stattfinden. Zwei Berechnungen benötigen exklusiven Zugriff auf dieselbe Ressource? Sie können nicht gleichzeitig laufen. In beiden Fällen wollen wir Objekten etwas zuweisen und dabei bestimmte Paare auseinanderhalten. Diese Zuweisung nennen wir eine Farbe; sie muss keine Farbe auf einem Bildschirm sein.

11.1 Knotenfärbung und chromatische Zahl

Nehmen wir einen einfachen ungerichteten Graphen $G = (V, E)$. Wir geben jedem Knoten eine Farbe und beachten dabei eine Regel: Benachbarte Knoten müssen unterschiedliche Farben erhalten. Nicht benachbarte Knoten dürfen dieselbe Farbe erhalten, müssen es aber nicht. Diesen letzten Punkt sollten wir uns merken. Eine Kante erzwingt einen Unterschied; eine fehlende Kante erzwingt keine Gleichheit.

Definition 11.1. Eine *zulässige Knotenfärbung* (engl. proper vertex coloring) eines Graphen $G = (V, E)$ mit einer Palette von k Farben ist eine Funktion

$$f : V \to \{1, 2, \ldots, k\} \qquad \text{sodass} \qquad \forall \{u, v\} \in E : \ f(u) \neq f(v).$$

Ein Graph heißt *k-färbbar*, wenn eine solche Funktion existiert. Die Palette enthält k verfügbare Farben; die Färbung darf weniger als k davon verwenden.

Warum verwenden wir Zahlen statt Rot, Grün und Blau? Weil die tatsächlichen Namen der Farben keine Rolle spielen. Wenn wir jedes Vorkommen von Farbe 1 durch Blau und jedes Vorkommen von Farbe 2 durch Orange ersetzen, bleibt die Färbung zulässig. Entscheidend ist, welche Knoten dieselbe Farbe haben. Im Folgenden bedeutet „Färbung“ eine zulässige Knotenfärbung, sofern nicht anders angegeben.

Irgendeine Färbung zu finden ist einfach: Geben wir jedem Knoten seine eigene Farbe. Interessant ist die Frage, wie viele Farben wir einsparen können.

Definition 11.2. Die *chromatische Zahl* (engl. chromatic number) $\chi(G)$ ist die kleinste Anzahl von Farben, die für eine zulässige Knotenfärbung von G benötigt wird. Eine Färbung mit genau $\chi(G)$ Farben heißt *optimale Färbung*. Für den Graphen ohne Knoten setzen wir $\chi(G) = 0$.

Ein nichtleerer Graph ohne Kanten benötigt nur eine Farbe. Der Graph K_2 benötigt zwei. In einem Dreieck ist der dritte Knoten zu beiden anderen benachbart und benötigt daher eine dritte Farbe. Allgemein gilt $\chi(K_n) = n$, da jedes Knotenpaar unterschiedliche Farben haben muss.

Innehalten und nachdenken – Kurzcheck

Angenommen, jemand zeigt Ihnen eine zulässige Färbung mit fünf Farben. Ist damit $\chi(G) = 5$ gezeigt? Welche zusätzliche Information würde Sie davon überzeugen, dass weniger Farben unmöglich sind?

[8] Zur Geschichte und einer Diskussion des Satzes siehe Robin Thomas, *The Four Color Theorem*, `https://thomas.math.gatech.edu/FC/fourcolor.html`.

Eine Färbung mit fünf Farben zeigt $\chi(G) \leq 5$. Um Gleichheit nachzuweisen, benötigen wir außerdem einen Grund, warum mindestens fünf Farben notwendig sind. Zum Beispiel könnte der Graph fünf Knoten enthalten, die alle paarweise benachbart sind.

Definition 11.3. Eine *Clique* ist eine Menge paarweise benachbarter Knoten. Die *Cliquenzahl* $\omega(G)$ ist die größte Anzahl von Knoten in einer Clique von G.

Alle Knoten einer Clique benötigen unterschiedliche Farben, selbst wenn die Clique nur ein kleiner Teil eines größeren Graphen ist. Daher gilt für jeden nichtleeren einfachen Graphen

$$\omega(G) \leq \chi(G) \leq |V|.$$

Die beiden Ungleichungen erfüllen unterschiedliche Zwecke: Eine Clique liefert eine *untere Schranke*, eine Färbung eine *obere Schranke*. Treffen beide Schranken zusammen, kennen wir das Optimum. Sie müssen jedoch nicht zusammentreffen. Der Kreis C_5 enthält kein Dreieck, seine größte Clique hat also nur zwei Knoten. Dennoch benötigt er drei Farben.

11.1.1 Wann zwei Farben ausreichen

Warum benötigt C_5 drei Farben? Beginnen wir an einem Knoten und wechseln entlang des Kreises zwischen den Farben 1 und 2. Die fünf Knoten erhalten $1, 2, 1, 2, 1$. Doch der letzte Knoten ist zum ersten benachbart, und beide haben Farbe 1. Wir können den Kreis nicht schließen. Geben wir dem letzten Knoten Farbe 3, ist der Konflikt gelöst. Bei C_4 funktioniert die abwechselnde Folge $1, 2, 1, 2$ bereits. Entscheidend ist, ob der Kreis ungerade oder gerade Länge hat.

Erinnern wir uns: Ein bipartiter Graph besitzt zwei Teile, innerhalb derer keine Kanten liegen. Färben wir einen Teil mit Farbe 1 und den anderen mit Farbe 2. Umgekehrt ergibt eine zulässige Färbung mit höchstens zwei Farben genau eine solche Zerlegung. Zusammen mit der Charakterisierung aus Abschnitt 1.4 erhalten wir

$$G \text{ ist bipartit} \iff \chi(G) \leq 2 \iff G \text{ hat keinen ungeraden Kreis.}$$

Besitzt der Graph mindestens eine Kante, benötigt ein bipartiter Graph genau zwei Farben. Ohne diese Einschränkung würde die Aussage „bipartit bedeutet $\chi(G) = 2$“ die kantenlosen Graphen übersehen.

Wir können diese Beobachtung in einen Algorithmus umsetzen. Starten wir eine BFS oder DFS an einem beliebigen ungefärbten Knoten, geben ihm Farbe 1 und jedem neu entdeckten Nachbarn die jeweils andere Farbe. Führt eine Kante zu einem bereits gefärbten Knoten, prüfen wir, ob ihre Endpunkte unterschiedliche Farben haben. Ein Konflikt bedeutet, dass der Graph nicht bipartit ist. Ist eine Komponente abgeschlossen, beginnen wir erneut in einer anderen ungefärbten Komponente. Mit Adjazenzlisten benötigt der gesamte Test $O(|V| + |E|)$ Zeit.

Innehalten und nachdenken – Kurzcheck

Ein Stern hat einen zentralen Knoten und acht Blätter. Sein Maximalgrad ist 8. Wie viele Farben benötigt er? Warum erfordern viele Nachbarn nicht unbedingt viele Farben?

11.1.2 Zurück zu den Landkarten

Um eine Landkarte in einen Graphen zu überführen, stellen wir jede Region durch einen Knoten dar und verbinden zwei Knoten, wenn ihre Regionen ein Grenzstück gemeinsam haben. Eine bloße Berührung in einem Punkt zählt nicht. Bei einer gewöhnlichen Landkarte mit zusammenhängenden Regionen in der Ebene entsteht so ein planarer Adjazenzgraph. Ein Graph heißt *planar*, wenn er sich in der Ebene zeichnen lässt, ohne dass sich Kanten außer an gemeinsamen Endpunkten kreuzen. Eine konkrete Zeichnung kann Kreuzungen haben, obwohl der Graph planar ist; vielleicht lassen sich die Kanten anders zeichnen.

Satz 11.4 (Vierfarbensatz)**.** *Jeder einfache planare Graph G erfüllt $\chi(G) \leq 4$.*

Der Satz sagt „höchstens vier“, nicht, dass jede Landkarte vier Farben benötigt. Er gilt auch nicht für beliebige Graphen: K_5 benötigt fünf Farben und ist nicht planar. Auch die Annahmen über Landkarten sind wichtig. Besteht ein politisches Land aus nicht zusammenhängenden Regionen, die alle dieselbe Farbe erhalten müssen, führt die Darstellung des gesamten Landes als ein Knoten nicht notwendigerweise zu einem planaren Graphen.

11.2 Planung eines Sportwettbewerbs

Angenommen, $n \geq 2$ Mannschaften nehmen an einem Sportwettbewerb teil. Jede Mannschaft muss gegen jede andere genau einmal spielen, und keine Mannschaft darf am selben Tag mehr als einmal spielen. Der Wettbewerb soll in möglichst wenigen Tagen abgeschlossen sein. Wir nehmen an, dass genügend Spielfelder verfügbar sind, sodass Spiele mit verschiedenen Mannschaften am selben Tag stattfinden können.

Innehalten und nachdenken – Selbst erarbeiten

Formulieren Sie dies als Graphfärbungsproblem. Was sollte ein Knoten darstellen? Wann sollten zwei Knoten benachbart sein? Was bedeutet eine Farbe? Eine Beschreibung in Worten genügt, aber stellen Sie sicher, dass sich eine Färbung wieder in einen tatsächlichen Spielplan übersetzen lässt.

Vielleicht war Ihre erste Idee, für jede Mannschaft einen Knoten anzulegen. Das erscheint natürlich: Die Mannschaften sind die im Problem genannten Objekte, und eine Kante kann ein Spiel zwischen ihnen darstellen. Nennen wir dies *Modell A*. Da jedes Mannschaftspaar gegeneinander spielen muss, ist der entstehende Graph K_n.

Färben wir ihn nun. Wir wissen bereits, dass K_n genau n Farben benötigt. Bedeutet das, dass wir n Tage brauchen? Was bedeutet es überhaupt, einer *Mannschaft* einen Tag zuzuweisen? Angenommen, Mannschaft A erhält Tag 1 und Mannschaft B Tag 2. An welchem Tag spielen sie gegeneinander? Ihrem Spiel haben wir gar keinen Tag zugewiesen. Wenn die Mannschaftsfarben die Tage bezeichnen, an denen Mannschaften verfügbar sind, erhalten die beiden sogar unterschiedliche Tage!

Die Schwierigkeit liegt nicht darin, K_n zu färben. Das können wir problemlos. Die Schwierigkeit ist, dass diese Färbung die Frage nach dem Spielplan nicht beantwortet. Der Graph hält fest, wer gegen wen spielt, aber Knotenfarben weisen den Mannschaften Werte zu, während wir den Spielen Tage zuweisen müssen. Eine von einem Algorithmus gelieferte Zahl ist noch keine Lösung des ursprünglichen Problems.

Versuchen wir *Modell B*. Diesmal legen wir für jedes *Spiel* einen Knoten an. Zwei Spielknoten sind genau dann benachbart, wenn die Spiele eine gemeinsame Mannschaft haben. Beispielsweise steht das Spiel AB im Konflikt mit AC, da Mannschaft A nicht beide Spiele an einem Tag bestreiten kann. Dagegen steht AB nicht im Konflikt mit CD. Diese Spiele können am selben Tag stattfinden. Nun hat eine Farbe eine klare Bedeutung: Sie bezeichnet den Tag, an dem ein Spiel stattfindet. Benachbarte Knoten müssen unterschiedliche Farben erhalten, also finden Spiele mit einer gemeinsamen Mannschaft an unterschiedlichen Tagen statt. Knoten derselben Farbe bilden die Spiele eines Tages. Abbildung 35 zeigt die beiden Modelle für vier Mannschaften, und die Färbung von Modell B liefert folgenden Spielplan:

Tag (Farbe)	Erstes Spiel	Zweites Spiel
1	AB	CD
2	AC	BD
3	AD	BC

Alle sechs Spiele kommen genau einmal vor, und jede Mannschaft spielt einmal pro Tag. Geht es besser als in drei Tagen? Nein: Mannschaft A hat drei Spiele und darf nur einmal pro Tag spielen. Wir haben sowohl einen Spielplan für drei Tage als auch einen Grund, warum weniger Tage unmöglich sind. In der Sprache der Graphen bilden die Knoten AB, AC und AD eine Clique der Größe drei, und die Färbung verwendet drei Farben.

Beachten Sie, dass der Zusammenhang in beide Richtungen gilt. Jede zulässige Färbung von Modell B ergibt einen gültigen Spielplan, indem Farben als Tage interpretiert werden. Jeder gültige

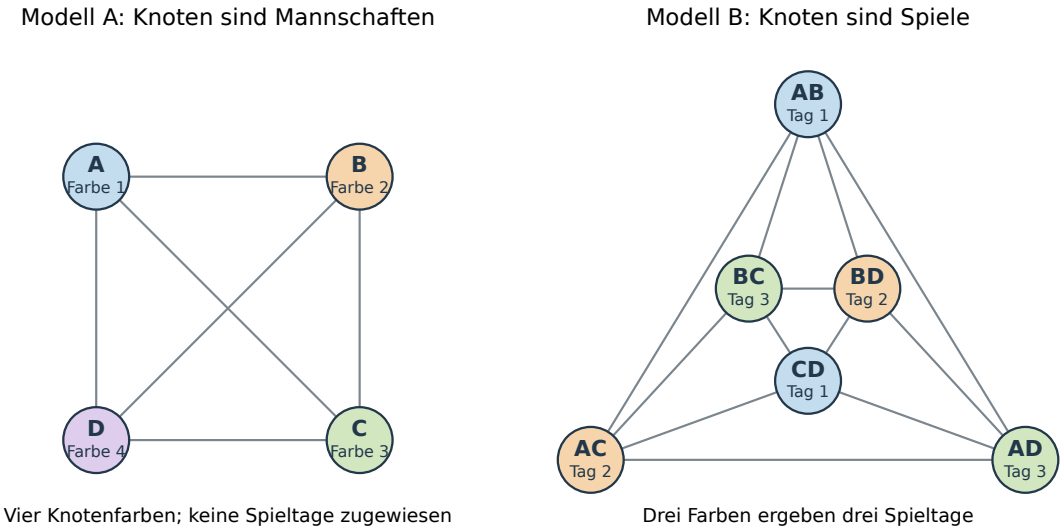


Abbildung 35: Zwei Modelle für vier Mannschaften. Modell A stellt Mannschaften als Knoten und Spiele als Kanten dar. Modell B stellt Spiele als Knoten und Konflikte als Kanten dar; seine Knotenfarben geben Tage an. Die Zahlen kennzeichnen die Farben auch ohne Farbausdruck.

Spielplan ergibt eine zulässige Färbung, indem jedem Spiel sein Tag zugewiesen wird. Die Minimierung der Farbanzahl minimiert daher tatsächlich die Anzahl der Tage. Eine beliebige zulässige Färbung liefert nur einen durchführbaren Spielplan; eine *optimale* Färbung liefert einen kürzesten Spielplan.

Hintergrund

Auch Modell A lässt sich verwenden, wenn wir ändern, was wir färben. Weisen wir jeder *Kante* eine Farbe zu und verlangen, dass Kanten, die zum selben Knoten inzident sind, unterschiedliche Farben erhalten. Das ist eine *zulässige Kantenfärbung*. Da Kanten Spiele darstellen, stehen ihre Farben nun für Tage. Modell B macht daraus ein gewöhnliches Knotenfärbungsproblem, indem die Kanten von Modell A zu Knoten werden. Der durch diese Konstruktion entstehende Graph heißt *Kantengraph* von Modell A.

Mannschaften als Knoten darzustellen war also keine nutzlose Idee. Erst die Kombination dieser Darstellung mit *Knoten*färbung verfehlte unsere Bedingung. Eine ähnliche Schwierigkeit begegnete uns bei den sieben Brücken von Königsberg: Die Wahl eines Graphen und die Wahl der Frage, die wir über diesen Graphen stellen, müssen zusammenpassen.

Für n Mannschaften besitzt Modell B $\binom{n}{2} = n(n-1)/2$ Knoten. Jede Mannschaft nimmt an $n-1$ Spielen teil, was eine untere Schranke von $n-1$ Tagen ergibt. Ist n ungerade, muss jeden Tag mindestens eine Mannschaft aussetzen, sodass höchstens $(n-1)/2$ Spiele an einem Tag stattfinden können. Teilen wir die Gesamtzahl der Spiele durch diese Tageskapazität, erhalten wir die stärkere untere Schranke n.

Diese Schranken lassen sich durch Rundenturnierpläne erreichen: Für gerades n ordnen wir die Mannschaften in zwei Reihen mit jeweils $n/2$ Positionen an und paaren Mannschaften derselben Spalte. Die Mannschaft links oben bleibt fest. Die übrigen Positionen lesen wir entlang der oberen Reihe von links nach rechts und dann entlang der unteren Reihe von rechts nach links; zwischen den Runden rückt jede Mannschaft in dieser zyklischen Liste eine Position weiter. Nach $n-1$ Runden hat jedes Paar einmal gegeneinander gespielt. Für ungerades n fügen wir eine fiktive Mannschaft hinzu; gegen sie zu spielen bedeutet einen spielfreien Tag. Das Minimum ist daher $n-1$ Tage für gerades n und n Tage für ungerades $n \geq 3$. Die obige Tabelle zeigt die drei benötigten Runden für vier Mannschaften.

Innehalten und nachdenken – Selbst erarbeiten

Was ändert sich, wenn nur ein Spielfeld verfügbar ist und pro Tag nur ein Spiel stattfinden kann? Was ändert sich bei zwei Spielfeldern und höchstens zwei Spielen pro Tag? Drückt der ursprüngliche Konfliktgraph diese zusätzlichen Einschränkungen aus?

11.2.1 Dieselbe Idee in anderen Anwendungen

Das nützliche Muster besteht darin, Objekte zu identifizieren, die eine Ressource benötigen, und dann Paare zu verbinden, die diese nicht gemeinsam nutzen dürfen. Bei der *Prüfungsplanung* sind Knoten Prüfungen, eine Kante zeigt mindestens einen gemeinsamen Studierenden an, und Farben sind Zeitfenster. Der entstehende Plan verhindert gleichzeitige Prüfungen für einzelne Studierende. Raumkapazitäten oder unterschiedlich lange Prüfungen würden zusätzliche Informationen erfordern.

In einem einfachen Modell der *Registerzuteilung* stellen Knoten Werte dar, die ein Programm hält. Eine Kante verbindet Werte, die gleichzeitig lebendig sind und getrennt gehalten werden müssen. Farben stehen für Register. Werte mit nicht überlappenden Lebensdauern können dasselbe Register wiederverwenden. Reichen die verfügbaren Register nicht aus, muss ein Compiler möglicherweise einige Werte stattdessen im Arbeitsspeicher halten.

Beide Beispiele verlangen, eine begrenzte Ressource wiederzuverwenden, ohne Konflikte zu erzeugen. Der Graph weiß nicht, was eine Prüfung, ein Spiel oder ein Register ist. Diese Interpretation liegt in unserer Verantwortung, wenn wir das Modell konstruieren und seine Lösung lesen.

11.3 Gierige Färbung

Wie würden Sie selbst einen Graphen färben? Vielleicht wählen Sie einen Knoten, geben ihm die erste Farbe, gehen dann zu einem anderen Knoten und prüfen, welche Farben noch verfügbar sind. Wenn möglich, verwenden Sie eine Farbe erneut. Eine neue Farbe führen Sie erst ein, wenn alle bisherigen durch Nachbarn blockiert sind. Das ist die Idee der *gierigen Färbung*, auch *First-Fit-Färbung* genannt.

Wir benötigen eine Reihenfolge für den Besuch der Knoten. Sobald sie feststeht, trifft Algorithmus 25 jede Entscheidung, indem er die kleinste positive Farbnummer wählt, die noch kein gefärbter Nachbar verwendet.

Algorithmus 25 Gierige Knotenfärbung (First Fit)

Eingabe: Ein einfacher ungerichteter Graph $G = (V, E)$ und eine Reihenfolge $(v_1, \ldots, v_m)$, die jeden Knoten genau einmal enthält
Ausgabe: Eine zulässige Färbung f von G

```
1:  f ← eine leere Zuordnung
2:  für i ← 1 bis m tue
3:      F ← {f(u) | u ∈ N(v_i) und u ∈ dom(f)}
4:      c ← 1
5:      solange c ∈ F tue
6:          c ← c + 1
7:      Ende solange
8:      f(v_i) ← c
9:  Ende für
10: gib zurück f
```

Hier ist F die Menge der aktuell verbotenen Farben und $\mathrm{dom}(f)$ die Menge der Knoten, denen bereits eine Farbe zugewiesen wurde. Nur *bereits gefärbte* Nachbarn tragen zu F bei. Der Algorithmus erzeugt stets eine zulässige Färbung: Wenn der spätere Endpunkt einer Kante seine Farbe erhält, vermeidet er die Farbe des früheren Endpunkts. Verwendet er aber immer möglichst wenige Farben?

11.3.1 Ein Graph, zwei Reihenfolgen

Betrachten wir den Graphen in Abbildung 36. In der oberen Reihe stehen vier Knoten $a_1, \ldots, a_4$, in der unteren vier Knoten $b_1, \ldots, b_4$. Jeder Knoten a_i ist zu jedem b_j *außer* b_i benachbart. Innerhalb der Reihen gibt es keine Kanten. Somit gilt

$$E = \{\{a_i, b_j\} \mid 1 \leq i, j \leq 4,\ i \neq j\}.$$

Dies ist ein bipartiter Graph mit Kanten, daher wissen wir bereits, dass seine chromatische Zahl zwei ist. Sehen wir uns an, was die gierige Färbung tut.

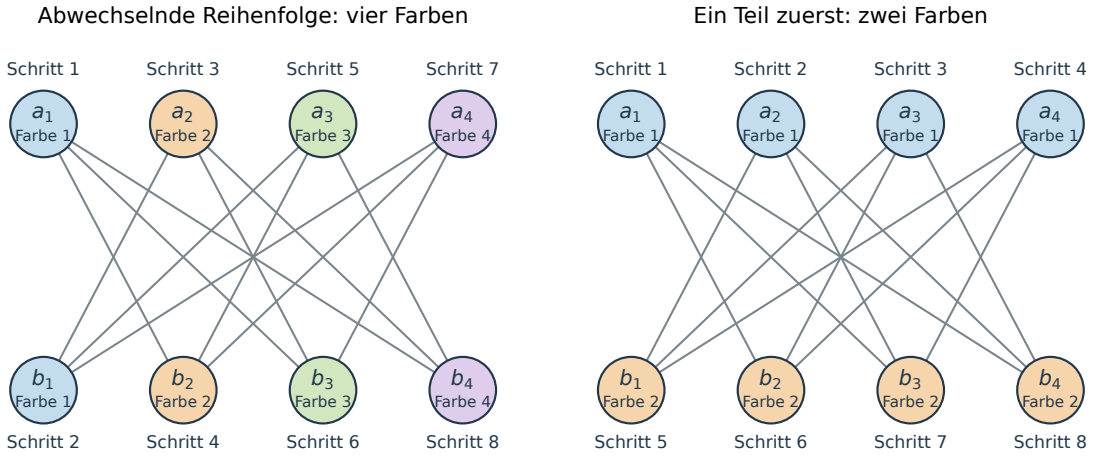


Abbildung 36: Derselbe Graph, gierig in zwei unterschiedlichen Reihenfolgen gefärbt. Das abwechselnde Besuchen entsprechender Knoten verwendet vier Farben; der Besuch einer ganzen Reihe zuerst verwendet zwei. Die Farbnummern sind ausdrücklich angegeben. Kantenkreuzungen sind keine zusätzlichen Knoten.

Nehmen wir zuerst die Reihenfolge $(a_1, b_1, a_2, b_2, a_3, b_3, a_4, b_4)$. Knoten a_1 erhält Farbe 1. Ebenso b_1, da diese beiden Knoten nicht benachbart sind. Das wirkt sparsam. Doch a_2 ist zu b_1 benachbart und benötigt daher Farbe 2. Knoten b_2 ist zu a_1, aber nicht zu a_2 benachbart und erhält ebenfalls Farbe 2.

Nun sieht a_3 die Farben 1 und 2 bei b_1 und b_2 und benötigt daher Farbe 3. Dasselbe geschieht bei b_3 durch a_1 und a_2. Schließlich sehen a_4 und b_4 jeweils alle drei bisherigen Farben und benötigen Farbe 4. Jede Entscheidung war zulässig, doch das Ergebnis verwendet doppelt so viele Farben wie nötig.

Vergleichen wir dies mit der Reihenfolge $(a_1, a_2, a_3, a_4, b_1, b_2, b_3, b_4)$. Kein Knoten der oberen Reihe besitzt einen bereits gefärbten Nachbarn, daher erhalten alle vier Farbe 1. Jeder Knoten der unteren Reihe sieht anschließend nur Farbe 1 bei seinen Nachbarn und erhält Farbe 2. Die vollständige Entscheidungsfolge lautet:

	Abwechselnde Reihenfolge			Eine Reihe zuerst		
Schritt	Knoten	Verboten	Farbe	Knoten	Verboten	Farbe
1	a_1	$\emptyset$	1	a_1	$\emptyset$	1
2	b_1	$\emptyset$	1	a_2	$\emptyset$	1
3	a_2	$\{1\}$	2	a_3	$\emptyset$	1
4	b_2	$\{1\}$	2	a_4	$\emptyset$	1
5	a_3	$\{1,2\}$	3	b_1	$\{1\}$	2
6	b_3	$\{1,2\}$	3	b_2	$\{1\}$	2
7	a_4	$\{1,2,3\}$	4	b_3	$\{1\}$	2
8	b_4	$\{1,2,3\}$	4	b_4	$\{1\}$	2

Innehalten und nachdenken – Kurzcheck

Können wir nach dem abwechselnden Durchlauf einfach zwei seiner Farbklassen zusammenlegen, um die Farbanzahl zu reduzieren? Oder müssen wir ändern, welche Knoten dieselbe Farbe haben? Prüfen Sie vor Ihrer Antwort die Kante zwischen a_1 und b_2.

Die frühe Entscheidung, a_1 und b_1 dieselbe Farbe zu geben, ist erlaubt, vermischt aber die beiden Teile des bipartiten Graphen. Der Algorithmus überdenkt frühere Entscheidungen nicht. Dadurch verliert er die Möglichkeit, den Graphen optimal zu färben. „Gierig“ beschreibt, wie ein Algorithmus Entscheidungen trifft; allein daraus folgt keine Optimalitätsgarantie.

11.3.2 Was garantiert die gierige Färbung?

Ein Knoten mit Grad d hat höchstens d verbotene Farben, wenn wir ihn erreichen. Unter den Farben $1, \dots, d+1$ ist daher mindestens eine verfügbar. Da kein Grad $\Delta(G)$ überschreitet, verwendet die gierige Färbung niemals mehr als $\Delta(G)+1$ Farben. Folglich gilt

$$\omega(G) \le \chi(G) \le \Delta(G) + 1 \le |V|$$

für einen nichtleeren einfachen Graphen. Im Beispiel mit acht Knoten ist $\Delta(G) = 3$, und die schlechte Reihenfolge erreicht die Schranke von vier. Beim Stern mit acht Blättern erlaubt die Schranke neun Farben, obwohl zwei genügen. Eine obere Schranke muss der tatsächlichen Antwort nicht nahekommen.

Es existiert immer eine Reihenfolge, die eine optimale gierige Färbung ergibt. Um die Idee zu verstehen, stellen wir uns vor, wir hätten bereits eine optimale Färbung, und listen ihre Farbklassen nacheinander auf. Knoten innerhalb einer Klasse sind nicht benachbart. Die gierige Färbung benötigt für jede Klasse höchstens eine neue Farbe und kann somit nicht mehr Farben als diese optimale Färbung verwenden. Diese Beobachtung liefert jedoch kein schnelles Verfahren, um die Reihenfolge zu *finden*: Wir haben vorausgesetzt, dass wir bereits eine optimale Färbung kennen!

Hintergrund

Die größte Anzahl von Farben, die die First-Fit-Färbung über alle Knotenreihenfolgen verwenden kann, heißt *Grundy-Zahl* $\Gamma(G)$. Somit gilt $\chi(G) \le \Gamma(G) \le \Delta(G)+1$. Unser Beispiel hat $\chi(G) = 2$ und $\Gamma(G) = 4$. Der Graph bleibt derselbe; nur die Reihenfolge ändert sich.

In der Praxis kann man zunächst Knoten hohen Grades versuchen oder wiederholt einen ungefärbten Knoten wählen, dessen gefärbte Nachbarn bereits die meisten unterschiedlichen Farben verwenden. Letztere Idee heißt *Reihenfolge nach Sättigungsgrad* und liegt DSATUR zugrunde. Solche Entscheidungen sollen stark eingeschränkte Knoten früh behandeln, doch eine gierige Wahl muss weiterhin nicht optimal sein. In unserem Beispiel sind alle Grade gleich, sodass eine Sortierung allein nach Grad jede Entscheidung der Regel zur Auflösung von Gleichständen überlassen würde.

11.4 Eine Färbung finden und das Optimum finden

Eine vorgeschlagene Färbung lässt sich leicht prüfen: Man betrachtet jede Kante und vergleicht die Farben ihrer Endpunkte. Mit Adjazenzlisten benötigt dies einschließlich der Prüfung, dass jedem Knoten eine Farbe zugewiesen wurde, $O(|V| + |E|)$ Zeit. Die *kleinste* Palette zu finden ist eine andere Aufgabe. Für allgemeine Graphen ist die Bestimmung von $\chi(G)$ NP-schwer, und die Entscheidung, ob ein Graph k-färbbar ist, ist für jedes feste $k \ge 3$ NP-vollständig. Das bedeutet nicht, dass jede Instanz schwierig ist; vollständige Graphen, bipartite Graphen und unser strukturiertes Planungsbeispiel haben wir bereits gelöst.

Ein exakter Ansatz fragt zunächst, ob eine Färbung mit k Farben existiert. Wir weisen den Knoten nacheinander Farben zu. Stehen alle k Farben im Konflikt mit bereits gefärbten Nachbarn, machen wir eine frühere Zuweisung rückgängig und versuchen eine andere Farbe. Das ist *Backtracking*. Sind alle Möglichkeiten ausgeschöpft, ist gezeigt, dass k Farben nicht ausreichen.

Beginnen wir bei einer unteren Schranke und erhöhen k, bis eine Färbung gefunden wird, erhalten wir die chromatische Zahl.

Vor dem Ausschließen unzulässiger Möglichkeiten gibt es $k^{|V|}$ Zuweisungen zu einer Palette mit k Farben; eine vollständige Suche wird daher schnell aufwendig. Backtracking kann Konflikte früh verwerfen, und Schranken können Zweige ausschließen, die die bisher beste Färbung nicht verbessern können. Diese Verbesserungen sind nützlich, beseitigen aber nicht die allgemeine Schwierigkeit.

Die gierige Färbung verfolgt den anderen Ansatz: schnell eine zulässige Antwort erhalten. Für die gegebene Knotenreihenfolge ergibt das Durchgehen der Nachbarfarben mit einer Menge, deren Mitgliedschaftstests konstante Zeit benötigen, eine Implementierung mit erwarteter Laufzeit $O(|V|+|E|)$. Die Knotenreihenfolge kann zusätzliche Arbeit erfordern. Verwendet der Algorithmus q Farben, ist q eine obere Schranke für $\chi(G)$, nicht automatisch deren Wert. Eine Clique der Größe q würde Optimalität zertifizieren; ohne passende untere Schranke kann es weiterhin eine bessere Färbung geben.

Innehalten und nachdenken – Selbst erarbeiten

Ein Planungsprogramm findet einen Fünf-Tage-Plan, während der Konfliktgraph eine Clique der Größe vier enthält. Was können Sie über den kürzestmöglichen Plan schließen? Was ändert sich, wenn ein zweiter Durchlauf einen Vier-Tage-Plan findet? Haben Sie bewiesen, dass fünf Tage nötig sind, wenn er keine Verbesserung findet?

Für eine praktische Anwendung müssen wir daher zwei Fragen auseinanderhalten. Erfüllt die Färbung die Bedingungen unseres Modells? Und haben wir einen Grund, warum weniger Farben unmöglich sind? Vor beiden steht jedoch die Frage, die uns beim Sportwettbewerb begegnet ist: Drückt das Modell tatsächlich das Problem aus, das wir lösen wollten?

Auswahl eines Algorithmus

Bestimmen Sie das Problem und seine Voraussetzungen, bevor Sie einen Algorithmus auswählen. Es seien $n = |V|$, $m = |E|$. Die folgenden Schranken setzen Adjazenzlisten, Arithmetik in konstanter Zeit und bei Bedarf binäre Heaps voraus; die Zeilen zur Hülle und zu Floyd verwenden Matrizen. Die Heap-Schranken beziehen sich auf Implementierungen, die Vorgänger speichern und nicht bei jeder Aktualisierung ganze Pfade kopieren.

Algorithmus	Problem und Voraussetzungen	Ausgabe	Zentrale Idee	Laufzeit
BFS	Wenigste Kanten; ungewichteter Graph	Distanzen, Pfade	Schichtweise erkunden	$O(n + m)$
Tarjan	Starker Zusammenhang; gerichteter Graph	SCC-Zerlegung	DFS-Indizes und Lowlinks	$O(n + m)$
Topologische Sortierung	Vorrangbeziehungen; DAG	Verträgliche totale Ordnung	Knoten mit Eingangsgrad null entfernen	$O(n + m)$
Warshall	Erreichbarkeit; gerichtete oder symmetrische Adjazenzmatrix	Hüllenmatrix	Weiteren Zwischenknoten zulassen	$O(n^3)$
Floyd	Distanzen aller Paare; keine negativen Kreise	Distanzmatrix	Gleiche Rekursion mit $\min, +$	$O(n^3)$
Bellman–Moore–Ford	Distanzen ab einer Quelle; negative Kanten erlaubt	Distanzen oder Erkennung eines erreichbaren negativen Kreises	Relaxation wiederholen	$O(nm + n)$
Dijkstra	Distanzen ab einer Quelle; nichtnegative Gewichte	Distanzen, Kürzeste-Pfade-Baum	Kleinste Distanz endgültig festlegen	$O((n + m) \log n)$
A*	Ein Ziel; nichtnegative Gewichte, konsistentes h	Optimale Route zum Ziel	Kleinstes $g + h$ endgültig festlegen	$O((n + m) \log n)$
Prim	MST; zusammenhängend, ungerichtet; beliebige reelle Gewichte	Baum minimaler Kosten	Günstigste Kante aus dem wachsenden Baum	$O(m \log n)$
Kruskal	Dasselbe MST-Problem	Baum minimaler Kosten	Kanten sortieren; Union-Find vermeidet Kreise	$O(m \log m)$
Chu–Liu/ Edmonds	Gerichteter spannender Wurzelbaum; jeder Knoten von der Wurzel erreichbar	Günstigster Wurzelbaum	Eingehende Kanten wählen; Kreise kontrahieren	$O(nm)$ bei einfacher Kontraktion
Edmonds–Karp	Maximaler Fluss; nichtnegative Kapazitäten	Fluss und Minimalschnitt-Zertifikat	BFS im Residualgraphen	$O(nm^2)$
Gierige Färbung	Knotenkonflikte; einfacher ungerichteter Graph	Zulässige Färbung, höchstens $\Delta + 1$ Farben	Erste verfügbare Farbe in gewählter Reihenfolge	$O(n + m)$ mit markierten Farben
Minimax	Endlicher alternierender Spielbaum; vollständige Information, Nullsumme	Wert bei optimalem Spiel	Abwechselnd Max/Min nach oben übertragen	$O(N)$ Baumknoten; $O(b^d)$

Für A* wird angenommen, dass die Auswertung von h konstante Zeit benötigt; das erneute Öffnen von Knoten bei einer inkonsistenten Heuristik hat eine andere Schranke. Bei Minimax zählt N die Knoten des Spielverlaufsbaums, nicht die verschiedenen Stellungen. Die gierige Färbung garantiert Zulässigkeit und eine obere Schranke, keine minimale Färbung. Ein Kürzeste-Pfade-Baum minimiert Distanzen von seiner Quelle aus; ein MST minimiert das Gesamtgewicht der ausgewählten Kanten.

Literatur

[1] László Babai. Graph Isomorphism in Quasipolynomial Time. In *Proceedings of the 48th Annual ACM Symposium on Theory of Computing (STOC)*, 2016. https://arxiv.org/abs/1512.03547

[2] Jan van den Brand, Li Chen, Rasmus Kyng, Yang P. Liu, Richard Peng, Maximilian Probst Gutenberg, Sushant Sachdeva, and Aaron Sidford. A Deterministic Almost-Linear Time Algorithm for Minimum-Cost Flow. arXiv:2309.16629, 2023. https://arxiv.org/abs/2309.16629

[3] Sergey Brin and Lawrence Page. The Anatomy of a Large-Scale Hypertextual Web Search Engine. *Computer Networks and ISDN Systems*, 30:107–117, 1998. https://snap.stanford.edu/class/cs224w-readings/Brin98Anatomy.pdf

[4] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The PageRank Citation Ranking: Bringing Order to the Web. Technical Report 1999-66, Stanford InfoLab, 1999. http://ilpubs.stanford.edu:8090/422/

[5] Ulrik Brandes. A Faster Algorithm for Betweenness Centrality. *Journal of Mathematical Sociology*, 25(2):163–177, 2001. https://www.uni-konstanz.de/algo/publications/b-fabc-01.pdf